

Contents

CHAPTER#1: PROPOSAL	4
Revision History	6
1.1 Introduction.....	7
1.2 Problem Statement	7
1.3 Proposed System.....	7
1.4 Benefits of Proposed System	7
1.5 Scope of The Project.....	8
1.6 Survey Analysis	8
Table no 1.6.1. Survey Analysis	8
1.7 Module & Submodules	10
1.7.1 Authentication.....	10
1.7.2 Profile Management.....	10
1.7.3 Medicine Management.....	10
1.7.4 Schedule Management	10
1.7.5 User Health Records	11
1.7.6 Inventory Control.....	11
1.7.7 Appointment Management.....	11
1.7.8 Journaling & Motivation.....	11
1.7.9 Patient Report Management.....	11
1.7.10 ChatBot Assistant.....	11
1.7.11 Help & Support	11
1.7.12 Setting & Preferences.....	12
1.7.13 Analytics & Data Visualization	12
1.8 Primary Actor.....	12
1.9 Tools & Technologies.....	12
1.10 System Design Approach.....	12
1.11 Process Model Used.....	13

1.12 Limitations / Constraints	13
1.13 References	13
CHAPTER#2: ANALYSIS	14
Revision History	16
2.1 Introduction	17
2.1.1 Purpose	17
2.1.2 Scope	17
2.1.3 Definitions, Acronyms and Abbreviation	18
2.1.4 Overview	19
2.2 Functional Requirements	19
2.2.1 Authentication	19
2.2.2 Profile Management	21
2.2.3 Medicine Management	23
2.2.4 Schedule Management	24
2.2.5 User Health Records	24
2.2.6 Inventory Control	25
2.2.7 Appointment Management	26
2.2.8 Journaling & Motivation	26
2.2.9 Report Management	27
2.2.10 ChatBot Assistant	27
2.2.11 Help & Support	28
2.2.12 Settings & Preferences	28
2.2.13 Analytics & Data Visualization	29
2.3 Non-Functional Requirements	29
2.3.1 Performance:	29
2.3.2 Security:	29
2.3.3 Usability:	29
2.3.4 Reliability:	29
2.3.5 Availability:	29

2.3.6 Scalability:	30
2.3.7 Maintainability:.....	30
2.3.8 Compatibility:	30
2.3.9 Data Integrity:	30
2.3.10 Backup and Recovery:	30
2.4 Use Case Diagram.....	33
2.5 FULLY DRESSED USE CASES	38
CHAPTER # 3: DESIGN.....	115
3.1 System Sequence Diagram.....	118
3.2 Firebase (Firestore)	145
3.3 Hive.....	146
3.4 Domain Model	150

CHAPTER#1: PROPOSAL

PROJECT PROPOSAL

For

Personal Medicine Reminder

Version 1.1

Prepared By

Atif Islam

&

Muhammad Awais Ali

27th Nov, 2025

Revision History

Version	Description	Author	Date
1.0	This is the Project Proposal for Personal Medicine Reminder App	Atif Islam & Muhammad Awais Ali	6 th October 2025
1.1	This is the Project Proposal for Personal Medicine Reminder App	Atif Islam & Muhammad Awais Ali	27 th Nov,2025

1.1 Introduction

Our client is a doctor working in a hospital where He/She often sees patients forget their medicines or struggle to manage multiple prescriptions. Since she does not have her own clinic, He/She deals with patients in the hospital environment and wants an easy solution that can help them take their medicines on time. To address this, we are developing an **Android mobile application** using **Flutter**, which will provide reminders, stock alerts, and health record management. A mobile app is chosen because smartphones are common and portable, making it easier for patients and caregivers to manage medicines anytime and anywhere.

1.2 Problem Statement

Right now, the client (doctor) does not have any special app or system for reminding patients about their medicines. Patients usually rely on paper prescriptions, memory, or basic phone alarms, which is not enough. Because of this, many patients forget their doses, get confused when using different medicines, or do not notice when their medicines are finished. Health records are also not managed properly, since most information is kept on paper and can be lost or forgotten. These problems cause patients to miss medicines, recover slowly, and create extra work for the doctor and caregivers.

1.3 Proposed System

The proposed system is an **Android mobile application** that will help patients take their medicines on time and manage their health more easily. The app will allow users to add their medicines, set reminders, and get notifications for every dose. It will also track medicine stock, give alerts for low quantity and store basic health information like allergies and prescriptions. Unlike simple alarm apps, this system will provide reports of medicine usage, show missed doses, and offer simple health tips to keep patients motivated. It will allow users to upload their medical reports for ease and also allows user to add doctor appointment dates and reminders. The aim is to give patients an easy tool that makes medicine management simple, reliable, and effective.

1.4 Benefits of Proposed System

- Reminds patients to take their medicines on time.
- Reduces chances of forgetting or taking the wrong dose.
- Helps manage many medicines at the same time.
- Gives alerts when medicine stock is low.
- Keeps health records safe and easy to access.
- Provides simple reports about medicine usage and missed doses.

- Easy design for elderly people and busy individuals.
- Gives users quick help through a built-in chatbot.
- Users can upload and store health-related reports within the application.
- Users can schedule, edit, and set reminders for upcoming appointments.

1.5 Scope of The Project

The Personal Medicine Reminder Application will provide all the basic features needed to help patients manage medicines and health records in one place. The system will allow users to add medicines, set reminders, get alerts for missed doses, and track medicine stock. It will also store important health details like allergies and prescriptions, and generate simple reports to check medicine usage.

This app is mainly designed for elderly people, patients with long-term illnesses, and busy individuals who often forget their medicines.

In the future, the system can be improved with advanced features like wearable device integration, cloud backup, or connections with doctors and pharmacies. For now, the focus is on building a reliable, user-friendly Android app that makes medicine management simple and effective.

1.6 Survey Analysis

Table no 1.6.1. Survey Analysis

	My Therapy	Mewdicate	Mediqation	Pill Reminder	Cute Pill	Proposed System
System Features ↓						
User- Authentication	✓	✗	✗	✗	✗	✓
Medicine Management	✓	✓	✓	✓	✓	✓

Profile Management	✓	✗	✗	✗	✗	✓
Schedule Management	✓	✓	✓	✓	✓	✓
User Health Management	✗	✗	✗	✗	✗	✓
Inventory Control	✓	✗	✗	✗	✓	✓
Analytics & Data Visualization	✓	✗	✓	✗	✗	✓
Setting & Preferences	✓	✓	✓	✓	✓	✓
Appointment Management	✓	✗	✗	✗	✗	✓
Journaling & Motivation	✗	✗	✗	✗	✗	✓
Patient Report Management	✗	✗	✗	✗	✗	✓

ChatBot Assistant	✓	✗	✗	✗	✗	✓
Help & Support	✗	✗	✗	✗	✗	✓

✓ (Check Mark): Indicates that the feature is available in the Application.

X: Indicates that the feature is not available in the Application.

1.7 Module & Submodules

1.7.1 Authentication

- SignIn(Register)
- Login
- Logout
- Forget Password
- Change Password
- Social Login

1.7.2 Profile Management

- View Profile
- Update Profile
- Basic Health Information
- Upload Photo

1.7.3 Medicine Management

- Add Medicine
- Update/Edit Medicine
- Medicine Reminder
- Remove/Disable Medicine Alert
- Categorize Medicines (Tablets/Injections/etc.)

1.7.4 Schedule Management

- Calendar View
- Daily Schedule view
- Date Schedule view
- Date Range Limitation

1.7.5 User Health Records

- Allergies list
- Current Medications
- Past Medications
- Restraint
- Chronic Illness

1.7.6 Inventory Control

- View Current Stock
- Update Current Stock
- Stock Alerts(Low Quantity)

1.7.7 Appointment Management

- Add Appointment
- Edit Appointment
- Appointment Status
- Appointment Reminder
- Visit Notes
- Appointment History

1.7.8 Journaling & Motivation

- Mood & Notes Journal
- User Challenges
- Visual Streak Tracker

1.7.9 Patient Report Management

- Add/Upload Report
- Search Report
- View Report

1.7.10 ChatBot Assistant

- Chat Interface
- Safety Filters & Rules

1.7.11 Help & Support

- User Guide
- Contact/Feedback Form
- About Section

1.7.12 Setting & Preferences

- App Theme (Light/Dark)
- Configure Notification Settings
- Time Zone Detection

1.7.13 Analytics & Data Visualization

- Medicine Consumption Reports
- Missed Doses Report
- Monthly Summary(Charts)

1.8 Primary Actor

- User(Patient)

1.9 Tools & Technologies

- **Front-End Tools**
 - Flutter (Dart)
 - Material Design Widgets
 - Flutter GetX / Provider (for State Management)
 - Dio Package (for API Integration)
 - Responsive UI with Media Query and Flexbox Layouts Flutter
- **Back-End Tools**
 - Firebase Authentication
 - Hive Database/sqlite
 - RESTful API
 - Firestore
- **Deployment & Testing**
 - **Testing Environment:** Local emulator/ physical devices(Android)
- **Technique**
 - **Cross-Platform Mobile & Web Application**
(Single codebase developed using Flutter for Android, Web, and potentially iOS)

1.10 System Design Approach

- We will use MVVM(Model-View-View Model) approach for better performance of our project.

- Unified Model Language(UML)

1.11 Process Model Used

- RUP (Rational Unified Process Model)

1.12 Limitations / Constraints

- The application requires a stable internet connection for services using Firebase (authentication, cloud backup, real-time sync). Core reminder functions work offline with locally stored data (after initial login).
- Dependence on third-party services such as Firebase and Gemini API means that provider downtime, data limits, or policy changes can effect app functionality.
- The system is optimized primarily for Android iOS and desktop versions are planned for later phases.
- The current version provides self-management features only it does not directly connect to doctors or pharmacies.
- Data security measures (authentication, encryption, access control) are in place, but ultimate responsibility for keeping credentials safe rests with the user.

1.13 References

- Google (2024). Firebase Official Documentation. Retrieved from <https://firebase.google.com/docs>.
- Flutter Team. (2024). Flutter Documentation- Build Apps for Any Screen. Retrieved from <https://docs.flutter.dev/>.
- Medium. (2023). State Management in Flutter: Provider vs GetX Explained Simply. Retrieved from <https://medium.com/flutter-community/>.
- ResearchGate. (2023). Mobile Health Applications for Medication Adherence: A Systematic Review. Retrieved from <https://www.researchgate.net/>.

CHAPTER#2: ANALYSIS

SOFTWARE REQUIREMENT SPECIFICATION

For

Personal Medicine Reminder

Version 1.0

Prepared By

Atif Islam

&

Muhammad Awais Ali

27th Nov, 2025

Revision History

Version	Description	Author	Date
1.0	This cover the major SRS documents	Atif Islam & Muhammad Awais Ali	27 th Nov,2025

2.1 Introduction

The Personal Medicine Reminder Application (PMRA) is a mobile-based health management system designed to assist patients in organizing their daily medication intake, appointments, and health information. The app addresses a common healthcare problem: forgetting to take medicines on time or managing multiple prescriptions incorrectly.

Developed using Flutter for cross-platform compatibility and integrated with Firebase (for storage, authentication) and local databases (for offline use), this system allows users to receive timely medicine reminders, manage stock levels, view health reports, and maintain essential medical records.

The application targets patients, elderly individuals, and people with chronic conditions, while also supporting caregivers and doctors who help monitor treatment adherence. Its intuitive interface, smart reminders, and data organization aim to improve medication discipline, health awareness, and patient independence.

2.1.1 Purpose

The main purpose of the Personal Medicine Reminder Application is to create an intelligent, userfriendly platform that automates medicine management and reminder functions while maintaining key health information securely.

This system will:

- Send timely notifications reminding users to take prescribed medicines.
- Enable multi-medicine management for users handling multiple prescriptions.
- Help users track medicine stock and receive alerts when supplies run low.
- Provide a daily, weekly, and monthly schedule overview for medicines and appointments.
- Store essential health details such as chronic conditions, allergies, and previous prescriptions.
- Offer report generation and visualization for missed doses and medicine trends.
- Allow profile customization, including personal information and health data.
- Support appointment scheduling, visit notes, and motivation journaling.
- Include a ChatBot Assistant for user guidance and a Help & Support section for troubleshooting.

2.1.2 Scope

The scope of the Personal Medicine Reminder Application includes all essential features needed for effective personal medicine and health management within a single mobile platform. It focuses on simplifying daily medication routines, organizing health information, and ensuring users never miss their prescribed doses through timely reminders.

This application is primarily designed for patients who manage multiple or chronic medications and need assistance in keeping track of their schedules. It also supports elderly users who require simple, clear, and visual reminders to help them take their medicines on time and maintain a consistent health routine.

The functional scope of the system includes the following main features:

- Authentication Module
- Profile Management
- Medicine Management
- Schedule Management
- User Health Records
- Inventory Control
- Analytics & Data Visualization
- Settings & Preferences
- Appointment Management
- Journaling & Motivation
- Report Management
- ChatBot Assistant
- Help & Support

2.1.3 Definitions, Acronyms and Abbreviation

Table 2.1.3.1 Definition, Acronyms and Abbreviation

Term/Abbreviation	Definition
GUI	Graphical User Interface
RUP	Rational Unified Process
UML	Unified Modeling Language
API	Application Programming Interface
Firebase	Backend as a Service platform for authentication, cloud storage, and data handling
AI	Artificial Intelligence (used in ChatBot Assistant)
DB	Database (used for local medicine and schedule storage)
MVVM	Model-view-viewmodel

2.1.4 Overview

This document describes the requirements for the Personal Medicine Reminder Application (PMRA). The requirements were gathered through consultations with healthcare professionals and patient feedback. The system aims to automate medicine scheduling, reminders, and health record management, ensuring accuracy, reliability, and convenience for users. It focuses on improving patient adherence, reducing missed doses, and providing an organized approach to personal healthcare management.

2.2 Functional Requirements

2.2.1 Authentication

2.2.1.1 Process SignIn

SRS-1	The system shall allow new users to register by providing required personal information such as full name, email address, password and confirm-password.
SRS-2	The system shall enforce the creation of a unique user ID or email during the registration process to ensure account uniqueness.
SRS-3	The system shall require users to create a password that meets defined security requirements (e.g. minimum 8 characters, at least one uppercase letter and one number).
SRS-4	The system shall validate that all mandatory fields are completed before allowing registration submission.
SRS-5	The system shall send a verification email containing a clickable link upon successful registration.
SRS-6	The system shall prevent duplicate accounts using the same email address.
SRS-7	The system shall redirect the user to the login screen after successful account creation.

2.2.1.2 Process Login

SRS-8	The system shall allow registered users to log in by providing a valid email address and password.
SRS-9	The system shall validate user credentials against stored authentication data before granting access.
SRS-10	The system shall redirect authenticated users to their respective home or dashboard screens upon successful login.

SRS-11	The system shall display an appropriate error message if login credentials are invalid (active internet connection is required for initial authentication and cloud sync).
---------------	--

2.2.1.3 Process Logout

SRS-12	The system shall allow users to securely log out using the logout button located at the bottom of the Profile screen.
SRS-13	The system shall terminate all active user sessions upon logout to prevent unauthorized access.
SRS-14	The system shall redirect users to the login screen after a successful logout.

2.2.1.4 Forget Password

SRS-15	The system shall allow users to request a password reset using their registered email address.
SRS-16	The system shall send a password reset link to the user's registered email.
SRS-17	The system shall verify the reset link and allow users to create a new password.
SRS-18	The system shall update and securely store the new password once it is successfully reset.

2.2.1.5 Change Password

SRS-19	The system shall allow logged-in users to change their password through the profile section.
SRS-20	The system shall require users to enter their current password before setting a new one.
SRS-21	The system shall validate that the new password meets defined security requirements (e.g. minimum 8 characters, one uppercase letter and one number).
SRS-22	The system shall confirm that the new password and confirm password fields match before submission.
SRS-23	The system shall display an appropriate success message after the password is changed successfully.
SRS-24	The system shall disable the change password option for users authenticated via Google Login, as their credentials are managed externally by Google.

2.2.1.6 Process SocialLogin

SRS-25	The system shall allow users to sign in using their Google account credentials.
SRS-26	The system shall retrieve basic user information such as full name and email address from Google.
SRS-27	The system shall automatically create a new user account if the Google account does not already exist in the system.
SRS-28	The system shall log in the user directly if their Google account is already registered.
SRS-29	The system shall redirect the user to the home screen upon successful Google authentication.
SRS-30	The system shall not allow users authenticated via Google to change passwords within the app, as credentials are managed by Google.

2.2.2 Profile Management

2.2.2.1 View Profile

SRS-31	The system shall display the user's full name and registered email address.
SRS-32	The system shall display the user's blood group.
SRS-33	The system shall display the user's weight and height
SRS-34	The system shall automatically calculate and display the user's BMI based on weight and height.
SRS-35	The system shall display the user's gender.
SRS-36	The system shall display the user's date of birth (DOB).
SRS-37	The system shall display the profile photo uploaded in the Update Profile screen.

2.2.2.2 Update Profile

SRS-38	The system shall allow users to update their full name and email address.
SRS-39	The system shall allow users to update their blood group.
SRS-40	The system shall allow users to update their weight and height.
SRS-41	The system shall automatically recalculate BMI after weight or height changes.
SRS-42	The system shall allow users to update their gender.
SRS-43	The system shall allow users to update their date of birth (DOB).

SRS-44	The system shall save all updated information securely in Firebase under the user's account.
SRS-45	The system shall provide a separate Change Password button at the bottom of the screen, which navigates users to the Change Password screen.
SRS-46	The system shall provide a Save button to save all profile changes to Firebase securely.
SRS-47	The system shall display a confirmation message after successfully saving profile updates.
SRS-48	The system shall allow users to upload or replace their profile photo, which will be reflected in the View Profile screen.

2.2.2.3 Manage Basic Health Information

SRS-49	The system shall allow users to enter their height in centimeters.
SRS-50	The system shall allow users to enter their weight in kilograms.
SRS-51	The system shall allow users to select their blood group from a predefined list.
SRS-52	The system shall automatically calculate and display BMI based on height and weight.
SRS-53	The system shall validate all fields before saving to ensure data accuracy.
SRS-54	The system shall securely store all basic health information in Firebase when the Save button is pressed.
SRS-55	The system shall display a confirmation message after successfully saving basic health information.

2.2.2.4 Upload Profile Photo

SRS-56	The system shall allow users to select a photo from their device gallery.
SRS-57	The system shall validate that the uploaded file is a supported image format (e.g. JPG, PNG).
SRS-58	The system shall resize or compress the image if needed to optimize storage and performance.
SRS-59	The system shall upload the photo securely to Firebase Storage and link it to the user's account.
SRS-60	The system shall update the profile view to display the newly uploaded photo immediately.

SRS-61	The system shall Provide an option to remove or replace the existing profile photo.
---------------	---

2.2.3 Medicine Management

2.2.3.1 Add Medicine

SRS-62	The system shall allow users to add a new medicine by providing required details such as name, dosage, and frequency.
SRS-63	The system shall let users set medicine reminders with multiple notification per dose.
SRS-64	The system shall support complex schedules, including specific days of the week, interval-based (every X days), and cyclic schedules (X days on, Y days off).
SRS-65	The system shall allow users to enter an optional initial stock quantity for each medicine.
SRS-66	The system shall validate that all required fields are completed before adding a new medicine.
SRS-67	The system shall save the new medicine information securely in local storage.
SRS-68	The system shall display a confirmation message after successfully adding a new medicine.

2.2.3.2 Update Medicine Details

SRS-69	The system shall allow users to edit the name, category, dosage.
SRS-70	The system shall allow users to update reminder schedules for each medicine.
SRS-71	The system shall validate all required fields before saving updates.
SRS-72	The system shall save updated medicine information securely in local storage.
SRS-73	The system shall display a confirmation message after successfully updating the medicine.

2.2.3.3 Set Medicine Reminder

SRS-74	The system shall send notifications for each medicine according to the defined reminder schedule.
SRS-75	The system shall allow multiple notifications per dose (e.g. 5 minutes before, at exact time, and 2 after).
SRS-76	The system shall provide action buttons on notifications: “Take”, “Missed”, and “Snooze”.

SRS-77	The system shall update the status of each dose based on user interaction with notifications.
SRS-78	The system shall save all reminder schedules and statuses locally.

2.2.3.4 Disable Medicine Alert

SRS-79	The system shall allow users to disable notifications for a selected medicine.
SRS-80	The system shall allow users to remove a medicine from the alert list while keeping the medicine in inventory.
SRS-81	The system shall update the reminder system immediately after any changes.

2.2.3.5 Categorize Medicine

SRS-82	The system shall allow users to select a category for each medicine (e.g. Tablet, Syrup, Injection, Insulin).
SRS-83	The system shall allow users to create a custom category if the predefined options do not match.
SRS-84	The system shall save category information locally with the medicine data.

2.2.4 Schedule Management

2.2.4.1 View Calendar

SRS-85	The system shall display a calendar view highlighting dates with scheduled medicines or appointments.
---------------	---

2.2.4.2 View Daily Schedule

SRS-86	The system shall display all medicines and appointments for the current day.
---------------	--

2.2.4.3 View Date Schedule

SRS-87	The system shall allow users to view details of medicines and appointments for a specific date.
---------------	---

2.2.4.4 Apply Date Range Limitation

SRS-88	The system shall display only 15 days of past data and 15 days of future data in the calendar relative to the current date.
---------------	---

2.2.5 User Health Records

2.2.5.1 Add Allergies List

SRS-89	The system shall allow users to add, view, and update a list of allergies.
---------------	--

SRS-90	The system shall validate allergy entries before saving.
---------------	--

2.2.5.2 View Current Medication

SRS-91	The system shall display all medicines the user is currently taking.
SRS-92	The system shall allow users to mark medicines as active or completed.

2.2.5.3 View Past Medication

SRS-93	The system shall display medicines previously taken by the user.
SRS-94	The system shall update past medications automatically when a medicine is removed or marked completed.

2.2.5.4 Add Restraints

SRS-95	The system shall allow users to record any medication restrictions or dietary restraints.
---------------	---

2.2.5.5 Add Chronic Illnesses

SRS-96	The system shall allow users to add, view, and update chronic illnesses.
---------------	--

2.2.6 Inventory Control

2.2.6.1 View Current Stock

SRS-97	The system shall display a list of all medicines with their current stock quantities.
SRS-98	The system shall automatically display medicines with stock ≤ 5 at the top of the list.

2.2.6.2 Update Current Stock

SRS-99	The system shall allow users to increase or decrease stock quantities for a selected medicine.
SRS-100	The system shall save the updated stock quantity locally.
SRS-101	The system shall display a confirmation message after successfully updating stock.

2.2.6.3 Generate Stock Alerts

SRS-102	The system shall trigger a notification when a medicine's stock reaches each threshold.
SRS-103	The system shall display low-stock medicines in a dedicated "Low Stock" section for quick reference.
SRS-104	The system shall update stock alerts immediately after stock changes.

2.2.7 Appointment Management

2.2.7.1 Add Appointment

SRS-105	The system shall allow users to add a new appointment with date, time, doctor/clinic name, and category.
SRS-106	The system shall allow users to set a reminder for the appointment.

2.2.7.2 Update Appointment

SRS-107	The system shall allow users to edit the appointment date, time, category, or doctor/clinic name.
SRS-108	The system shall allow users to update visit notes or reminder settings.

2.2.7.3 Manage Appointment categories/Status

SRS-109	The system shall categorize appointments as “Upcoming” or “Completed” based on the date and time.
SRS-110	The system shall display upcoming appointments at the top for quick access.
SRS-111	The system shall allow users to manually mark an appointment as completed.

2.2.7.4 Set Appointment Reminder

SRS-112	The system shall send a notification at a user-specified time before the appointment.
SRS-113	The system shall allow recurring reminders if enabled by the user.

2.2.7.5 Add Visit Notes

SRS-114	The system shall allow users to add visit notes for an appointment
----------------	--

2.2.7.6 View Appointment History

SRS-115	The system shall display a history of completed appointments.
SRS-116	The system shall allow users to filter or search past appointments by date or category.

2.2.8 Journaling & Motivation

2.2.8.1 Record Mood & Notes Journal

SRS-117	The system shall allow users to add daily mood entries using predefined mood options (e.g. happy, sad, neutral).
SRS-118	The system shall allow users to add optional text notes along with mood entries.
SRS-119	The system shall allow users to update mood entries and notes.

2.2.8.2 Attempt User Challenges

SRS-120	The system shall utilize the Gemini API to generate personalized daily health challenges based on the user's current mood, chronic illnesses, and adherence history.
SRS-121	The system shall provide a static fallback list of standard challenges (e.g. "Drink Water") to ensure functionality when the device is offline or the API is unreachable.
SRS-122	The system shall make all challenges optional, ensuring users are not penalized for skipping them.

2.2.8.3 Display Visual Streak Tracker

SRS-123	The system shall calculate a "Streak" based on consecutive days where the user achieves 100% medication adherence
SRS-124	The system shall visually represent the streak using a prominent counter
SRS-125	The system shall reset the streak counter to zero if a user fails to log their medication for a full calendar day.

2.2.9 Report Management

2.2.9.1 Add/Upload Report

SRS-126	The system shall allow users to add new reports with title and date.
SRS-127	The system shall save reports securely in device storage.

2.2.9.2 Search Report

SRS-128	The system shall allow users to search reports by title, date, or type.
SRS-129	The system shall display search results in a list.

2.2.9.3 View Report

SRS-130	The system shall allow users to view reports within the app using a built-in viewer.
----------------	--

2.2.10 ChatBot Assistant

2.2.10.1 Access Chat Interface

SRS-131	The system shall allow users to open a chat interface to interact with the assistant.
SRS-132	The system shall display messages from both user and chatbot clearly.

2.2.10.2 Apply Safety Filters & Rules

SRS-133	The system shall implement safety filters to block inappropriate or harmful content.
SRS-134	The system shall update safety rules on the backend without requiring an app update.

2.2.11 Help & Support

2.2.11.1 View User Guide

SRS-135	The system shall provide a user guide explaining app features and navigation.
----------------	---

2.2.11.2 Submit Contact/Feedback Form

SRS-136	The system shall allow users to send feedback by launching their default email client.
----------------	--

2.2.11.3 View About Section

SRS-137	The system shall provide an About section with app version, developer names, and contact info.
----------------	--

2.2.12 Settings & Preferences

2.2.12.1 Change App Theme

SRS-138	The system shall allow users to toggle between light and dark theme.
SRS-139	The system shall save the selected theme preference locally for future sessions.

2.2.12.2 Configure Notification Settings

SRS-140	The system shall allow users to choose the appearance of notifications (e.g. large alert or standard).
SRS-141	The system shall allow users to set recurring reminders with a specified time and frequency.
SRS-142	The system shall allow users to select custom sounds for notifications.

2.2.12.3 Detect & Adjust Time Zone Automatically

SRS-143	The system shall detect the device's current time zone automatically.
SRS-144	The system shall adjust reminder notifications based on the detected time zone.

2.2.13 Analytics & Data Visualization

2.2.13.1 View Medicine Consumption Reports

SRS-145	The system shall display the number of doses taken for each medicine.
SRS-146	The system shall allow filtering by medicine or time period (daily/weekly/monthly).
SRS-147	The system shall highlight medicines with irregular intake.

2.2.13.2 View Missed Doses Reports

SRS-148	The system shall list medicines with missed doses and corresponding dates/times.
SRS-149	The system shall provide a total count of missed doses per period.

2.2.13.3 Display Monthly Summary

SRS-150	The system shall generate monthly charts for medicine intake and missed doses.
----------------	--

2.3 Non-Functional Requirements

2.3.1 Performance:

The system should handle multiple user operations simultaneously without lag and load each screen within 3 seconds on standard mobile devices.

2.3.2 Security:

All user credentials, personal health data, and medical records must be securely encrypted and protected from unauthorized access.

2.3.3 Usability:

The interface should be simple, accessible, and intuitive, allowing users of all age groups—especially elderly patients—to navigate easily.

2.3.4 Reliability:

The application must ensure accurate reminder notifications and maintain consistent local and cloud data synchronization.

2.3.5 Availability:

The app should remain operational 24/7, ensuring timely delivery of medicine reminders even in offline mode.

2.3.6 Scalability:

The system should support an increasing number of users and patients without performance degradation and allow future integration with additional features such as wearable devices, pharmacy systems, or telemedicine modules.

2.3.7 Maintainability:

The codebase should follow modular design principles and be well-documented to simplify debugging, updates, and new feature enhancements.

2.3.8 Compatibility:

The application should function smoothly across Android devices and support various screen sizes and orientations.

2.3.9 Data Integrity:

All health and medicine data must remain consistent, accurate, and error-free during updates, reminders, and offline synchronization.

2.3.10 Backup and Recovery:

The system should perform regular cloud backups and enable data recovery in case of device loss, corruption, or system failure.

USE CASE DIAGRAM
FOR
Personal Medicine Reminder
Version 1.0

Prepared By

Atif Islam

&

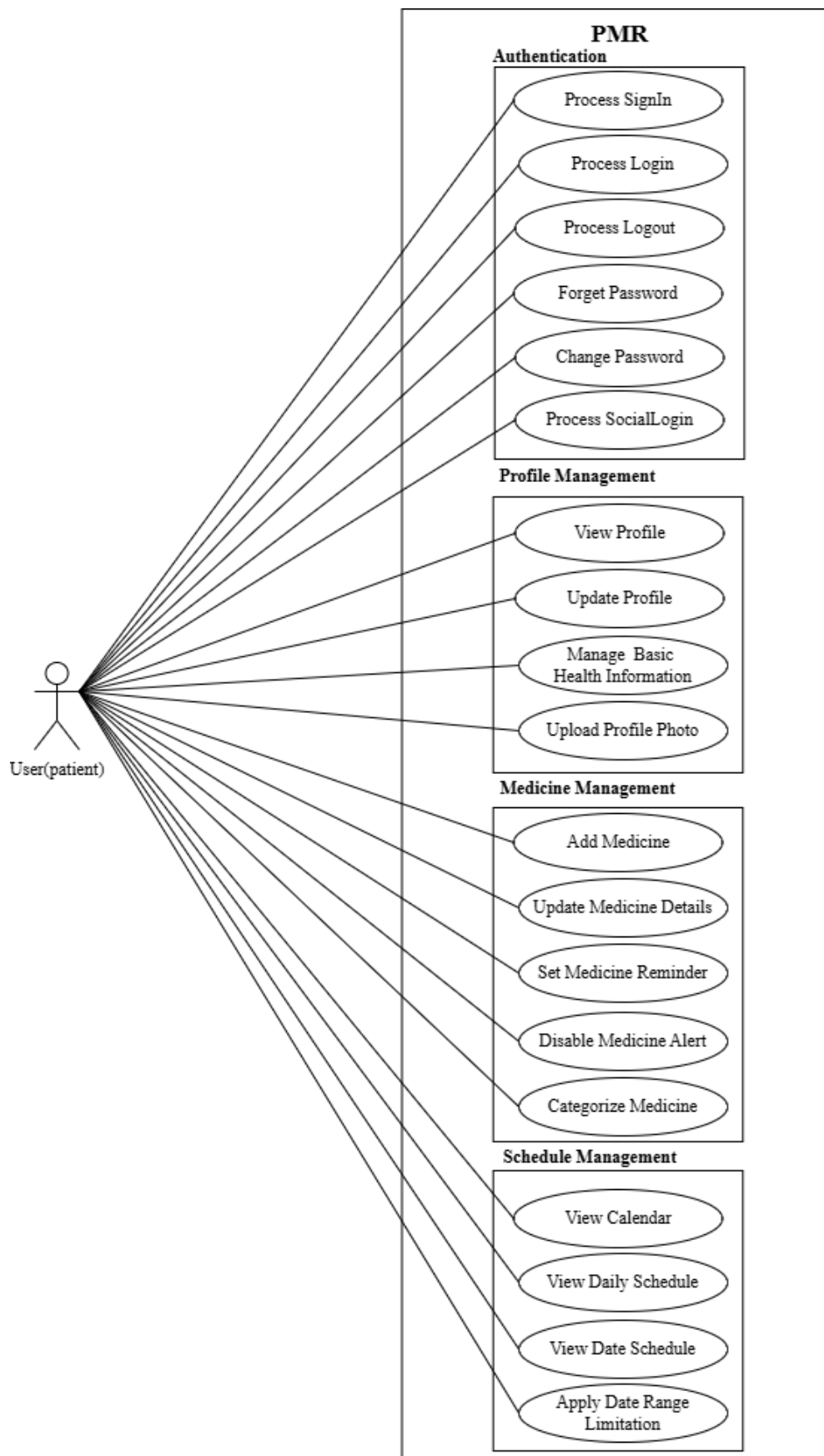
Muhammad Awais Ali

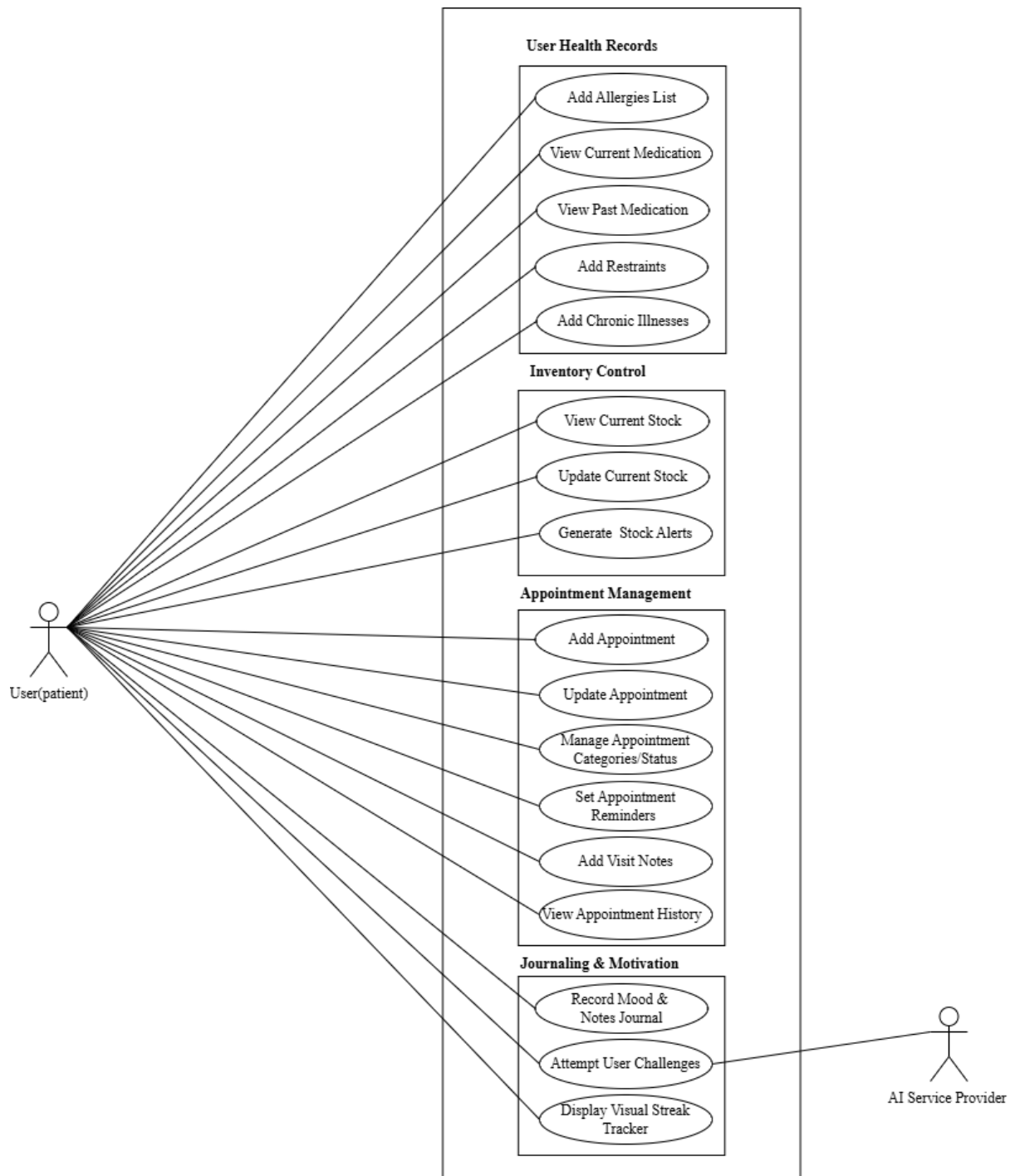
27th Nov, 2025

Revision History

Version	Description	Author	Date
1.0	This contains the Use Case Diagram for Personal Medicine Reminder App	Atif Islam & Muhammad Awais Ali	27 th Nov,2025

2.4 Use Case Diagram







FULLY DRESSED USE CASES

FOR

Personal Medicine Reminder

Version 1.0

Prepared By

Atif Islam

&

Muhammad Awais Ali

27th Nov, 2025

Revision History

Version	Description	Author	Date
1.0	This contains the fully dressed use cases for Personal Medicine Reminder App	Atif Islam & Muhammad Awais Ali	27 th Nov,2025

2.5 FULLY DRESSED USE CASES

Use Case ID: UC-01

Use Case Name: Process SignIn

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a new user creates an account by providing personal details and setting up security credentials.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to create a secure account to access app features.
System: Needs to ensure data uniqueness and security.

Pre-Conditions:

1. System must be in running state.
2. User must not be currently logged in.

Main Success Scenario:

	User Action		System Response
1	User clicks on the Register or Sign Up button.		
		2	System displays the registration form requiring Full Name, Email, Password, and Confirm password.
3	User enters full name and email address.		
4	User enters a password and confirms it.		
5	User clicks the Submit or Register button.		
		6	System validates that all mandatory fields are completed.
		7	System validates that the password meets security requirements (8 chars, 1 uppercase, 1 number).
		8	System checks if the email address is unique and not already registered.

		9	System creates the new user account in the database.
		10	System sends a verification email with a clickable link to the user.
		11	System redirects the user to the Login screen.

Post-Conditions: A new account is created, and the user is positioned at the Login screen awaiting email verification.

Extension Points:

6a. Missing Required Fields:

- System highlights missing fields and prevents submission.
- User is prompted to complete all fields.

7a. Weak Password:

- System displays an error message “indicating password requirements” (min 8 chars, uppercase, number).
- User re-enters a valid password.

8a. Duplicate Email/Account Exists:

- System detects the email is already in use.
- System displays an error message “(Account already exists) and prevents duplicate creation”.

Priority: HIGH

Frequency: Less frequent in use.

Cross-Reference: SRS-1, SRS-2, SRS-3, SRS-4, SRS-5, SRS-6, SRS-7

Use Case ID: UC-02

Use Case Name: Process Login

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a registered user authenticates into the system using their credentials to access their personal dashboard.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to access their personal health data securely.

System: Needs to prevent unauthorized access to private medical records.

Pre-Conditions:

1. System must be in running state.
2. User must have a valid, registered account.
3. User is on the Login screen.

Main Success Scenario:

	User Action		System Response
1	User navigates to the login screen.		
		2	System displays the login form requiring email and password.
3	User enters their registered email address and password.		
4	User clicks the Login button.		
		5	System validates the entered credentials against stored authentication data.
		6	System confirms the credentials are valid.
		7	System redirects the user to the Home/Dashboard screen.

Post-Conditions: The user is successfully authenticated, an active session is started, and the user lands on the dashboard.

Extension Points:

3a. Empty Fields:

- System detects missing email or password.
- System highlights the empty fields and prompts the user to fill them.

5a. Invalid Credentials:

- System checks credentials and finds a mismatch (wrong password or email).
- System displays an appropriate error message (e.g., Invalid Email or Password).
- User is prompted to re-enter credentials.

5b. Network Error:

- System cannot connect to the authentication server.
- System displays a “Connection Error” message and asks the user to check their internet.

Priority: HIGH

Frequency: High frequent in use.

Cross-Reference: SRS-8, SRS-9, SRS-10, SRS-11

Use Case ID: UC-03

Use Case Name:	Process Logout
-----------------------	----------------

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description:	This use case describes how an authenticated user securely ends their session via the profile section.
------------------------------	--

Primary Actors: <Patient>

Stakeholders & Interest:	User: Wants to securely exit the app. System: Needs to terminate the session to prevent unauthorized access.
-------------------------------------	--

Pre-Conditions:

1. System must be in running state.
2. User must be currently logged in.
3. User must be on the Profile Management screen.

Main Success Scenario:

	User Action		System Response
1	User navigates to the profile section.		
		2	System displays the user profile details.
3	User scrolls to the bottom of the screen.		
4	User clicks on the Logout button.		
		5	System receives the logout request.
		6	System terminates all active user sessions immediately.
		7	System securely clears user authentication tokens from local storage.
		8	System redirects the user to the Login screen.

Post-Conditions: The user session is terminated, and the user is returned to the Login interface.

Extension Points:**5a. Network Failure:**

- System fails to reach the server.
- System performs a force logout locally to ensure device security.
- System redirects to Login.

Priority: HIGH**Frequency:** Most frequent in use.**Cross-Reference:** SRS-12, SRS-13, SRS-14**Use Case ID:** UC-04**Use Case Name:** Forget Password**Use Case Prepared by:** Atif Islam **Use Case Prepared On:** 26th Nov, 2025**Use Case Updated by:** M.Awais Ali **Use Case Updated On:** 27th Nov, 2025**Use Case Description:** This use case describes how a user recovers their account by requesting a password reset link via email and setting a new password.**Primary Actors:** <Patient>**Stakeholders & Interest:**
User: Wants to regain access to their account.
System: Needs to verify the user's identity before allowing a password change.**Pre-Conditions:**

1. System must be in running state.
2. User is on the Login screen.
3. User possesses a valid, registered email address.

Main Success Scenario:

	User Action		System Response
1	User clicks on the Forgot Password? link on the Login screen.		
		2	System displays a prompt requesting the user's registered email address.

3	User enters their email address and clicks Send Reset Link.		
		4	System verifies that the email address exists in the database.
		5	System sends a password reset link to the provided email address.
		6	System displays a notification: Reset link sent. Please check your email.
7	User opens the email and clicks the reset link.		
		8	System verifies the validity (and expiration) of the reset link.
		9	System displays the Create New Password screen.
10	User enters a new password and confirms it.		
11	User clicks Reset Password.		
		12	System updates and securely stores the new password.
		13	System redirects the user to the Login screen.

Post-Conditions: The user's password is updated in the database. The user must now log in with the new credentials.

Extension Points:

4a. Email Not Found:

- System checks database and finds no matching account.
- System displays a generic message (If an account exists, a link has been sent) to prevent email enumeration, or displays “Email not registered” (depending on security policy).

8a. Expired/Invalid Link:

- User clicks a link that is old or already used.
- System displays an error: This link has expired.
- System prompts user to request a new link.

10a. Weak Password:

- User enters a password that does not meet security standards (e.g., too short).
- System highlights the error and requests a stronger password.

Priority: HIGH

Frequency: Less frequent in use.

Cross-Reference: SRS-15, SRS-16, SRS-17, SRS-18

Use Case ID: UC-05

Use Case Name: Change Password

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a logged-in user updates their account password for security purpose. It restricts access for users who authenticated via external providers (Google).

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to secure their account with a new credential.
System: Needs to enforce security policies and update records securely.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. User must be on the Profile Section.
4. User must NOT be logged in via Google (Social Login users have this option disables).

Main Success Scenario:

	User Action		System Response
1	User navigates to the profile section and clicks Change Password.		
		2	System displays the Change Password form requiring: Current Password, New Password, and Confirm Password.
3	User enters their Current Password.		
4	User enters a New Password and re-enters it in Confirm Password.		
5	User clicks the Update or Save button.		

		6	System verifies that the Current Password matches the stored credentials.
		7	System validates that the New Password meets security requirements (min 8 chars, 1 uppercase, 1 number).
		8	System confirms that New Password and Confirm Password fields match.
		9	System updates the password in the secure database.
		10	System displays a success message confirming the change.

Post-Conditions: The user's password is updated. The user must use the new password for future logins.

Extension Points:

6a. Incorrect Current Password:

- System detects the current password entered is wrong.
- System displays an error: Incorrect current password.
- User is prompted to retry.

7a. Weak Password:

- System detects the new password is too simple.
- System displays an error listing the missing requirements (e.g., Must contain 1 number).

8a. Password Mismatch:

- System detects New and Confirm passwords do not match.
- System displays an error: Passwords do not match.

Alternate Flow (Google User):

- If a Google-authenticated user views the Profile, the Change Password button is disabled (greyed out) or hidden entirely.

Priority: HIGH

Frequency: Less frequent in use.

Cross-Reference: SRS-19, SRS-20, SRS-21, SRS-22, SRS-23, SRS-24

Use Case ID: UC-06

Use Case Name: Process SocialLogin

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user authenticates using their Google account. The system handles both registration (new users) and login (existing users) automatically.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants a quick, one-tap login without remembering a new password.
System: Needs to verify identity via a trusted third-party provider (Google).

Pre-Conditions:

1. System must be in running state.
2. User must have an active internet connection (Required for Google API).
3. User must be on the Login or Registration screen.

Main Success Scenario:

	User Action		System Response
1	User clicks the Sign in with Google button.		
		2	System redirects the user to the Google Authentication interface.
3	User selects their Google account and grants permission.		
		4	System retrieves basic user information (Full Name, Email) from Google.
		5	System checks if the retrieved email address already exists in the database.
		6	Scenario A: Existing User) System identifies the account exists and logs the user in directly. (Scenario B: New User) System automatically creates a new user account using the Google details.
		7	System establishes a secure session.

		8	System redirects the user to the Home screen.
--	--	---	---

Post-Conditions: The user is authenticated and directed to the dashboard. The system flags this user as Social Login disabling the ability to change password locally.

Extension Points:

3a. User Cancels / Denies Permission:

- User clicks Cancel on the Google screen.
- System returns the user to the Login screen without authenticating.

4a. Google API/Network Failure:

- System cannot connect to Google services.
- System displays an error: Google Sign-In failed. Please check your connection.

6c. Account Conflict:

- If the email exists but was created with a standard password (not Google), the system links the Google ID to the existing account (optional logic depending on security preference), or prompts the user to the link them.

Priority: HIGH

Frequency: High frequent in use.

Cross-Reference: SRS-25, SRS-26, SRS-27, SRS-28, SRS-29, SRS-30

Use Case ID: UC-07

Use Case Name: View Profile

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user views their personal details, physical attributes, and calculated health metrics(BMI).

Primary Actors: <Patient>

Stakeholders & Interest:
User: Wants to verify their personal and health information.
System: Needs to present accurate, up-to-date data and perform real-time calculations (BMI).

Pre-Conditions: 1. System must be in running state.

2. User must be logged in.
3. User data (weight, height, etc.) must exist in the database (or show as empty/default).

Main Success Scenario:

	User Action		System Response
1	User clicks on the Profile tab or menu option.		
		2	System retrieves user details (Name, Email, Photo) from authentication or database storage.
		3	System retrieves health data (Blood Group, Weight, Height, Gender, DOB).
		4	System automatically calculates the BMI using the stored weight and height values.
		5	System displays the Profile Screen containing: Profile Photo, Full Name & Email, Gender & Date of Birth , Blood Group , Weight & Height , Calculated BMI

Post-Conditions: The user is viewing their current profile information.

Extension Points:

2a. Data Loading Error:

- System fails to fetch data due to network or database issues.
- System displays a “Failed to load profile” message with a “Retry” button.

3a. Missing Health Data:

- If weight/height are not set, System displays “Not Set” or “—” in those fields.
- System displays “—” for BMI (calculation cannot be performed).
- System provides a prompt/button: “Complete your Health Profile”.

5a. No Profile Photo:

- If no photo was uploaded (SRS-37), System displays a default placeholder avatar.

Priority: HIGH

Frequency: High frequent in use.

Cross-Reference: SRS-31, SRS-32, SRS-33, SRS-34, SRS-35, SRS-36, SRS-37

Use Case ID: UC-08

Use Case Name: Update Profile

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user edits their personal information, health metrics, and profile photo. It also includes the ability to navigate to the password change screen.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to keep their personal and health data current.
System: Needs accurate weight/height data to provide correct BMI calculations.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. User is viewing the Profile screen.

Main Success Scenario:

	User Action		System Response
1	User clicks the Edit Profile button.		
		2	System displays the Update Profile screen with editable fields pre-filled with current data.
3	User modifies personal details (Full Name, Email, Date of Birth, Gender).		
4	User modifies health details (Blood Group, Weight, Height).		
5	(Optional) User clicks on the profile image to upload or replace their photo.		
6	User clicks the Save button.		
		7	System automatically recalculates the BMI based on the new weight/height values.
		8	System validates the input formats (e.g. valid email).

		9	System saves all updated information securely to Firebase under the user's account.
		10	System displays a confirmation message: Profile updated successfully.
		11	System redirects user back to the View Profile screen showing the updated details and photo.

Post-Conditions: User profile data is updated in the cloud (Firebase) and locally. The displayed BMI reflects the new weight/height.

Extension Points:

8a. Invalid Input:

- User enters an invalid email format or unrealistic weight/height.
- System highlights the error and prevents saving.

9a. Network/Firebase Error:

- System fails to connect to Firebase.
- System displays an error: "Failed to save changes. Check internet connection".
- System retains the user's edits on screen so they can try again.

Priority: MEDIUM

Frequency: Less frequent in use.

Cross-Reference: SRS-38, SRS-39, SRS-40, SRS-41, SRS-42, SRS-43, SRS-44, SRS-45, SRS-46, SRS-47, SRS-48

Use Case ID: UC-09

Use Case Name: Manage Basic Health Information

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user records specific physical attributes (height, weight, blood group) to allow the system to track health metrics and automatically calculate Body Mass Index(BMI).

Primary Actors: <Patient>

Stakeholders & Interest:**User:** Wants to track physical health changes.**System:** Needs accurate data for BMI calculation and medical context.**Pre-Conditions:**

1. System must be in running state.
2. User must be logged in.
3. User is on the Basic Health Information or Edit Profile screen.

Main Success Scenario:

	User Action		System Response
1	User navigates to the Health Information section.		
		2	System displays input fields for Height, Weight, and Blood Group.
3	User enters their height in centimeters		
4	User enters their weight in kilograms.		
		5	System automatically calculates and displays the BMI based on the entered height and weight values.
6	User selects their blood group from a predefined dropdown list (e.g., A+, O-).		
7	User clicks the Save button.		
		8	System validates all fields to ensure data accuracy (e.g., non-negative values).
		9	System securely stores the health information in Firebase.
		10	System displays a confirmation message indicating the data was saved successfully.

Post-Conditions: The user's health metrics are updated in the cloud database, and the latest BMI is visible on the profile.

Extension Points:**5a. Incomplete Data for BMI:**

- If user enters only Weight but deletes Height (or vice versa).
- System clears the BMI display or shows “—” until both fields are valid.

8a. Validation Failure:

- User enters unrealistic data (e.g., Height = 0 or negative weight).

- System highlights the invalid field and displays an error message.
- System prevents saving until corrected.

9a. Network Failure:

- System fails to connect to Firebase.
- System displays a “Save failed” error and prompts the user to retry.

Priority: MEDIUM

Frequency: Less frequent in use.

Cross-Reference: SRS-49, SRS-50, SRS-51, SRS-52, SRS-53, SRS-54, SRS-55

Use Case ID: UC-10

Use Case Name: Upload Profile Photo

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user personalizes their account by uploading, validating, and saving a profile picture from their device gallery.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to personalize their profile experience.
System: Needs to manage storage efficiently and ensure file safety.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. User must be on the Update Profile or View Profile screen.

Main Success Scenario:

	User Action		System Response
1	User clicks on the current profile picture or Edit Photo icon.		
		2	System displays options: Choose from Gallery or Remove Photo.
3	User selects Choose from Gallery.		
		4	System opens the device's native image

			picker (Gallery).
5	User selects an image file.		
		6	System validates that the file is a supported format (JPG, PNG).
		7	System resizes or compresses the image to optimize performance and storage.
		8	System uploads the processed image securely to Firebase Storage and links it to the user's account.
		9	System updates the profile view immediately to display the newly uploaded photo.

Post-Conditions: The user's profile photo is updated in the cloud and visible in the app.

Extension Points:

6a. Invalid File Format:

- User selects an unsupported file (e.g. PDF, GIF).
- System displays an error: "Invalid format. Please select a JPG or PNG image".
- System aborts the upload.

8a. Upload Failure (Network):

- System fails to complete the upload to Firebase.
- System displays "Upload failed. Please check your internet connection".

Alternate Flow:

3b. User selects "Remove Photo".

- System asks for confirmation.
- System deletes the current photo association from the database.
- System reverts the profile view to a default avatar.

Priority: LOW

Frequency: Less frequent in use.

Cross-Reference: SRS-56, SRS-57, SRS-58, SRS-59, SRS-60, SRS-61

Use Case ID: UC-11

Use Case Name: Add Medicine

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user adds a new medicine to their schedule, defining complex dosage routines, reminders, and inventory tracking.

Primary Actors: <Patient>

Stakeholders & Interest:
User: Wants accurate tracking and reminders for complex prescriptions.
System: Needs precise scheduling data to trigger timely local notifications.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. User is on the Medicine Management screen.

Main Success Scenario:

	User Action		System Response
1	User clicks on the Add Medicine Button.		
		2	System displays the add medicine form requesting name, dosage, frequency, and stock.
3	User enters the medicine name and dosage (e.g. 500mg).		
4	User selects a frequency/schedule type.		
		5	System presents scheduling options: daily, specific days, interval, or cyclic.
6	User configures the complex schedule: • Specific Days: Select mon, wed, Fri. • Interval: Selects every 2 days. • Cyclic: Sets 21 days, 7 days off.		
7	User sets reminders, defining multiple notification times per dose (e.g. 8:00 AM, 8:00 PM).		

8	(Optional) User enters the initial stock quantity.		
9	User clicks the Save button.		
		10	System validates that all required fields (name, dosage, schedule) are completed.
		11	System saves the medicine data and schedule securely in local storage.
		12	System schedules the background notifications based on the complex rules defined.
		13	System displays a confirmation message ("Medicine added successfully").

Post-Conditions: The medicine is added to the local database, and reminders are active. The inventory count is initialized if provided.

Extension Points:

9a. Missing Required Fields:

- System detects missing Name or Frequency.
- System highlights the empty fields and displays an error message.
- User must complete fields to proceed.

10a. Storage Error:

- System fails to write to the local database (e.g. insufficient storage).
- System displays an error: "Failed to save medicine". Check device storage.

Priority: HIGH

Frequency: Most frequent in use.

Cross-Reference: SRS-62, SRS-63, SRS-64, SRS-65, SRS-66, SRS-67, SRS-68

Use Case ID: UC-12

Use Case Name: Update Medicine Details

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user modifies an existing medicine

entry, including changing the dosage, category, or rescheduling the reminder notification.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to adjust their medication regimen as prescriptions change.

System: Needs to update local data and reset background alarm triggers.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. User has selected a specific medicine from the inventory list.

Main Success Scenario:

	User Action		System Response
1	User clicks the edit button on the Medicine Details Screen.		
		2	System displays the update medicine form with existing data pre-filled.
3	User modifies the name, category, or dosage.		
4	User modifies the reminder schedule (time, frequency, or complex intervals).		
5	User clicks the update or save button.		
		6	System validates that all required fields are completed and valid.
		7	System overwrites the existing record in local storage with the new information.
		8	System cancels old notifications and reschedules new ones based on the updated schedule.
		9	System displays a confirmation message (“medicine updated successfully”).
		10	System redirects user to the updated medicine details view.

Post-Conditions: The medicine record is updated locally. Old alarms are cancelled, and new alarms are active.

Extension Points:**6a. Validation Failure:**

- System detects that the Medicine Name is empty.
- System highlights the field and displays an error.
- User must enter a valid name to proceed.

7a. Database Error:

- System fails to update the record in local storage.
- System displays “Update failed. Please try again.”

Priority: HIGH

Frequency: Most frequent in use.

Cross-Reference: SRS-69, SRS-70, SRS-71, SRS-72, SRS-73

Use Case ID: UC-13

Use Case Name: Set Medicine Reminder

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user configures multiple alerts for a single dose and how they interact with the notification (take/ snooze/ miss) when its arrives.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants flexible reminders (e.g., 5 min before) and quick actions to mark medicines as taken.

System: Needs to reliably trigger alerts and log the adherence status locally.

Pre-Conditions:

1. System must be in running state.
2. A medicine with a dosage schedule must exist in the database.

Main Success Scenario:

	User Action		System Response
1	User navigates to the Reminder Settings for a specific medicine.		

2	User selects Notification Preferences.		
		3	System displays options for before time, exact time, and after time.
4	User enables multiple notifications per dose (e.g. 5 mins before, Exact time).		
5	User clicks Save.		
		6	System saves the reminder schedule locally.
		7	System continually monitors the current time against the saved schedule.
		8	System detects the scheduled time and triggers the notification.
		9	System displays the notification with action buttons: take, missed, and snooze.
10	User taps the take button on the notification.		
		11	System marks the dose as taken and updates the status locally.
		12	System cancels any remaining pending alerts for this specific dose (e.g. the "2 mins after" alert).

Post-Conditions: The medicine dose status is updated in the local database (used for reports), and the reminder cycle for that specific dose is completed.

Extension Points:

8a. User Taps “Snooze”:

- System reschedules the notification to trigger again after a predefined interval.
- Status remains “Pending”.

8b. User Taps “Missed”:

- System marks the dose as “Missed” in the local database.
- System stops further notifications for this dose.

8c. No Action (User ignores):

- System keeps the notification visible in the tray.
- If “Post-Time” alerts are set (SRS-74), System triggers the next reminder (e.g., “2 mins after”).

Priority: HIGH

Frequency: High frequent in use.

Cross-Reference: SRS-74, SRS-75, SRS-76, SRS-77, SRS-78

Use Case ID: UC-14

Use Case Name: Disable Medicine Alert

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user temporarily or permanently turns off reminders for a specific medicine without deleting the medicine from their inventory list.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to stop annoying notifications for meds they aren't currently taking but wants to keep the record.

System: Needs to cancel scheduled tasks while preserving data integrity.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. The target medicine must exist in the list and have active reminders.

Main Success Scenario:

	User Action		System Response
1	User navigates to the Medicine Management or Medicine Details screen.		
2	User selects the specific medicine they wish to silence.		
		3	System displays the medicine details, including a toggle switch or checkbox for reminders active.
4	User toggles the switch to OFF (Disable).		

		5	System prompts for confirmation (optional, depending on UI design) or processes the request immediately.
		6	System removes the medicine from the active alert queue immediately.
		7	System cancels any pending background notifications for this medicine.
		8	System updates the local database to flag reminders as disabled while keeping the medicine record in the inventory
		9	System displays a confirmation message (e.g. Alerts disabled for [Medicine Name]).

Post-Conditions: The medicine remains in the user's list (inventory), but no further notifications will be triggered.

Extension Points:

4a. Re-enabling Alerts:

- User toggles the switch back to ON.
- System checks the previously defined schedule.
- System re-schedules the notifications starting from the current time.

6a. System Error:

- System fails to cancel the background alarm service.
- System displays an error “Failed to update settings” and reverts the toggle to ON.

Priority: MEDIUM

Frequency: Less frequent in use.

Cross-Reference: SRS-79, SRS-80, SRS-81

Use Case ID: UC-15

Use Case Name: Categorize Medicine

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user assigns a form/type to a medicine

(e.g. tablet, syrup) to organize their inventory. It includes the ability to define custom categories if the default options are insufficient.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to visually distinguish between different types of medication.

System: Needs to structure data for filtering and reporting.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. User is currently in the Add Medicine or Update Medicine process.

Main Success Scenario:

	User Action		System Response
1	User taps the Category field while adding or editing a medicine.		
		2	System displays a list of predefined categories (e.g. Tablet, Syrup, Injection, Insulin).
3	User selects a specific category from the list.		
		4	System highlights the selection and closes the list.
		5	User proceeds to save the medicine.
		6	System saves the selected category information locally linked to the specific medicine data.

Post-Conditions: The medicine is associated with a specific category, allowing for organized views and icons in the schedule.

Extension Points:

3a. Create Custom Category:

- User does not find the desired option in the list.
- User selects “Add Custom Category” (or “Other”).
- System displays a text input prompt.
- User enters the new category name (e.g. “Inhaler”).
- System saves the new category to the local list.

- System automatically selects this new category for the current medicine.

3b. Cancel Selection:

- User taps outside the list.

- System retains the previously selected (or default) category.

Priority: LOW

Frequency: Less frequent in use.

Cross-Reference: SRS-82, SRS-83, SRS-84

Use Case ID: UC-16

Use Case Name: View Calendar

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how the user views a monthly or weekly calendar interface that visually marks dates containing scheduled medication doses or medical appointments.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants a high-level overview of their upcoming and past health schedule.

System: Needs to query stored records to visualize data distribution.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. Local database must contain scheduled medicines or appointments.

Main Success Scenario:

	User Action		System Response
1	User taps on the Schedule or Calendar tab in the bottom navigation bar		
		2	System loads the calendar interface (defaulting to the current month/week).

		3	System queries the local database for any medicines or appointments scheduled for the displayed dates.
		4	System highlights the dates that have events using visual indicators (e.g. a dot, color coding, or icon).
5	User swipes to view the next or previous month.		
		6	System updates the view and loads markers for the new date range.

Post-Conditions: The user is presented with a visual representation of their schedule.

Extension Points:

3a. No Events Found:

- System finds no scheduled medicines or appointments for the current month.
- System displays the calendar without any date highlights/markers.

4a. Distinction of Event Types:

- (Optional Enhancement) System uses different colors (e.g., Red for Meds, Blue for Appointments) to distinguish the type of highlight on the calendar.

Priority: MEDIUM

Frequency: High frequent in use.

Cross-Reference: SRS-85

Use Case ID: UC-17

Use Case Name: View Daily Schedule

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how the system presents the user with a comprehensive list of all medication doses and medical appointments scheduled specifically for the current day.

Primary Actors: <Patient>

Stakeholders & Interest: **Users:** Wants to know exactly what they need to do today to stay adherent.

System: Needs to filter the database for the current date stamp.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. The device date/time must be set correctly.

Main Success Scenario:

	User Action		System Response
1	User navigates to the Home Screen or Schedule Tab.		
		2	System retrieves the current system date.
		3	System queries the local database for all medicines scheduled for this specific date.
		4	System queries the local database for all appointments scheduled for this specific date.
		5	System organizes the items (likely chronologically by time).
		6	System displays the Daily Schedule list containing all medicines and appointments for the current day.

Post-Conditions: The user is informed of their immediate health tasks for the day.

Extension Points:

3a/4a. No Scheduled Items:

- System finds no medicines or appointments for the current day.
- System displays a “No tasks for today” or “You are all caught up!” message (Empty State).

6a. Interaction:

- User taps on an item in the list.
- System navigates to the details of that specific Medicine or Appointment.

Priority: HIGH

Frequency: High frequent in use.

Cross-Reference: SRS-86

Use Case ID: UC-18

Use Case Name: View Date Schedule

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user selects a specific date from the calendar to view the detailed list of medicines and appointments scheduled for that day.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to plan ahead or review past adherence for a specific day.
System: Needs to filter database records based on the user-selected date.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. User is viewing the Calendar screen.

Main Success Scenario:

	User Action		System Response
1	User taps on a specific date in the Calendar view.		
		2	System captures the selected date.
		3	System queries the local database for medicines scheduled for that date.
		4	System queries the local database for appointments scheduled for that date.
		5	System displays a list or modal view showing the details of medicines and appointments for the selected date.
6	User views the details (time, dosage, doctor name).		

Post-Conditions: The user is presented with the schedule for the chosen date.

Extension Points:

3a/4a. No Events:

- System finds no entries for the selected date.

- System displays an empty state message (e.g. “No schedule for this date”).

Alternate Flow: Date Switching

- User swipes or taps a different date immediately.

- System refreshes the list view with data for the newly selected date.

Priority: MEDIUM

Frequency: Most frequent in use.

Cross-Reference: SRS-87

Use Case ID: UC-19

Use Case Name: Apply Date Range Limitation

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how the system restricts the calendar view to a specific time window (15 days past and 15 days future) to maintain relevance and performance.

Primary Actors: <Patient><System>

Stakeholders & Interest: **User:** Wants a focused view of their immediate schedule without clutter from distant dates.

System: Wants to minimize data fetching load by limiting the query range.

Pre-Conditions:

1. System must be in running state.
2. User navigates to the Calendar View.

Main Success Scenario:

	User Action		System Response
1	User opens the Calendar screen.		
		2	System identifies the Current Date from the device settings.

		3	System calculates the valid date range: Current Date - 15 days (Start Limit) and Current Date + 15 days (End Limit).
		4	System queries the local database for medicines and appointments only within this calculated 30-day window.
		5	System renders the calendar view, enabling interaction only for dates within this range.
6	User attempts to swipe or scroll to a date outside this range.		
		7	System prevents navigation beyond the limit (e.g., disables the Next Month button or stops scrolling).

Post-Conditions: The calendar is displayed with a restricted view. Data outside the ± 15 day window is hidden or not loaded.

Extension Points:

7a. Visual Feedback:

- When a user tries to scroll past the limit, the system displays a toast message: “Calendar is limited to 15 days past and future”.

Alternate Flow (Date Change):

- If the user changes the device date, the system recalculates the window and refreshes the view immediately.

Priority: LOW

Frequency: Most frequent in use.

Cross-Reference: SRS-88

Use Case ID: UC-20

Use Case Name: Add Allergies List

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user maintains a digital record of their

allergies. It covers adding new allergies, viewing the current list and updating existing entries.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to store critical health data for reference by doctors or caregivers.

System: Needs to store this data to provide a complete health profile.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. User is on the User Health Records screen.

Main Success Scenario:

	User Action		System Response
1	User taps on the Allergies section.		
		2	System displays the list of currently recorded allergies (View).
3	User taps the Add Allergy button.		
		4	System displays a text input field.
5	User enters the allergy name (e.g. Penicillin or Peanuts).		
6	User taps Save.		
		7	System validates the entry (checks for empty strings or special characters).
		8	System saves the new allergy to the user's health record.
		9	System refreshes the list to display the newly added allergy.

Post-Conditions: The allergy list is updated in the database and visible to the user.

Extension Points:

7a. Validation Failure:

- System detects an empty field or invalid characters.
- System displays an error: "Please enter a valid allergy name".
- User is prompted to re-enter.

Alternate Flow: Update/Edit Allergy:

3b. User taps on an existing allergy.

4b. System enables editing mode for that entry.

5b. User modifies the text.

6b. User saves, and system validates before updating.

Alternate Flow: Duplicate Entry:

- System detects the allergy is already in the list.

- System displays: "This allergy is already recorded".

Priority: MEDIUM

Frequency: Less frequent in use.

Cross-Reference: SRS-89, SRS-90

Use Case ID: UC-21

Use Case Name: View Current Medication

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user views their list of currently active prescriptions and manages their status by marking them as active or completed.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to see a clear list of what they should be taking now vs. what they have finished.

System: Needs to segregate active reminders from historical records.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. User must have added at least one medicine.

Main Success Scenario:

	User Action		System Response
1	User navigates to the Current Medication tab in Health Records.		

		2	System queries the local database for medicines with status Active.
	.	3	System displays the list of all medicines the user is currently taking.
4	User identifies a medicine course that has ended.		
5	User selects the option to mark the medicine as Completed.		
		6	System prompts for confirmation (e.g. Are you sure you finished this course?).
7	User confirms the action.		
		8	System updates the medicine status from Active to Completed in the database.
		9	System removes the medicine from the Current list and moves it to the Past Medication history.
		10	System disables further reminders for this medicine.

Post-Conditions: The medicine status is updated. It no longer appears in the active list but is preserved in the history records.

Extension Points:

3a. No Active Medicines:

- System finds no active records.
- System displays an empty state message (“No active medications”).

Alternate Flow: Re-activate Medicine:

- User views a “Completed” medicine (in Past view).
- User marks it as “Active”.
- System moves it back to the “Current Medication” list and re-enables reminders.

Priority: MEDIUM

Frequency: Less frequent in use.

Cross-Reference: SRS-91, SRS-92

Use Case ID: UC-22

Use Case Name: View Past Medication

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user accesses their medication history to view a list of prescription they have finished taking or discontinued.

Primary Actors: <Patient>

Stakeholders & Interest:
User: Wants to recall previous prescriptions for doctor visits or personal reference.
System: Needs to archive completed treatments separately from active alerts.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. Medicines must have been previously marked as Completed or removed from the active list.

Main Success Scenario:

	User Action		System Response
1	User navigates to the User Health Records section.		
2	User selects the Past Medication tab/option.		
	.	3	System queries the local database for all medicines with the status Completed or Inactive.
		4	System automatically updates the list to include any medicine recently marked as completed or removed from the active view.
		5	System displays the list of past medicines, showing details like Name, Dosage, and Date Stopped.
6	User scrolls through the history list.		

Post-Conditions: The user has successfully viewed their medication history.

Extension Points:

3a. No History Found:

- System finds no records with “Completed” status.
- System displays an empty state message (e.g. “No past medication history”).

Alternate Flow: Search History:

- User enters a medicine name in a search bar within the Past Medication screen.
- System filters the historical list to match the query.

Priority: LOW

Frequency: Less frequent in use.

Cross-Reference: SRS-93, SRS-94

Use Case ID: UC-23

Use Case Name: Add Restraints

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user records specific medication restrictions (e.g. No Aspirin) or dietary restraints (e.g. Low Sugar) to maintain a complete health profile.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to document restrictions to avoid adverse health effects.
System: Needs this context to potentially warn users (future enhancement) or display to caregivers.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. User is on the User Health Records screen.

Main Success Scenario:

	User Action		System Response
1	User taps on the Restraints or Restrictions section.		
		2	System displays the list of currently recorded restraints (if any).
3	User taps the Add Restraint button.		
		4	System displays a form with fields for Type (e.g. Dietary, Medication) and Description.
5	User selects the Type and enters the Description (e.g. Gluten Intolerance).		
6	User clicks the Save button.		
		7	System validates that the description field is not empty.
		8	System securely saves the restraint entry to the user's health record.
		9	System updates the list view to include the new restraint.

Post-Conditions: The user's health record is updated with the new restriction data.

Extension Points:**7a. Missing Information:**

- System detects the Description is empty.
- System displays an error message.
- User must enter text to proceed.

Alternate Flow: Remove Restraint:

- User selects an existing restraint.
- User taps "Delete".
- System removes the entry from the list.

Priority: LOW

Frequency: Less frequent in use.

Cross-Reference: SRS-95

Use Case ID: UC-24

Use Case Name: Manage Chronic Illnesses

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user maintains a record of their long-term health conditions (e.g Diabetes, Hypertension). It covers viewing the list, adding new conditions, and updating existing ones.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to keep a central record of their medical history for easy reference.
System: Needs this data to potentially correlate with medication types (future scope).

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. User is on the User Health Records screen.

Main Success Scenario:

	User Action		System Response
1	User taps on the Chronic Illnesses section.		
		2	System displays the list of currently recorded illnesses (View).
3	User taps the Add Illness button.		
		4	System displays a form field for the illness name (and optional start date).
5	User enters the illness name (e.g. Asthma).		
6	User clicks the Save button.		
		7	System validates that the name field is not empty.
		8	System securely saves the new entry to the user's health profile.
		9	System updates the list view to include the new illness.

Post-Conditions: The user's medical profile is updated with the new chronic condition.

Extension Points:

Alternate Flow: Update Illness:

3b. User taps on an existing illness in the list.

4b. System opens the entry in “Edit Mode”.

5b. User modifies the name or details.

6b. User saves the changes.

7a. Validation Error:

- User attempts to save an empty field.

- System displays “Please enter an illness name”.

Priority: MEDIUM

Frequency: Less frequent in use.

Cross-Reference: SRS-96

Use Case ID: UC-25

Use Case Name: View Current Stock

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user monitors their medicine inventory. The system prioritizes low-stock items to ensure the user is aware of what needs replenishment.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to avoid running out of critical medication.
System: Needs to sort data to highlight urgent items.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. Medicines must exist in the database with stock tracking enabled.

Main Success Scenario:

	User Action		System Response
1	User navigates to the Inventory or Stock section.		
		2	System queries the local database for all registered medicines and their current stock counts.
		3	System analyzes the stock levels for each item.
		4	System applies a sorting algorithm to identify medicines with stock ≤ 5 .
		5	System displays the list of medicines with their quantities, placing the low-stock items (≤ 5) at the very top of the list.
6	User views the stock status.		

Post-Conditions: The user is presented with a prioritized view of their medicine inventory.

Extension Points:**2a. No Medicines Found:**

- System finds no medicine records.
- System displays an empty state (“No inventory to display”).

5a. Visual Highlighting:

- (UI Enhancement) System displays the low-stock items in Red or with a warning icon to Distinguish them further from normal stock items.

Priority: MEDIUM

Frequency: Most frequent in use.

Cross-Reference: SRS-97, SRS-98

Use Case ID: UC-26

Use Case Name: Update Current Stock

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user manually adjusts the inventory count for a medicine, typically when refilling a prescription or correction a count discrepancy.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to keep inventory accurate to ensure alerts work correctly.

System: Needs accurate data to trigger "Low Stock" warnings.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. The user must have medicines added to their inventory.

Main Success Scenario:

	User Action		System Response
1	User navigates to the Inventory or Medicine Details screen.		
		2	System displays the current stock quantity.
3	User selects a specific medicine to update.		
4	User inputs the new quantity or uses “+” / “-” buttons to increase or decrease the count.		.
5	User clicks the Update or Save button.		
		6	System validates the new quantity (e.g. ensures it is not a negative number).
		7	System saves the updated stock quantity securely in the local database.
		8	System displays a confirmation message (Stock updated successfully).
		9	System refreshes the view to show the new stock level.

Post-Conditions: The local database reflects the new stock count. If the new count is low (≤ 5), the system may trigger a low-stock alert (referenced in next module).

Extension Points:**5a. Negative Value Entered:**

- User attempts to decrease stock below zero.
- System prevents the action and displays an error: “Stock cannot be negative”.

6a. Write Error:

- System fails to update the local record.
- System displays an error message and retains the previous value.

Priority: HIGH

Frequency: Most frequent in use.

Cross-Reference: SRS-99, SRS-100, SRS-101

Use Case ID: UC-27

Use Case Name: Generate Stock Alerts

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how the system automatically monitors inventory levels and triggers alerts to the user when medicine stock falls below a defined threshold, ensuring the user has time to refill prescriptions.

Primary Actors: <System>

Stakeholders & Interest: **User:** Wants to be reminded to buy medicine before running out.
System: Needs to enforce inventory rules and trigger proactive notifications.

Pre-Conditions:

1. System must be in running state.
2. Medicines must have stock tracking enabled.
3. A Stock Change event has just occurred (e.g., user took a dose or manually adjusted stock).

Main Success Scenario:

	User Action		System Response
1	User performs an action that reduces stock (e.g. marks a dose as Taken or manually edits stock).		
		2	System calculates the new remaining stock quantity for that medicine.
		3	System compares the new quantity against the defined Low Stock Threshold (e.g. ≤ 5 units).
		4	(Condition Met: Low Stock) System detects the stock has reached or fallen below the threshold.
		5	System triggers a notification alerting the user that the specific medicine is running low.
		6	System immediately updates the Low Stock section, adding this medicine to the list for quick reference.
		7	System ensures the alert status persists until the stock is replenished.

Post-Conditions: The user receives a notification, and the dashboard/inventory screen highlights the low-stock item.

Extension Points:**3a. Stock Above Threshold:**

- System calculates stock is still sufficient (> 5).
- System takes no further action regarding alerts.
- If the item was previously in the “Low Stock” list (e.g., user just refilled it), System removes it from that section.

5a. Notification Settings:

- User has disabled “Inventory Alerts” in settings.
- System skips the push notification but still updates the “Low Stock” UI section.

Priority: HIGH

Frequency: High frequent in use.

Cross-Reference: SRS-102, SRS-103, SRS-104

Use Case ID: UC-28

Use Case Name: Add Appointment

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user schedule a new visit to a doctor or clinic, categorizes the visit, and configures reminder notification to ensure they attend on time.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to organize their medical visits and receive timely reminders.

System: Needs to store schedule data and trigger alerts at the correct time.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. User is on the Appointment Management screen.

Main Success Scenario:

	User Action		System Response
1	User taps the Add Appointment button.		
		2	System displays a form with fields for Doctor/Clinic Name, Date, Time, Category, and Reminder Settings.
3	User enters the Doctor or Clinic Name.		
4	User selects the Date and Time of the visit.		
5	User selects a Category (e.g. General Checkup, Dentist, Cardiology).		
6	User enables the Reminder option and sets a notification time (e.g. 1 hour before).		
7	User clicks the Save button.		

		8	System validates that mandatory fields (Name, Date, Time) are filled.
		9	System saves the appointment details locally.
		10	System schedules the reminder notification based on the user's preference.
		11	System displays a confirmation message and adds the appointment to the calendar/ list view.

Post-Conditions: The appointment is recorded in the system, and a background alert is set for the specified reminder time.

Extension Points:

4a. Invalid Date:

- User selects a date/time in the past.
- System displays a warning: “Appointment cannot be in the past”. (Unless intended for Record-keeping, depending in business logic).

8a. Missing Fields:

- System detects missing Doctor Name or Date.
- System highlights the field and displays “Required field”.
- User must complete the form to proceed.

Priority: HIGH

Frequency: Most frequent in use.

Cross-Reference: SRS-105, SRS-106

Use Case ID: UC-29

Use Case Name: Update Appointment

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user modifies the details of an existing

appointment, such as rescheduling the date/time, changing the doctor's name, or updating reminder preferences and visit notes.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to keep their schedule accurate when plans change.
System: Needs to update local records and reschedule any pending notification triggers.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. An existing appointment must be selected.

Main Success Scenario:

	User Action		System Response
1	User navigates to the appointment details view and clicks the Edit button.		
		2	System displays the Update Appointment form with current data pre-filled.
3	User modifies the Doctor/Clinic Name, Category, Date, or Time.		
4	User updates the Visit Notes or adjusts the Reminder Settings (e.g. changing the alert time).		
5	User clicks the Update or Save button.		
		6	System validates the updated information (e.g. ensures the date is valid).
		7	System saves the changes securely in local storage.
		8	System cancels the old reminder notification and schedules a new one based on the updated settings.
		9	System displays a confirmation message (Appointment updated successfully).
		10	System redirects the user to the updated Appointment Details view.

Post-Conditions: The appointment record is updated locally. The previous reminder is cancelled, and

the new reminder is active.

Extension Points:

6a. Validation Failure:

- System detects that the new Date/Time is invalid.
- System highlights the error and prompts the user to correct it.

8a. System Error:

- System fails to update the local database.
- System displays an error message and discards the changes.

Priority: MEDIUM

Frequency: Most frequent in use.

Cross-Reference: SRS-107, SRS-108

Use Case ID: UC-30

Use Case Name: Manage Appointment Categories/Status

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how the system organizes appointments by status (upcoming/completed) and allow the to manually update the status of a visit.

Primary Actors: <Patient><System>

Stakeholders & Interest: **User:** Wants to see immediate upcoming tasks first and clear out finished tasks.

System: Needs to maintain order and accurate history records.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. Appointments must exist in the database.

Main Success Scenario:

	User Action		System Response
1	User navigates to the Appointments list view.		

		2	System compares the current date and time against stored appointment schedules.
		3	System automatically categorizes appointments as Upcoming (future dates) or Completed (past dates).
		4	System sorts and displays the list, placing Upcoming appointments at the top for quick access.
5	User identifies an appointment that they have attended.		
6	User selects the option to Mark as Completed (via checkbox, swipe action, or button).		
		7	System updates the appointment status in the local database to Completed.
		8	System moves the appointment from the Upcoming section to the Completed/ History section.
		9	System displays a confirmation message (Appointment marked as completed).

Post-Conditions: The appointment status is updated, and the list is re-organized to prioritize remaining future visits.

Extension Points:

2a. No Appointments:

- System finds no records.
- System displays an empty state (“No upcoming appointments”).

6a. Accidental Completion:

- User marks an appointment as completed by mistake.
- User navigates to the “Completed” section.
- User selects “Mark as Upcoming/Incomplete”.
- System restores the appointment to the active list.

Priority: MEDIUM

Frequency: High frequent in use.

Cross-Reference: SRS-109, SRS-110, SRS-111

Use Case ID: UC-31

Use Case Name: Set Appointment Reminder

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user configures notification alerts for medical visits, including setting a specific lead time (e.g. 30 mins before) and enabling recurring alerts for regular treatments.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to be reminded sufficiently in advance to travel to the clinic.
System: Needs to register a scheduled task in the background service.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. User is in the process of Adding or Editing an appointment.

Main Success Scenario:

	User Action		System Response
1	User locates the Reminder Settings section within the Appointment form.		
		2	System displays a toggle to Enable Reminder and options for timing.
3	User toggles the reminder to ON.		
4	User selects a User-Specified Time (e.g. 15 minutes, 1 hour, or 1 day before).		
5	(Optional) User enables the Recurring Reminder option for regular visits (e.g. weekly therapy).		
6	User clicks Save or Done.		
		7	System calculates the exact trigger time based on the appointment date and the user-specified lead time.
		8	System schedules the notification in the local background service.

		9	System saves the preference locally and displays a confirmation icon (e.g. a bell icon next to the appointment).
--	--	---	--

Post-Conditions: A background alarm is active. At the calculated time, the system will trigger a push notification.

Extension Points:

4a. Invalid Time:

- User sets a reminder time that has already passed (e.g., reminder 1 hour before an appointment that is in 30 mins).
- System displays a warning: “Reminder time is in the past”.

5a. Recurring Logic:

- If “Recurring” is selected, System prompts for frequency (e.g., Weekly, Monthly).
- System creates a series of appointment instances or sets a repeating alarm logic.

Priority: MEDIUM

Frequency: Most frequent in use.

Cross-Reference: SRS-112, SRS-113

Use Case ID: UC-32

Use Case Name: Add Visits Notes

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user attaches textual notes to a specific appointment, allowing them to record doctor's instructions, diagnosis details, or personal observations.

Primary Actors: <Patient>

Stakeholders & Interest:

User: Wants to remember key details discussed during the consultation.

System: Needs to store unstructured text data linked to the appointment record.

Pre-Conditions: 1. System must be in running state.

2. User must be logged in.
3. An appointment record (upcoming or past) must exist.

Main Success Scenario:

	User Action		System Response
1	User navigates to the Appointment Details screen for a specific visit.		
2	User taps on the Add Visit Notes or Edit Notes field		
		3	System displays a multi-line text input area.
4	User enters relevant information (e.g., Doctor said to increase water intake, Next follow-up in 2 weeks).		
5	User clicks the Save button.		
		6	System validates the input (e.g. checks character limits).
		7	System saves the notes securely in the local database linked to the specific appointment ID.
		8	System displays a confirmation message and shows the saved notes on the Appointment Details screen.

Post-Conditions: The appointment record is updated with the new notes, which are viewable in the appointment history.

Extension Points:

6a. Empty Note:

- User saves an empty field.
- System either clears previous notes (if editing) or saves nothing, returning to the detail view without error.

7a. Storage Error:

- System fails to write to the database.
- System displays “Failed to save notes” and preserves the user's text so they can try again.

Priority: LOW

Frequency: Most frequent in use.

Cross-Reference: SRS-114

Use Case ID: UC-33

Use Case Name: View Appointment History

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user accesses the archive of their past medical visits. It includes functionality to search for specific records or filter the list by date and category.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to review past doctor visits, diagnosis notes, or follow-up dates.
System: Needs to retrieve archived data efficiently without loading active schedules.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. User must have appointments marked as Completed in the database.

Main Success Scenario:

	User Action		System Response
1	User navigates to the Appointments section.		
2	User selects the History or Completed tab.		
		3	System queries the local database for all appointments with status Completed.
		4	System displays the list of past appointments, showing details like Date, Doctor Name, and Category.
		5	System displays options to filter by Category (e.g. Dentist) or search by Date.

6	(Optional) User clicks the Filter/Search icon.		
7	User selects a specific category or enters a date range.		
		8	System filters the displayed list to show only the matching records.
9	User taps on a specific history item.		
		10	System opens the detailed view of the past appointment (including visit notes).

Post-Conditions: The user has successfully retrieved specific information regarding their past medical visits.

Extension Points:

3a. No History Found:

- System finds no completed appointments.
- System displays an empty state message (e.g. “No appointment history available”).

7a. No Search Results:

- User applies a filter that yields no matches.
- System displays: “No appointments found matching your criteria”.
- System provides a “Clear Filters” button.

Alternate Flow: Reset Filter:

- User taps “Clear Filters”.
- System reloads the full list of completed appointments.

Priority: LOW

Frequency: Less frequent in use.

Cross-Reference: SRS-115, SRS-116

Use Case ID: UC-34

Use Case Name: Record Mood & Notes Journal

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user logs their daily emotional state using predefined icons and optional text notes, and how they can modify these entries later.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to track their mental well-being alongside their physical health.

System: Needs to store subjective health data for trend analysis.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. User is on the Journaling or Dashboard screen.

Main Success Scenario:

	User Action		System Response
1	User taps on the Daily Journal or Mood Tracker section.		
		2	System displays a set of predefined mood options (e.g. Happy, Sad, Neutral, Energetic).
3	User selects a Mood Icon representing their current state.		
		4	System highlights the selection.
5	(Optional) User taps the Add Note field.		
		6	System displays a text input area.
7	User types a personal reflection or note (e.g. Feeling tired after medication).		
8	User clicks the Save button.		
		9	System saves the mood selection and text note locally with the current timestamp.
		10	System displays a confirmation (Entry saved) and updates the daily streak or journal view.

Post-Conditions: The user's mood and notes are recorded for the specific date.

Extension Points:**Alternate Flow: Update Entry:**

- 1a. User navigates to a past date in the journal.
 - 2a. System displays the previously recorded mood/note.
 - 3a. User taps “Edit”.
 - 4a. User changes the mood icon or modifies the text note.
 - 5a. User saves.
 - 6a. System updates the existing record in the database.
- 6a. Empty Selection:**
- User attempts to save without selecting a mood.
 - System prompts: “Please select a mood to save your entry”.

Priority: LOW

Frequency: High frequent in use.

Cross-Reference: SRS-117, SRS-118, SRS-119

Use Case ID: UC-35

Use Case Name: Attempt User Challenges

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how system uses the Gemini API to generate personalized daily health challenges based on the user's mood and health profile, and how the user attempts them.

Primary Actors: <Patient><Gemini API>

Stakeholders & Interest: **User:** Wants relevant, non-repetitive tasks to improve motivation.
System: Wants to leverage AI to increase user retention.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. Internet connection must be available (for API generation).

Main Success Scenario:

	User Action		System Response
1	User navigates to the Motivation or Challenges tab.		
		2	System retrieves user context locally (e.g. Current Mood: Tired, Recent Adherence: High, Chronic Illness: Diabetes).
		3	System sends a prompt to the Gemini API: Generate 3 simple, optional health challenges for a diabetic user who is feeling tired. Return as JSON.
		4	System receives the API response and parses the JSON data.
		5	System displays the daily challenges (e.g. Drink Sugar-Free Electrolytes, 10 Min Stretch, Read a positive quote).
6	User views the challenges and taps Accept on one.		
		7	System marks the challenge as In Progress.
8	User completes the task and taps Complete.		
		9	System updates the visual streak tracker.
		10	System saves the completion status locally.

Post-Conditions: The user's streak is updated. The challenges for the day are cached locally.

Extension Points:**3a. Network Unavailable (Offline Mode):**

- System detects no internet connection.
- System falls back to a local list of pre-defined static challenges (e.g. “Drink Water”, “Walk 1000 steps”).
- System displays: “Showing offline challenges”.

5a. API Error / Safe Content Filter:

- Gemini API returns an error or flags content.
- System defaults to the local static list to ensure the screen is never empty.

Alternate Flow: User Ignores:

- User does not interact with challenges.
- System resets challenges the next day without penalty.

Priority: LOW

Frequency: Most frequent in use.

Cross-Reference: SRS-120, SRS-121, SRS-122

Use Case ID: UC-36

Use Case Name: Display Visual Streak Tracker

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how the system visualizes the user consistency (streaks) in taking medication or completing challenges to motivate them.

Primary Actors: <Patient><System>

Stakeholders & Interest: **User:** Wants to see progress and feel motivated by a "winning streak."
System: Wants to retain users by gamifying adherence.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. User must have at least one day of activity recorded.

Main Success Scenario:

	User Action		System Response
1	User navigates to the Dashboard or Motivation tab.		
		2	System calculates the Current Streak (consecutive days with 100% medication adherence or completed challenges).
		3	System retrieves the Max Streak (highest record ever achieved).
		4	System displays the streak visually (e.g. a Fire icon with a number, a filled calendar ring, or a progress bar).

5	User taps on the Streak icon.		
		6	System displays a detailed view (e.g. You have been consistent for 5 days! Keep it up).

Post-Conditions: The user is informed of their progress.

Extension Points:

2a. Broken Streak:

- System detects the user missed a day.
- System resets the “Current Streak” to 0.
- System displays an encouraging message (e.g. “Don’t worry, start a new streak today!”).

2b. Offline Calculation:

- System calculates streaks based on the last known local data sync.

Priority: LOW

Frequency: Most frequent in use.

Cross-Reference: SRS-123, SRS-124, SRS-125

Use Case ID: UC-37

Use Case Name: Add/Upload Report

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user uploads a digital copy of a medical report (e.g., lab result, prescription) by providing a title, date, and selecting a file from their device.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to store and organize medical documents centrally.
System: Needs to handle file input and local storage securely.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. User is on the Report Management screen.

Main Success Scenario:

	User Action		System Response
1	User taps the Add Report button.		
		2	System displays a form requesting Report Title and Date.
3	User enters the Title (e.g. Blood Test) and selects the Date.		
4	User taps the Upload File button.		
		5	System opens the device's native file picker or gallery.
6	User selects a document (PDF) or image file.		
7	User clicks the Save button.		
		8	System validates that the Title and File are present.
		9	System saves the report file securely in Device Storage.
		10	System saves the report metadata (Title, Date, Path) in the local database.
		11	System displays the new report in the list and shows a success message.

Post-Conditions: The report is saved locally and is viewable in the app list.

Extension Points:**5a. File Selection Cancelled:**

- User closes the file picker without selecting a file.
- System returns to the form view without attaching a file.

8a. Storage Error:

- System attempts to save but device storage is full or permission is denied.
- System displays an error message: "Failed to save report. Please check storage permissions".

Priority: MEDIUM

Frequency: Less frequent in use.

Cross-Reference: SRS-126. SRS-127

Use Case ID: UC-38

Use Case Name: Search Report

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user finds specific medical documents within their archive by filtering results based on report title, date, or category type.

Primary Actors: <Patient>

Stakeholders & Interest:
User: Wants to quickly locate a specific document among many uploads.
System: Needs to query metadata efficiently to filter the display list.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. Reports must already be saved in the system.

Main Success Scenario:

	User Action		System Response
1	User navigates to the Report Management screen.		
		2	System displays the full list of existing reports.
3	User taps the Search Bar or Filter Icon.		
4	User enters a keyword (Title) OR selects a Date or chooses a Type (e.g. Lab Result).		
		5	System processes the input and filters the stored report records based on the criteria.
		6	System updates the view to display the search results in a list format.
7	User taps on a specific result to view the document.		

Post-Conditions: The view displays only the reports matching the search criteria.

Extension Points:**4a. No Matches Found:**

- System queries the database and finds no matching records.
- System displays an empty state message (e.g. “No reports found matching ‘Xray’”).
- System allows the user to clear the search and try again.

Alternate Flow: Clear Search:

- User taps the “X” or “Clear” button in the search bar.
- System restores the full list of all reports.

Priority: Less

Frequency: Less frequent in use.

Cross-Reference: SRS-128. SRS-129

Use Case ID: UC-39

Use Case Name: View Report

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user opens and views a stored medical document (PDF or Image) directly within the application interface without needing external apps.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to quickly read their medical results without leaving the app.
System: Needs to render different file formats (PDF, JPG) securely.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. A valid report file must exist in local storage.

Main Success Scenario:

	User Action		System Response
1	User taps on a Report Item in the list view.		

		2	System retrieves the file path of the selected report.
		3	System identifies the file type (e.g. PDF or Image).
		4	System launches the Built-in Viewer (PDF Renderer or Image Viewer).
		5	System displays the document content on the screen.
6	User interacts with the document (e.g. pinch-to-zoom, scroll).		
7	User taps the Back or Close button.		
		8	System closes the viewer and returns the user to the Report List.

Post-Conditions: The user has successfully viewed the document content.

Extension Points:

2a. File Not Found:

- System attempts to load the file but the path is broken (e.g. file deleted outside app).
- System displays an error: "File not found".
- System prompts user to remove the broken record.

3a. Unsupported Format:

- System detects a format the built-in viewer cannot handle.
- System prompts: "Cannot preview this file. Open in external app?".

Priority: MEDIUM

Frequency: Less frequent in use.

Cross-Reference: SRS-130

Use Case ID: UC-40

Use Case Name: Access Chat Interface

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user opens the AI Assistant, sends

queries, and views a clear, readable conversation history between themselves and the bot.

Primary Actors: <Patient><Gemini API>

Stakeholders & Interest: **User:** Wants instant answers to health questions in a readable format.
System: Wants to facilitate natural language interaction.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. Internet connection must be active.

Main Success Scenario:

	User Action		System Response
1	User taps the Assistant floating action button or menu item.		
		2	System opens the Chat Interface overlay or screen.
3	User types a health-related question (e.g. How do I take this med?) and taps Send.		
		4	System immediately renders the User's message on the right side of the screen (or distinct color) to distinguish it.
		5	System sends the text to the AI Backend (Gemini).
		6	System receives the response and renders the Chatbot's message on the left side (or distinct color) clearly.
7	User scrolls through the conversation to read past messages.		

Post-Conditions: The conversation is displayed clearly, and the user has received an answer.

Extension Points:

2a. Offline Mode:

- System detects no internet.
- System disables the input field and displays: "Assistant is offline".

6a. Long Response:

- Bot response is very long.

- System formats the text with proper spacing/bullets to ensure it remains “clear” and readable.

Priority: MEDIUM

Frequency: Most frequent in use.

Cross-Reference: SRS-131,SRS-132

Use Case ID: UC-41

Use Case Name: Apply Safety Filters & Rules

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how the system monitors chat interactions to detect and block harmful, inappropriate, or unsafe content using dynamic backend rules.

Primary Actors: <Patient><Gemini API>

Stakeholders & Interest: **User:** Wants a safe environment free from harassment or dangerous medical misinformation.

System: Needs to mitigate liability and ensure safe AI usage.

Pre-Conditions:

1. User is engaged in a chat session.
2. Safety rules are defined on the backend server.

Main Success Scenario:

	User Action		System Response
1	User types a query into the chat interface and hits Send.		
		2	System intercepts the message before processing the AI response.
		3	System checks the content against the current safety rules retrieved from the backend.
		4	(Scenario: Harmful Content) System

			detects inappropriate keywords or unsafe medical requests (e.g. self-harm queries).
		5	System blocks the message processing.
		6	System displays a standard safety warning (e.g. I cannot answer this query. If this is an emergency, please call 911).

Post-Conditions: The unsafe interaction is halted, and the user is redirected to safe resources.

Extension Points:

3a. Rule Update:

- Developers update the safety keyword list on the server.
- System applies these new rules immediately to the next message without the user needing to update the app from the App Store.

4a. Content is Safe:

- System validates the content is clean.
- System passes the query to the Gemini API for a normal response.

Priority: HIGH

Frequency: High frequent in use.

Cross-Reference: SRS-133,SRS-134

Use Case ID: UC-42

Use Case Name: View User Guide

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user accesses the built-in text-based documentation to read instructions on how to use the application features.

Primary Actors: <Patient>

Stakeholders & Interest:
User: Wants clear, readable instructions on how to use the app.
System: Wants to provide lightweight, offline-accessible help.

Pre-Conditions:

1. System must be in running state.
2. User navigates to the Help & Support screen.

Main Success Scenario:

	User Action		System Response
1	User taps on User Guide in the Help menu.		
		2	System displays a list of help topics (e.g. Adding Medicine, Setting Reminders).
3	User taps on a specific topic title.		
		4	System opens the detail view for that topic.
		5	System displays the text-only instructions explaining the feature step-by-step.
6	User reads the text and scrolls down if necessary.		
7	User taps Back to return to the topic list.		

Post-Conditions: The user has read the information.

Extension Points:**2a. Search Guide:**

- User types a keyword into the search bar.
- System filters the text topics to show only those matching the keyword.

5a. Offline Access:

- Since the guide is text-only, it loads instantly from the local JSON/Database without needing an internet connection.

Priority: LOW

Frequency: Less frequent in use.

Cross-Reference: SRS-135

Use Case ID: UC-43

Use Case Name: Submit Contact/Feedback Form

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user sends feedback by launching their device's default email client. The system pre-fills the recipient address and subject line for convenience.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to send an email to support without copying/pasting the address.

System: Offloads the message handling to the external email app.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. User must have an email app (e.g., Gmail) installed and configured on their device.

Main Success Scenario:

	User Action		System Response
1	User navigates to the Help & Support section.		
2	User taps the Send Feedback or Contact Developer button.		
		3	System constructs a mailto link containing the developer's email address and a default subject (e.g. App Feedback).
		4	System launches the external Default Email Application (e.g. Gmail).
		5	The Email App opens a Compose window with the To field pre-filled with the developer's email.
6	User types their message in the email body.		
7	User taps the Send arrow/button within the email app.		
		8	The email is sent via the user's personal email account.
9	User manually switches back to the Medicine Reminder App.		

Post-Conditions: An email is sent to the developer. The user returns to the app manually.

Extension Points:**4a. No Email App Installed:**

- System attempts to launch the email client but fails.
- System displays an error: “No email app found. Please contact us at dev@example.com”.
- System copies the email address to the user's clipboard as a fallback.

Priority: LOW**Frequency:** Less frequent in use.**Cross-Reference:** SRS-136**Use Case ID:** UC-44**Use Case Name:** View About Section**Use Case Prepared by:** Atif Islam **Use Case Prepared On:** 26th Nov, 2025**Use Case Updated by:** M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user views administrative details about the application, including the current software version, developer credits, and contact information.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to check the installed version or developer details.
Developer: Wants to provide credit and version context for support.

Pre-Conditions:

1. System must be in running state.
2. User navigates to the Help & Support or Settings screen.

Main Success Scenario:

	User Action		System Response
1	User taps on the About or App Info menu option.		
		2	System displays the About Screen.
		3	System renders the App Logo and App Name.
		4	System displays the Current Version

			Number (e.g. v1.1).
		5	System lists the Developer Names (Atif Islam & Muhammad Awais Ali) and Contact Info.
6	User reads the information and taps Back.		

Post-Conditions: The user is informed about the app details.

Extension Points:

Alternate Flow: Copy Info:

- User long-presses the contact email.
- System copies the text to the clipboard.

Priority: LOW

Frequency: Less frequent in use.

Cross-Reference: SRS-137

Use Case ID: UC-45

Use Case Name: Change App Theme

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user customizes the visual appearance of the application by switching between Light and Dark modes to suit their lighting environment or preference.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to reduce eye strain (Dark mode) or improve visibility in bright light (Light mode).

System: Needs to persist visual preferences across app restarts.

Pre-Conditions:

1. System must be in running state.
2. User navigates to the Settings screen.

Main Success Scenario:

	User Action		System Response
1	User taps on the Appearance or Theme option in Settings.		
		2	System displays the current selection (e.g. Light) and a toggle switch or radio buttons.
3	User selects the alternative theme (e.g. switches from Light to Dark).		
		4	System immediately updates the application's colors, backgrounds, and text styles to match the Dark Theme.
		5	System saves the selected theme preference locally so it remains active for future sessions.
6	User navigates back to other screens.		
		7	System maintains the selected theme throughout the application.

Post-Conditions: The application UI is updated, and the preference is stored. If the user closes and reopens the app, it will load in the selected theme.

Extension Points:**3a. System Default:**

- User selects "System Default".
- System detects the device's OS setting (Android Dark/Light mode).
- System matches the app theme to the OS setting automatically.

4a. UI Glitch:

- System fails to update a specific icon or text color immediately.
- System refreshes the current screen to correct the display.

Priority: LOW

Frequency: Less frequent in use.

Cross-Reference: SRS-138, SRS-139

Use Case ID: UC-46

Use Case Name: Configure Notification Settings

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user personalizes the alert experience by changing notification styles, selecting custom sounds, and configuring recurrence preferences.

Primary Actors: <Patient>

Stakeholders & Interest:
User: Wants alerts that are distinct (sound) and easy to read (size) without being annoying.
System: Needs to apply these preferences to the local notification manager.

Pre-Conditions:

1. System must be in running state.
2. User navigates to the Settings screen.

Main Success Scenario:

	User Action		System Response
1	User taps on Notifications within the Settings menu.		
		2	System displays options for Appearance, Sound, and Schedule Defaults.
3	User taps Appearance and selects Large Alert (full screen) or Standard (banner).		
		4	System previews the selected style.
5	User taps Notification Sound.		
		6	System lists available system ringtones and custom app sounds.
7	User selects a specific sound (e.g., Chime).		
		8	System plays a short audio sample.
9	User configures Recurring Settings (e.g., enabling Repeat Daily as a default option).		
10	User taps Save or navigates back.		
		11	System saves the frequency preference.

		12	System persists the preferences locally.
--	--	----	--

Post-Conditions: Future notifications will use the selected sound and visual style.

Extension Points:

3a. Permission Denied:

- User selects “Large Alert” (which requires “Draw over other apps” permission).
- System requests the permission.
- User grants it.

5a. Silent Mode:

- User selects “None” or “Silent”.
- System warns: “You may miss reminders in Silent mode”.

Priority: MEDIUM

Frequency: Less frequent in use.

Cross-Reference: SRS-140, SRS-141, SRS-142

Use Case ID: UC-47

Use Case Name: Detect & Adjust Time Zone Automatically

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how the system automatically detects changes in the device's time zone (e.g. during travel) and adjusts pending medication reminders to ensure they trigger at the correct local time.

Primary Actors: <Patient><System>

Stakeholders & Interest: **User:** Wants to take medication at the correct “wall-clock” time (e.g. 8 AM) regardless of location.
System: Needs to resync the local notification schedule to match the new device time.

Pre-Conditions: 1. System must be in running state.
2. User device settings allow Automatic Date & Time.

Main Success Scenario:

	User Action		System Response
1	User travels to a different time zone (or manually changes device time zone).		
		2	System detects the device's current time zone change automatically.
		3	System retrieves the list of pending reminders.
		4	System recalculates the trigger times relative to the new local time.
		5	System adjusts reminder notifications to ensure they ring at the user-defined time (e.g. 8:00 AM) in the new time zone.
		6	System displays a toast/notification: Time zone updated. Reminders adjusted.

Post-Conditions: All future reminders are synchronized with the new local time.

Extension Points:**4a. Skipped Dose (Time Jump):**

- The time zone change moves the clock forward past a scheduled dose (e.g. 1 PM to 4 PM, skipping a 2 PM dose).
- System detects a missed alarm due to the shift.
- System triggers a notification: "You may have missed your 2 PM dose due to time zone change".

4b. Duplicate Dose (Time Fallback):

- The time zone change moves the clock backward (e.g. 2 PM back to 11 AM).
- System ensures the 8 AM dose (already taken) does not ring again for the "same" day, preventing double dosing.

Priority: MEDIUM

Frequency: Less frequent in use.

Cross-Reference: SRS-143, SRS-144

Use Case ID: UC-48

Use Case Name: View Medicine Consumption Report

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how a user views detailed medication adherence reports, filters them by time or medicine, and identifies irregular intake patterns.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants to identify adherence gaps or trends over time.
System: Needs to query historical data and apply logic to detect irregularities.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. Medication logs exist in the database.

Main Success Scenario:

	User Action		System Response
1	User navigates to the Analytics section.		
		2	System retrieves medication history from the local database.
		3	System displays the number of doses taken for each medicine.
		4	System analyzes timestamps to identify medicines with irregular intake (e.g. frequently missed or taken late).
		5	System highlights these irregular medicines visually (e.g. with an amber warning icon or red text).
6	User taps the Filter option.		
		7	System displays filter criteria: Medicine Name and Time Period (Daily, Weekly, Monthly).

8	User selects a specific time period (e.g. Monthly).		
		9	System refreshes the report to show data only for the selected period.

Post-Conditions: The user views a filtered, analyzed report of their medication history.

Extension Points:

4a. No Irregularities:

- System analysis finds 100% consistent adherence.
- System displays a “Perfect Adherence” badge instead of warning highlights.

6a. Specific Medicine Filter:

- User filters by a specific medicine name (e.g. “Insulin”).
- System hides all other medicines and shows the detailed timeline for that specific drug only.

Priority: MEDIUM

Frequency: Most frequent in use.

Cross-Reference: SRS-145, SRS-146, SRS-147

Use Case ID: UC-49

Use Case Name: View Misses Doses Report

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how the system generates a specific report highlighting non-adherence, showing exactly which doses were missed, when they were missed, and the total count for the selected period.

Primary Actors: <Patient>

Stakeholders & Interest:

User: Wants to understand patterns in their forgetfulness to improve future adherence.

System: Needs to track negative compliance data for accurate health scoring.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. The database must contain logs where the status is recorded as missed.

Main Success Scenario:

	User Action		System Response
1	User navigates to the Missed Doses section within Reports or Analytics.		
		2	System queries the local database for medication logs with the status Missed.
		3	System aggregates the data to calculate the total count of missed doses for the current/selected period.
		4	System generates a detailed list displaying: • Medicine Name • Date & Time of the missed dose.
		5	System displays the Total Missed Count prominently at the top of the screen.
6	User reviews the list to identify specific days or times where they struggled.		

Post-Conditions: The user is informed of their specific non-adherence instances.

Extension Points:**2a. No Missed Doses:**

- System finds no logs with “Missed” status.
- System displays a “Great Job!” or “Perfect Adherence” message.
- Total count is shown as **0**.

Alternate Flow: Filter by Date:

- User changes the view from “Weekly” to “Monthly”.
- System recalculates the total count and refreshes the list for the new range.

Priority: MEDIUM

Frequency: Less frequent in use.

Cross-Reference: SRS-148, SRS-149

Use Case ID: UC-50

Use Case Name: Display Montly Summary

Use Case Prepared by: Atif Islam **Use Case Prepared On:** 26th Nov, 2025

Use Case Updated by: M.Awais Ali **Use Case Updated On:** 27th Nov, 2025

Use Case Description: This use case describes how the system aggregates 30 days of medication data to generate visual charts comparing successful intake versus missed doses, helping the user see long-term trends.

Primary Actors: <Patient>

Stakeholders & Interest: **User:** Wants a visual overview of their monthly performance.
System: Needs to render graphical charts (bar/pie) from statistical data.

Pre-Conditions:

1. System must be in running state.
2. User must be logged in.
3. Sufficient data (at least a few days) exists to generate a meaningful chart.

Main Success Scenario:

	User Action		System Response
1	User navigates to the Reports or Summary tab.		
2	User selects the Monthly View option.		
		3	System retrieves all medication logs for the current calendar month.
		4	System calculates the totals for Taken and Missed doses.
		5	System generates a visual chart (e.g. Bar Chart or Pie Chart) representing these statistics.
		6	System displays the chart with a legend (Green = Taken, Red = Missed).
7	User interacts with the chart (e.g. taps a bar to see the exact count).		

Post-Conditions: The user is presented with a graphical summary of their monthly health adherence.

Extension Points:

3a. New Month/No Data:

- System detects the current month has no logs yet.
- System displays an empty chart placeholder or text: “No data for this month yet”.

5a. Interactive Drill-down:

- User taps on the “Missed” section of the pie chart.
- System navigates to the “Missed Doses Report” for that specific month.

Priority: MEDIUM

Frequency: Less frequent in use.

Cross-Reference: SRS-150

CHAPTER # 3: DESIGN

SYSTEM SEQUENCE DIAGRAM

FOR

Personal Medicine Reminder

VERSION 1.0

Prepared by

Atif Islam

&

Muhammad Awais Ali

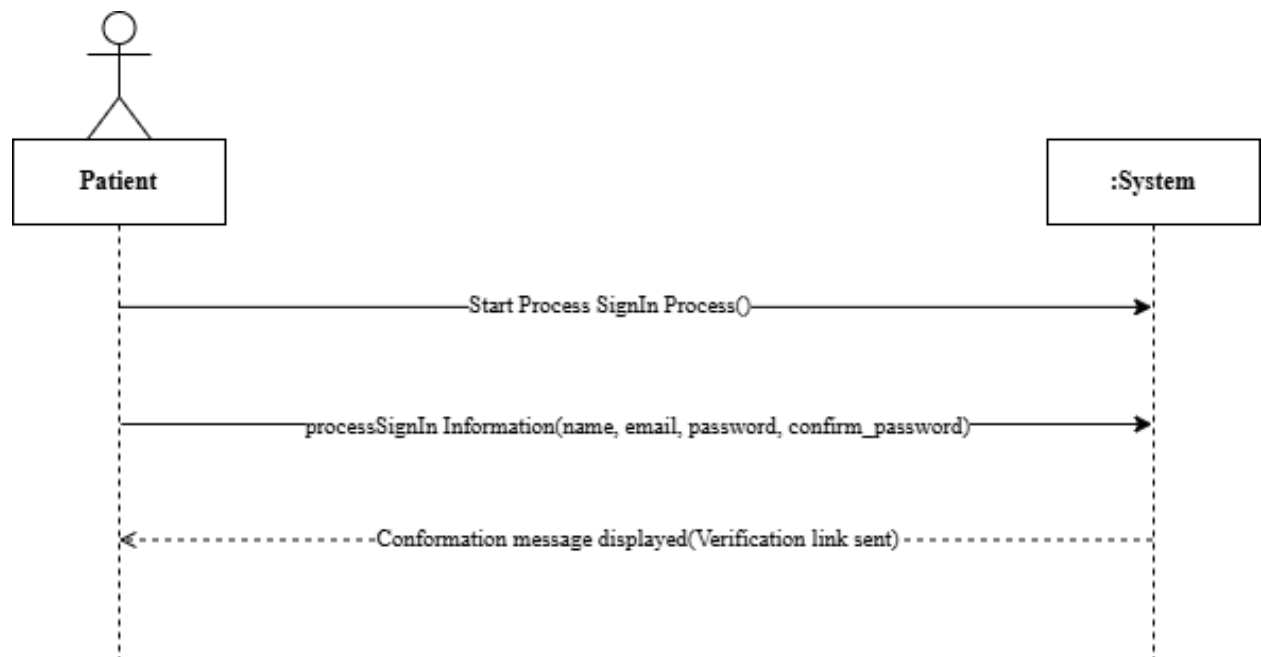
4th Jan, 2026

Revision History

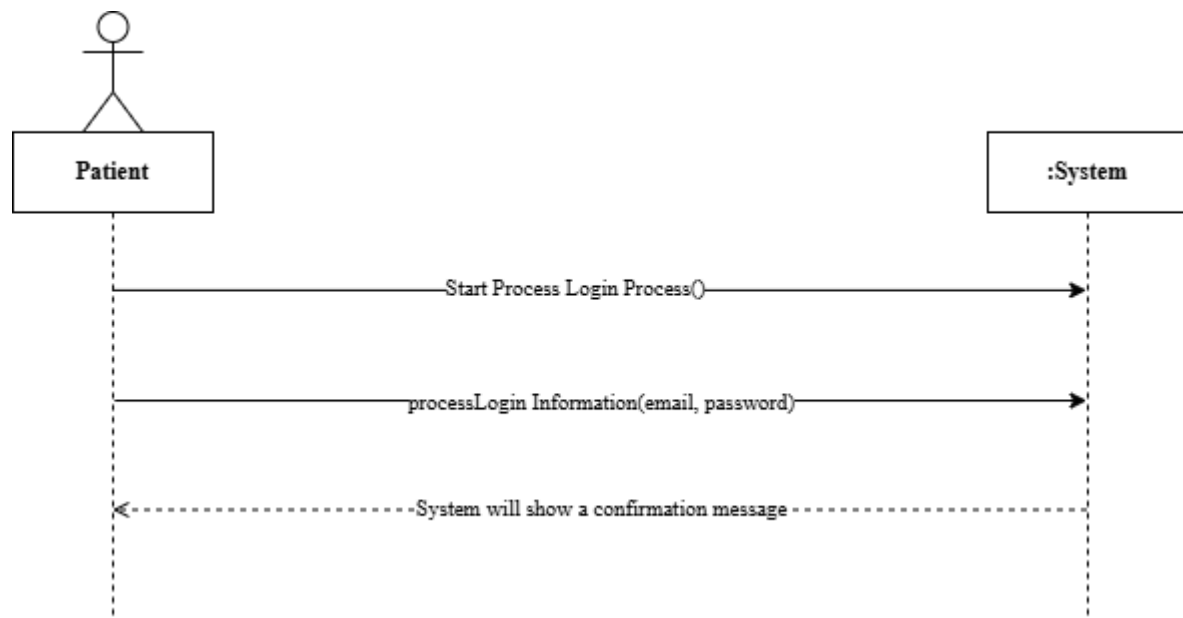
Version	Description	Author	Date
1.0	This contains the system sequence diagram for Personal Medicine Reminder App	Atif Islam & Muhammad Awais Ali	4 th Jan, 2026

3.1 System Sequence Diagram

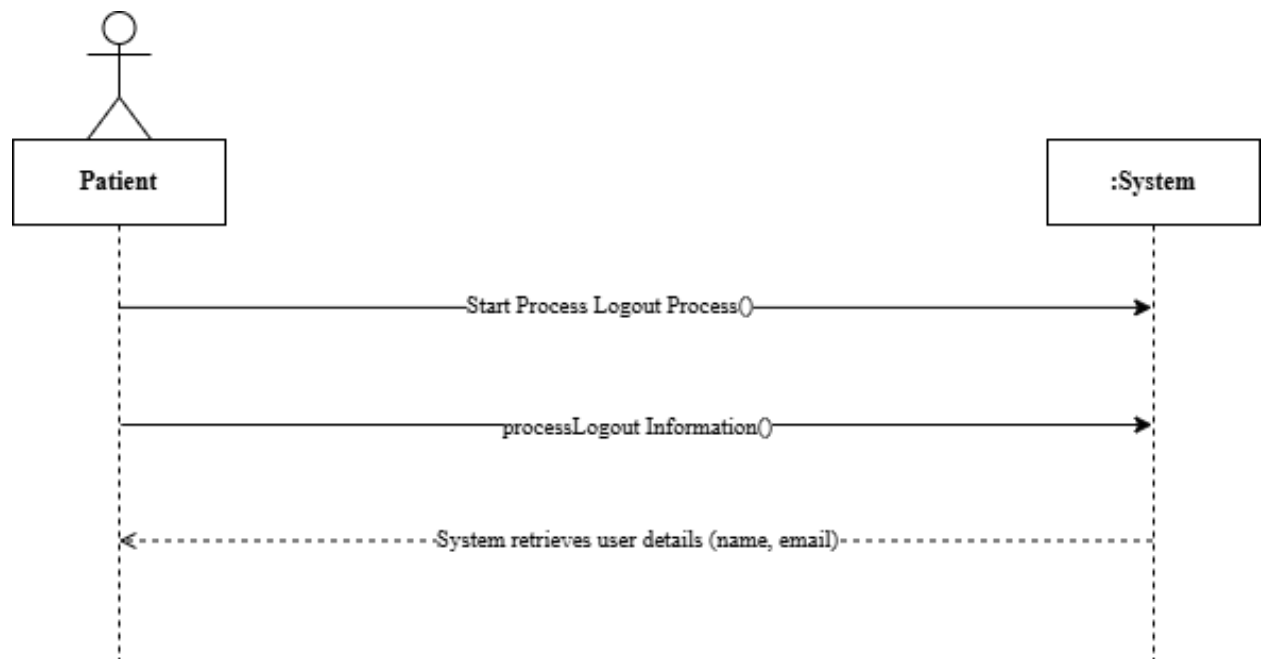
SSD 01: Process SignIn



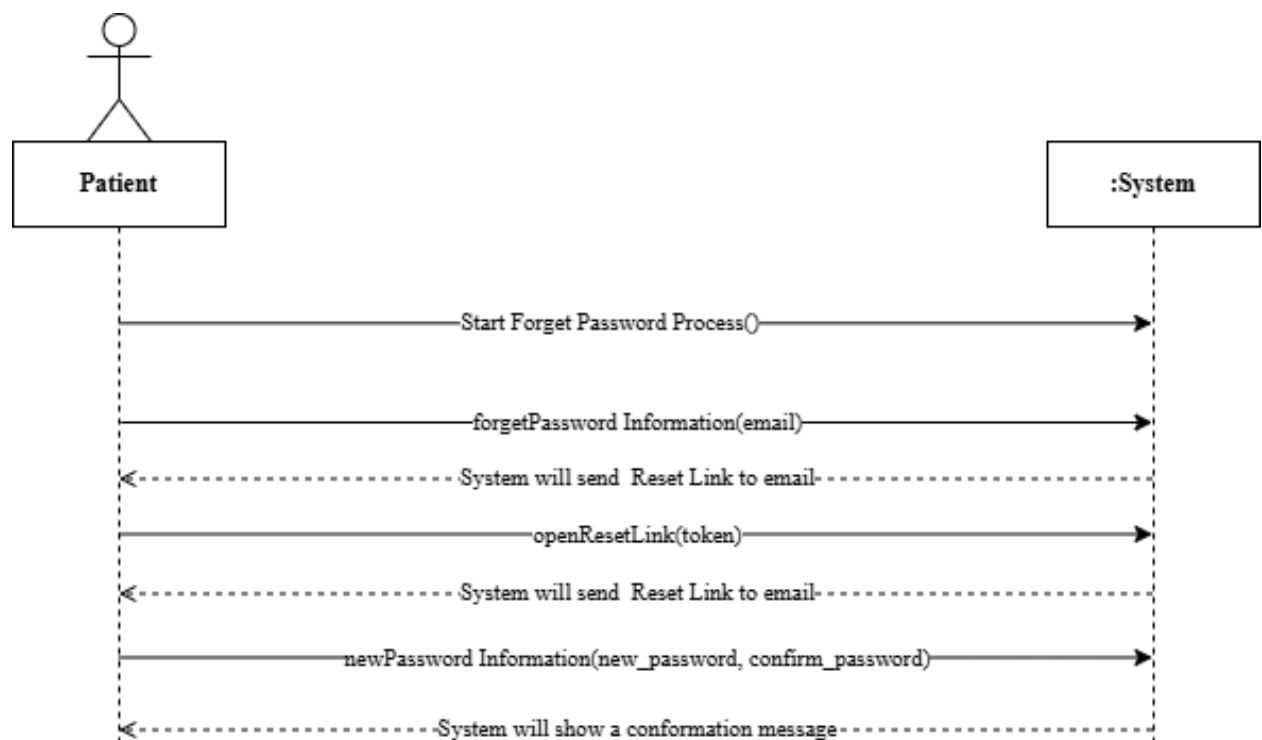
SSD 02: Process Login



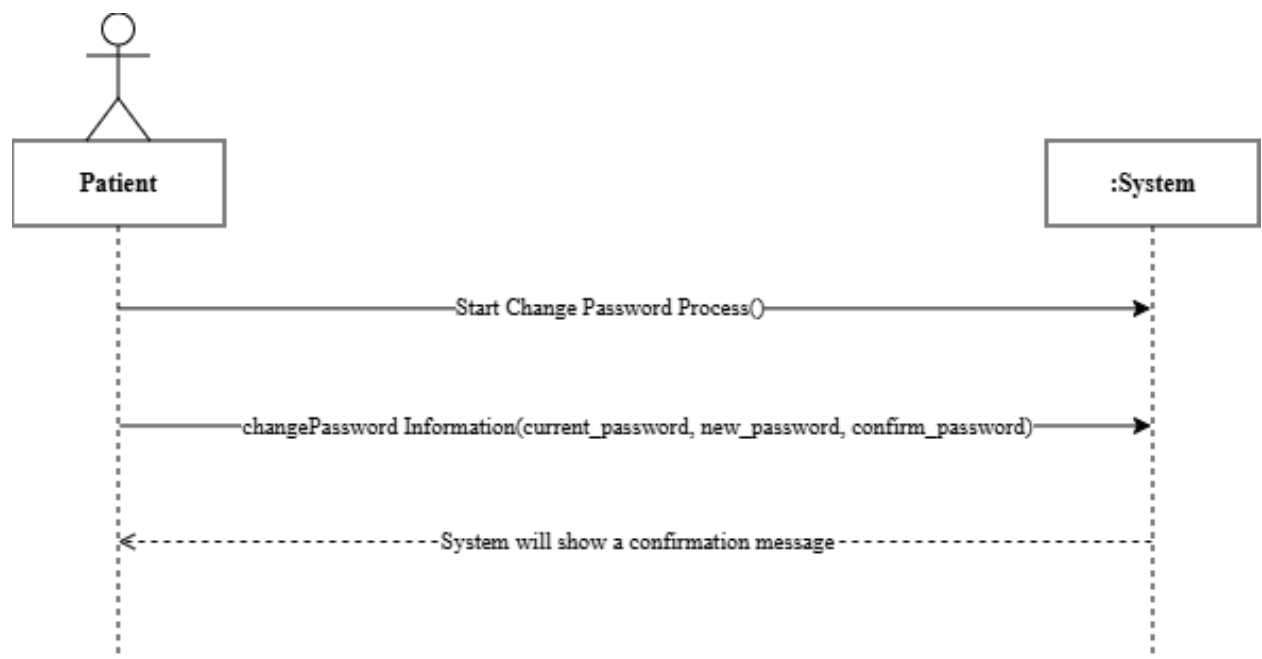
SSD 03: Process Logout



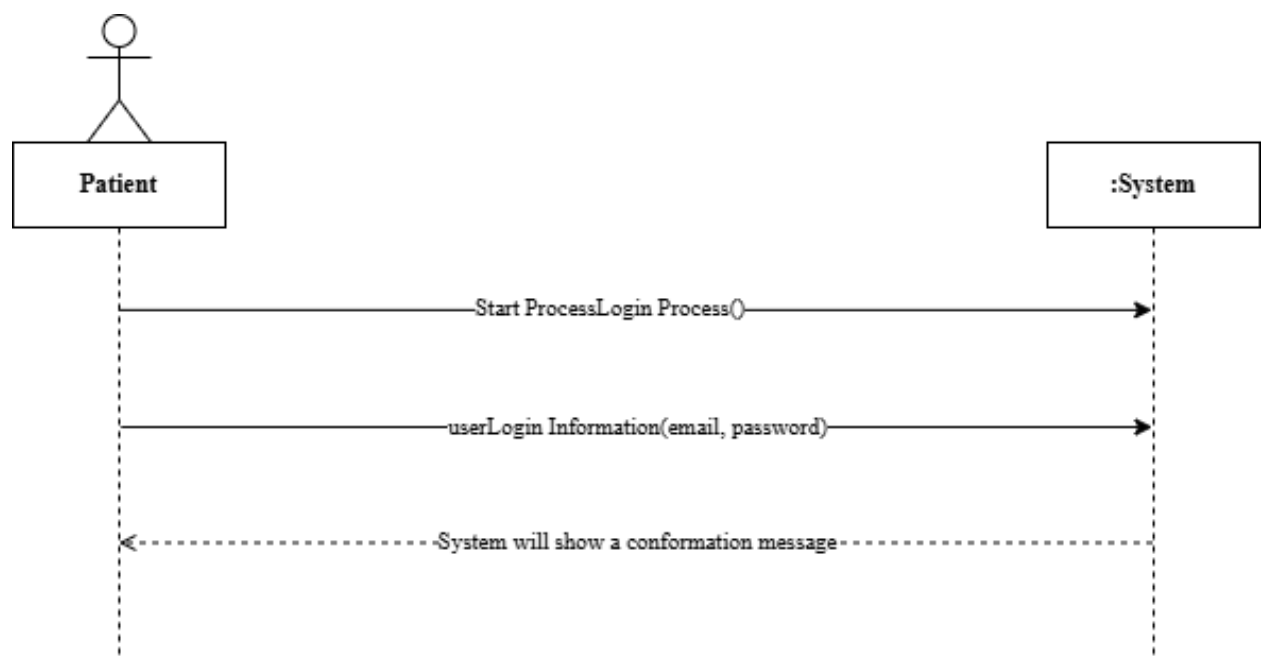
SSD 04: Forget Password



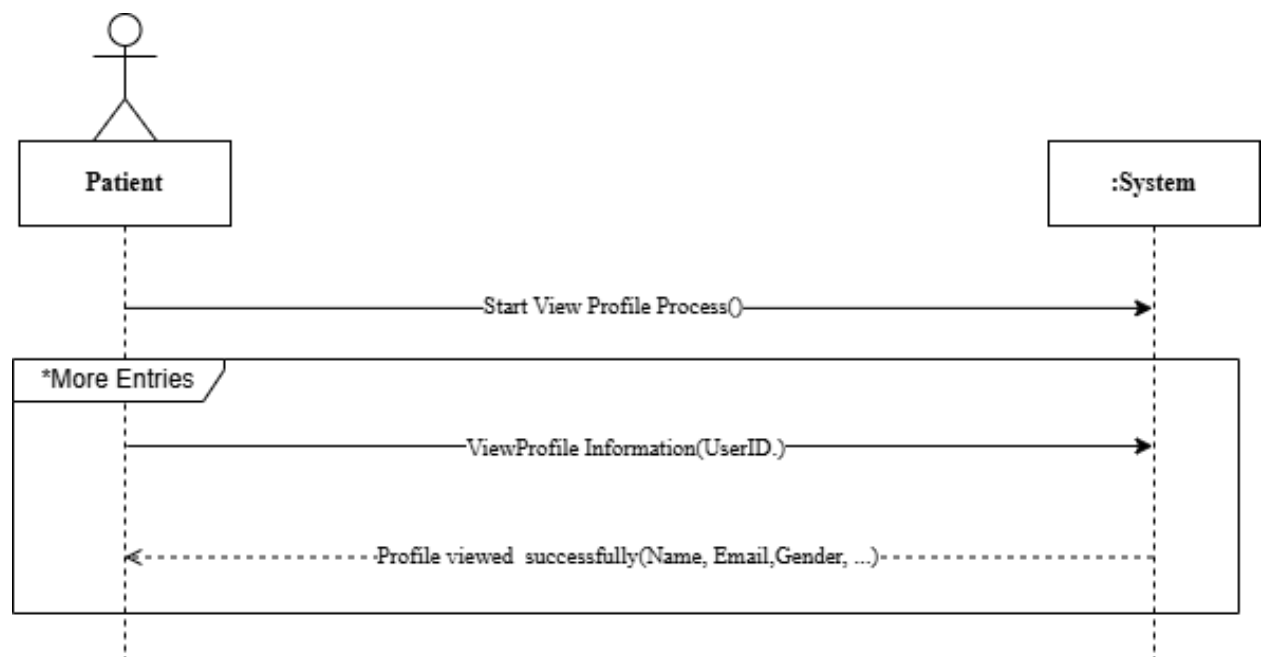
SSD 05: Change Password



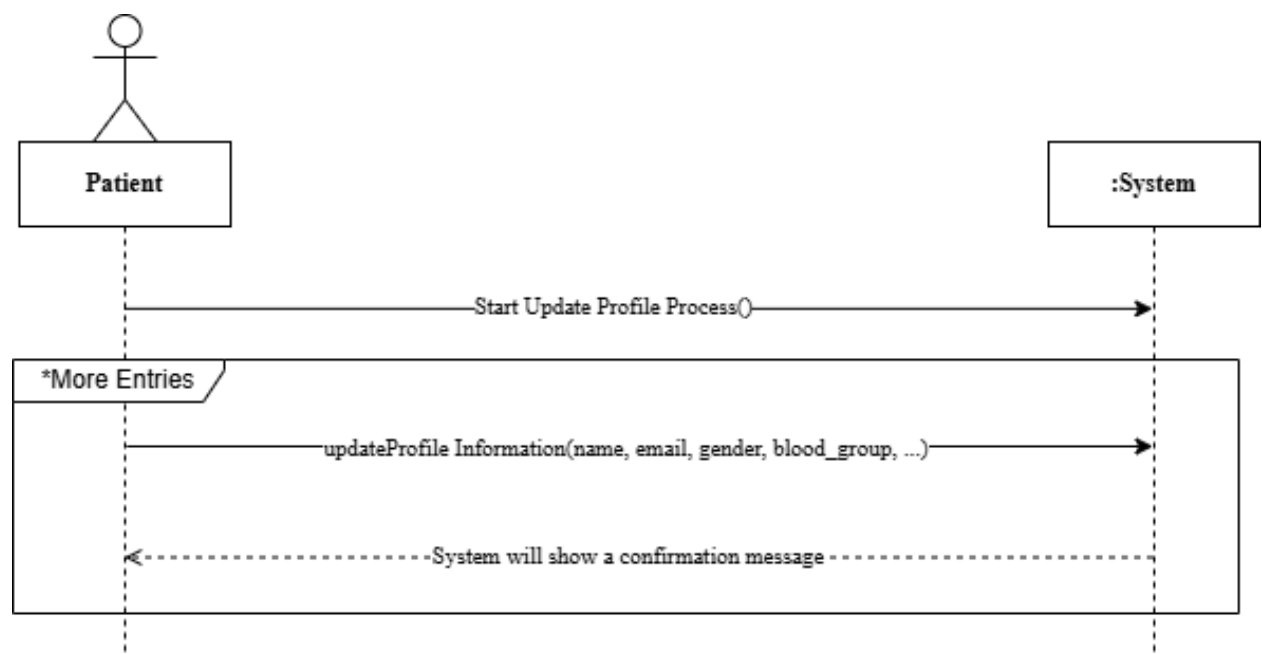
SSD 06: Process SocialLogin



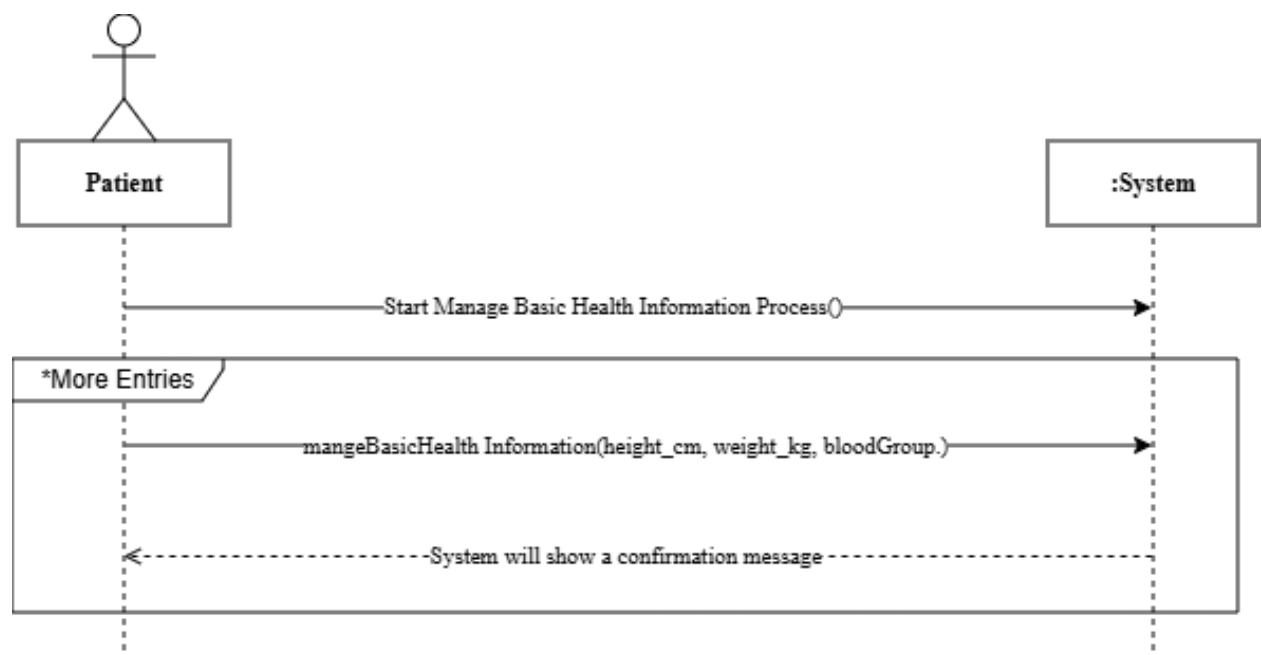
SSD 07: View Profile



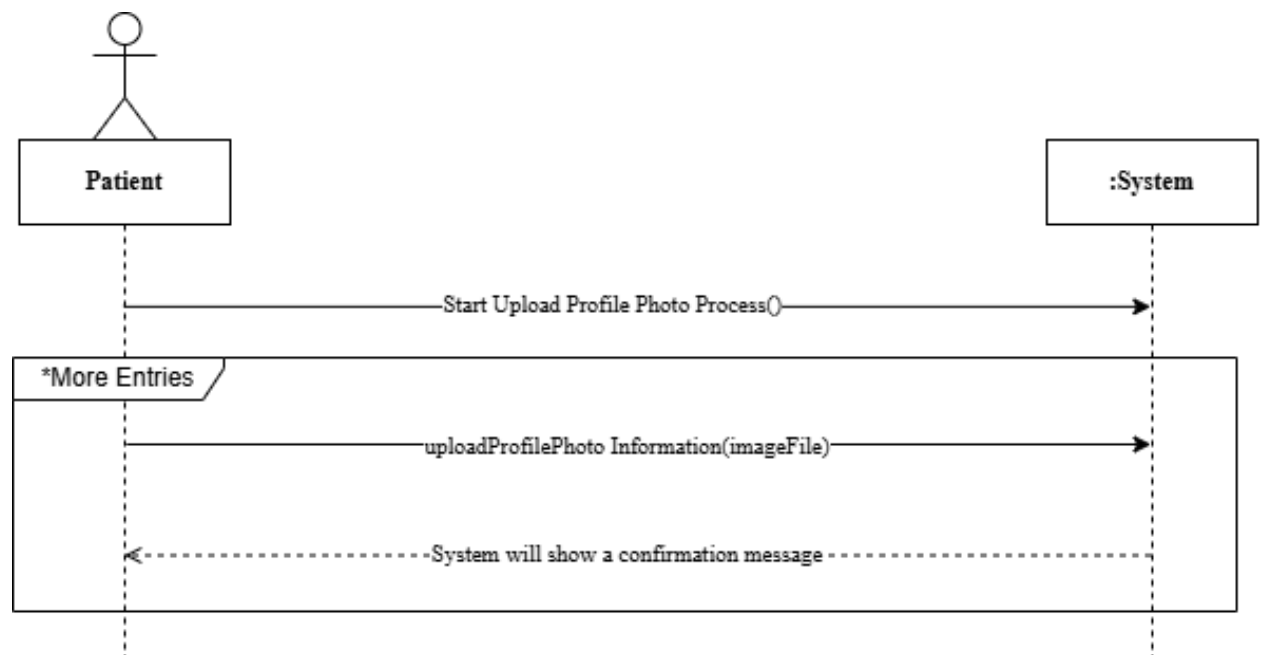
SSD 08: Update Profile



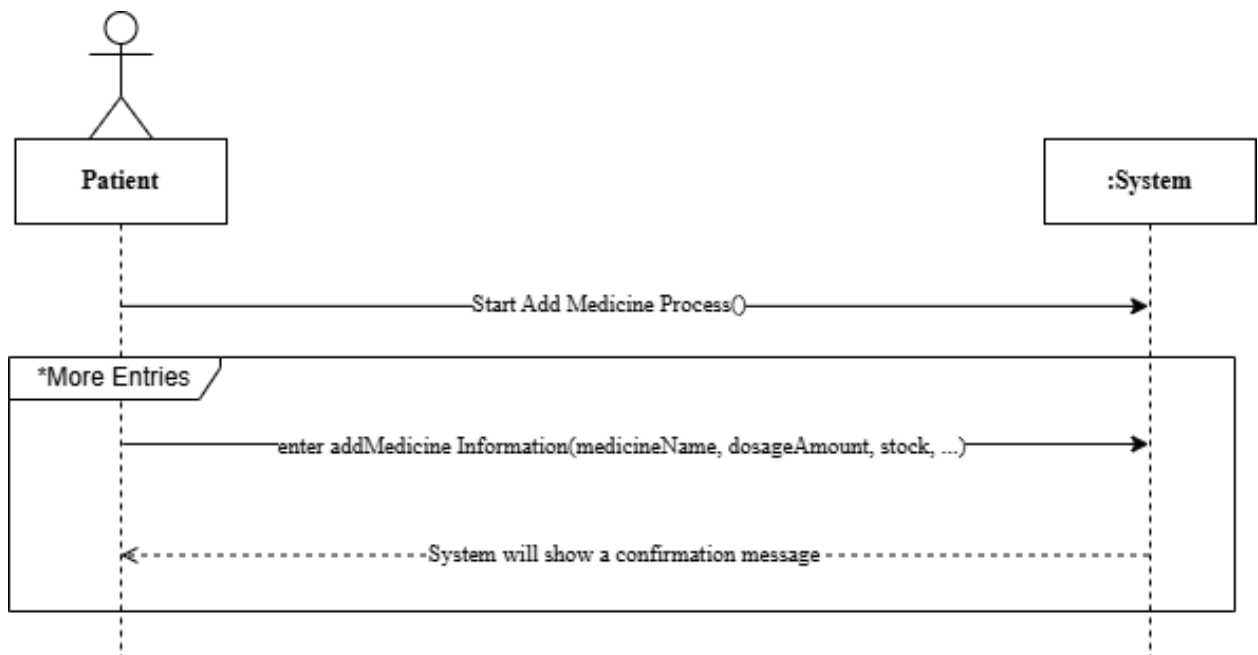
SSD 09: Manage Basic Health Information



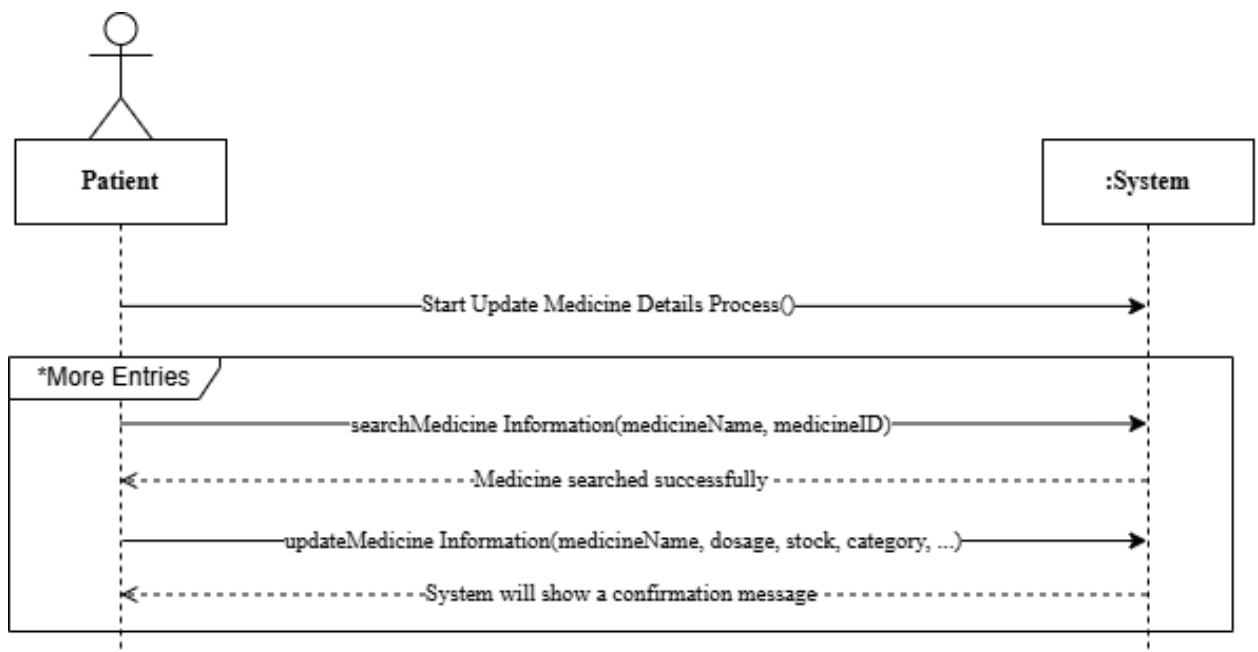
SSD 10: Upload Profile Photo



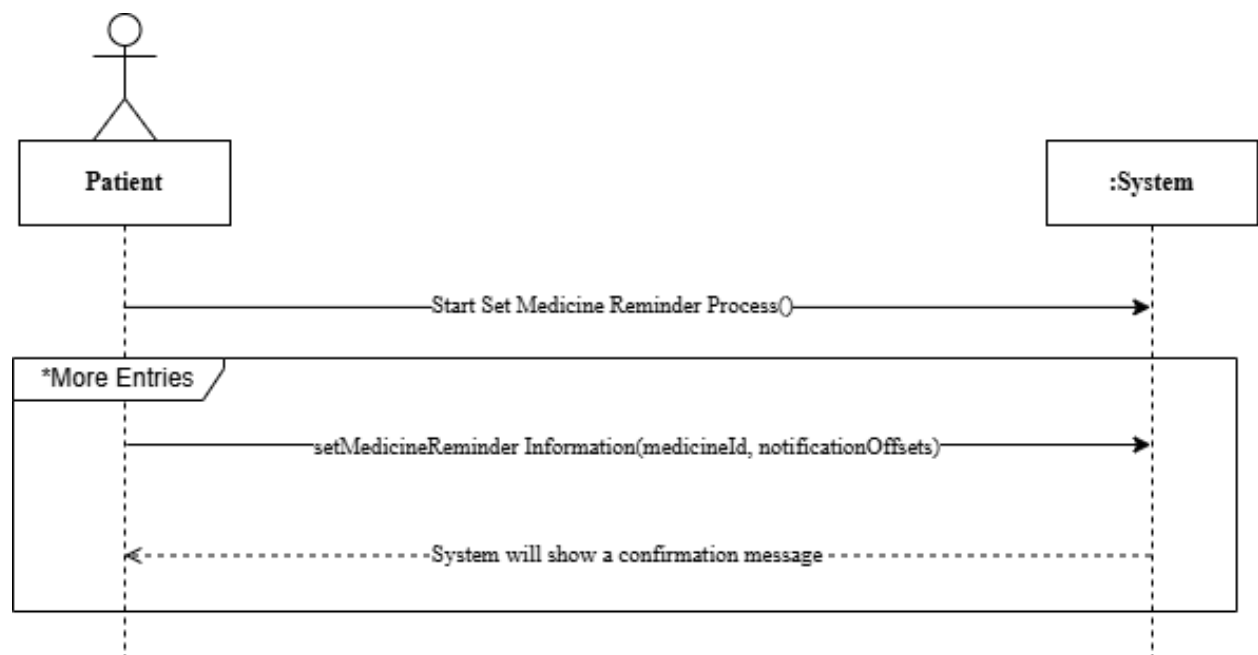
SSD 11: Add Medicine



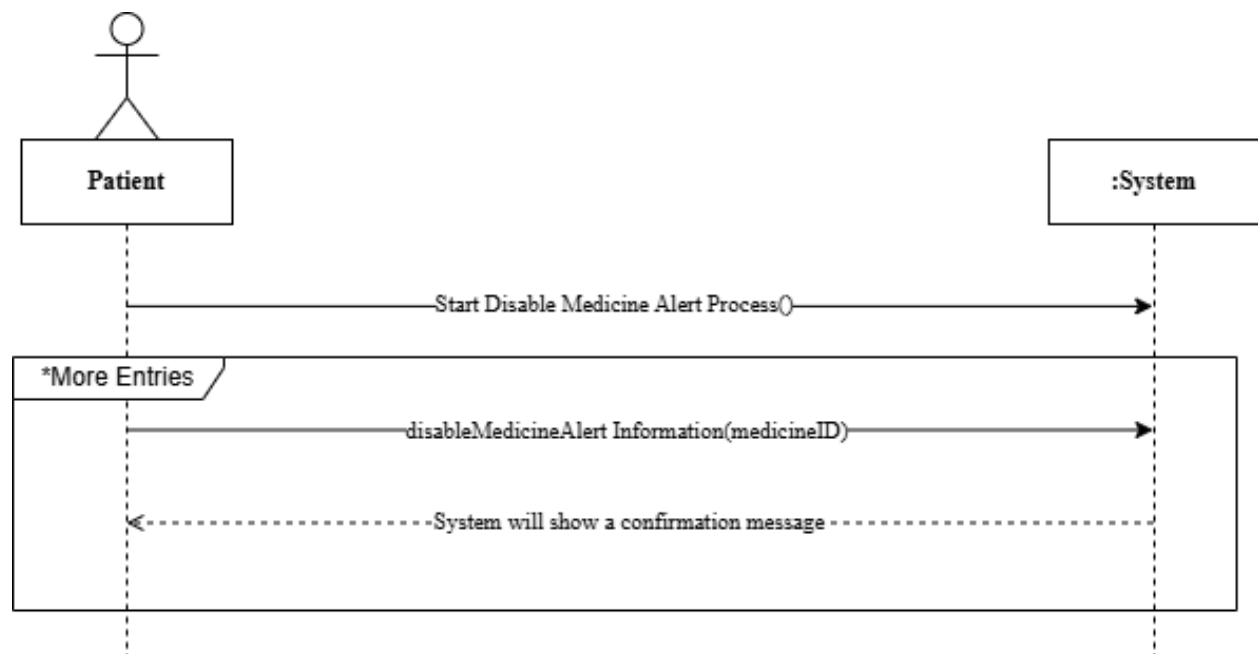
SSD 12: Update Medicine Details



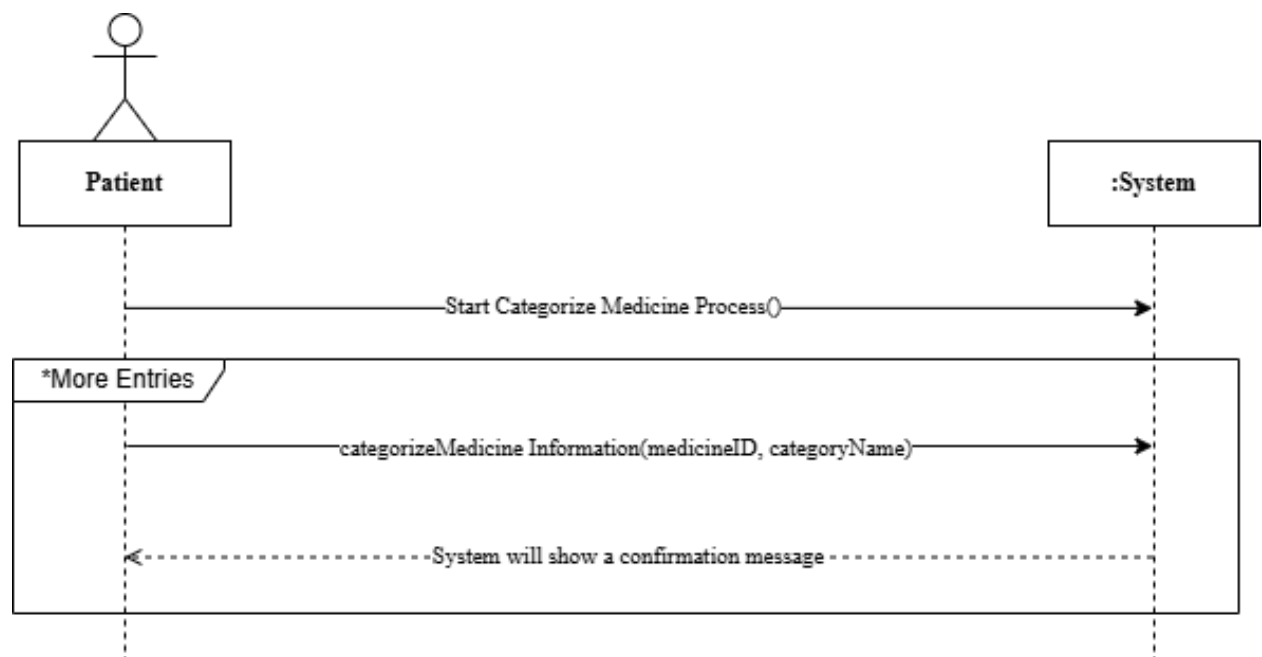
SSD 13: Set Medicine Reminder



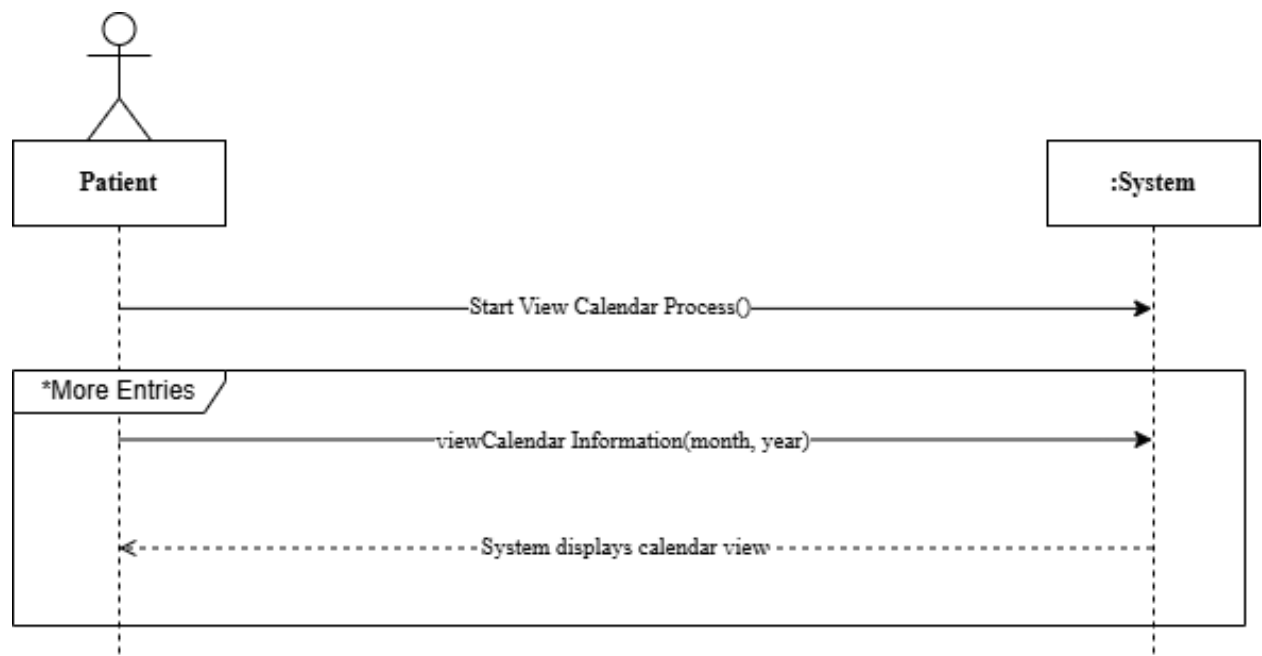
SSD 14: Disable Medicine Reminder



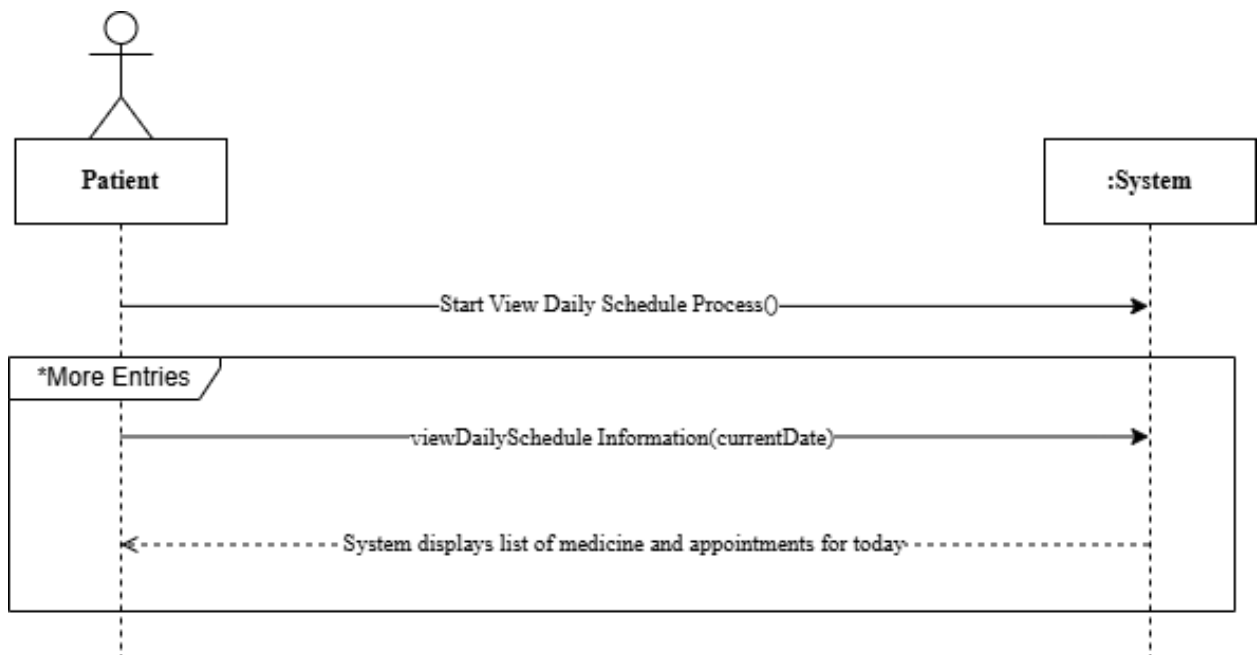
SSD 15: Categorize Medicine



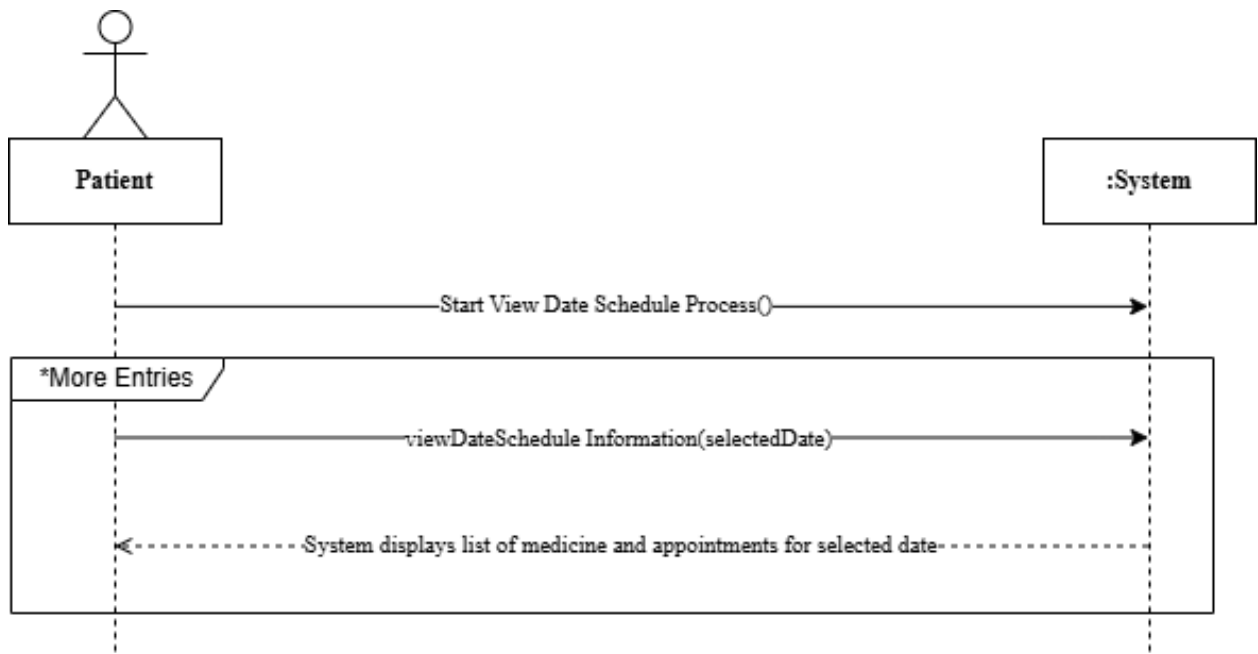
SSD 16: View Calendar



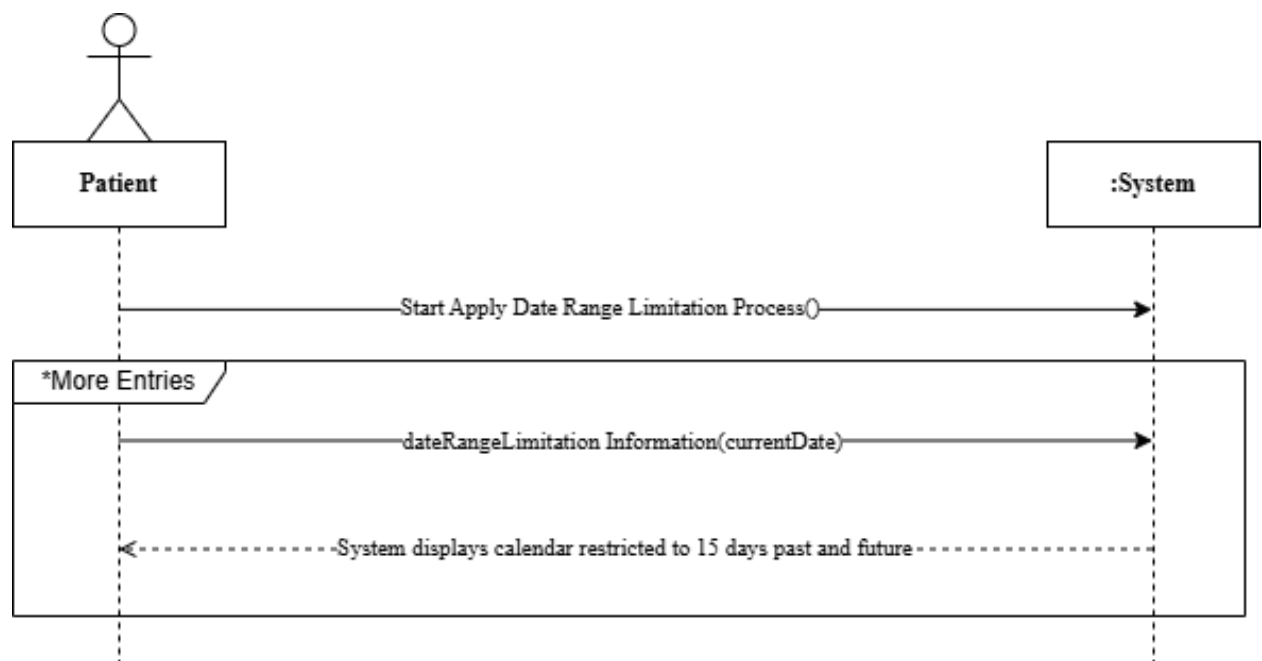
SSD 17: View Daily Schedule



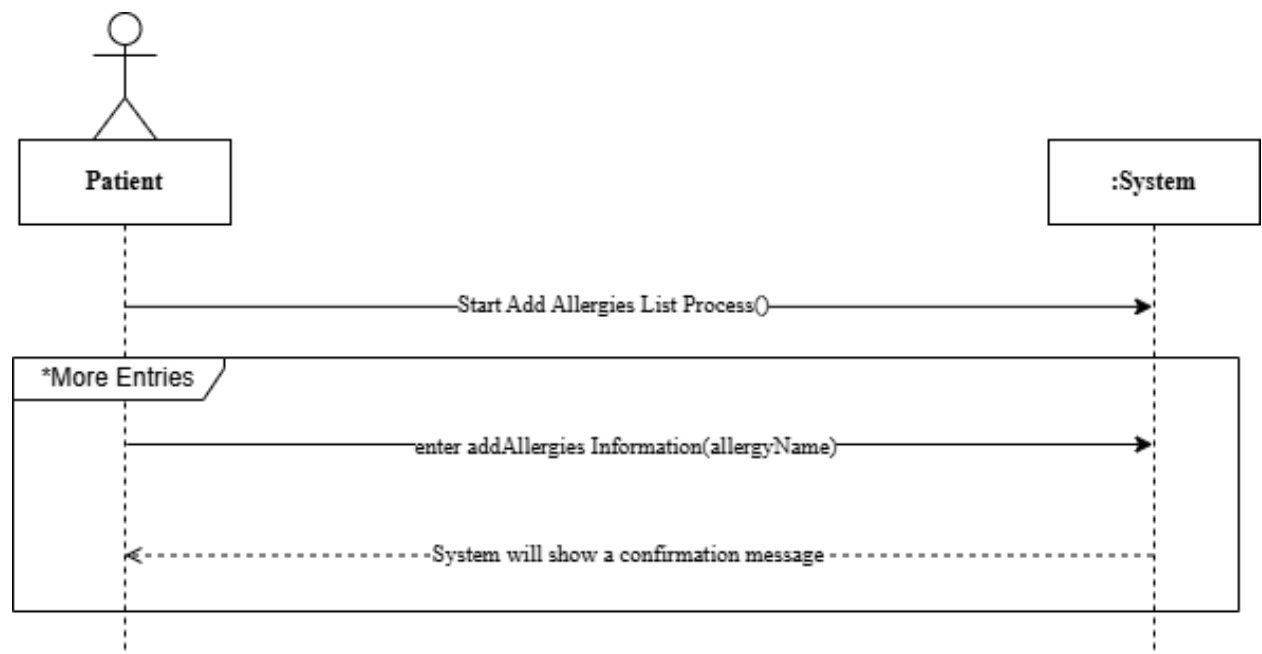
SSD 18: View Date Schedule



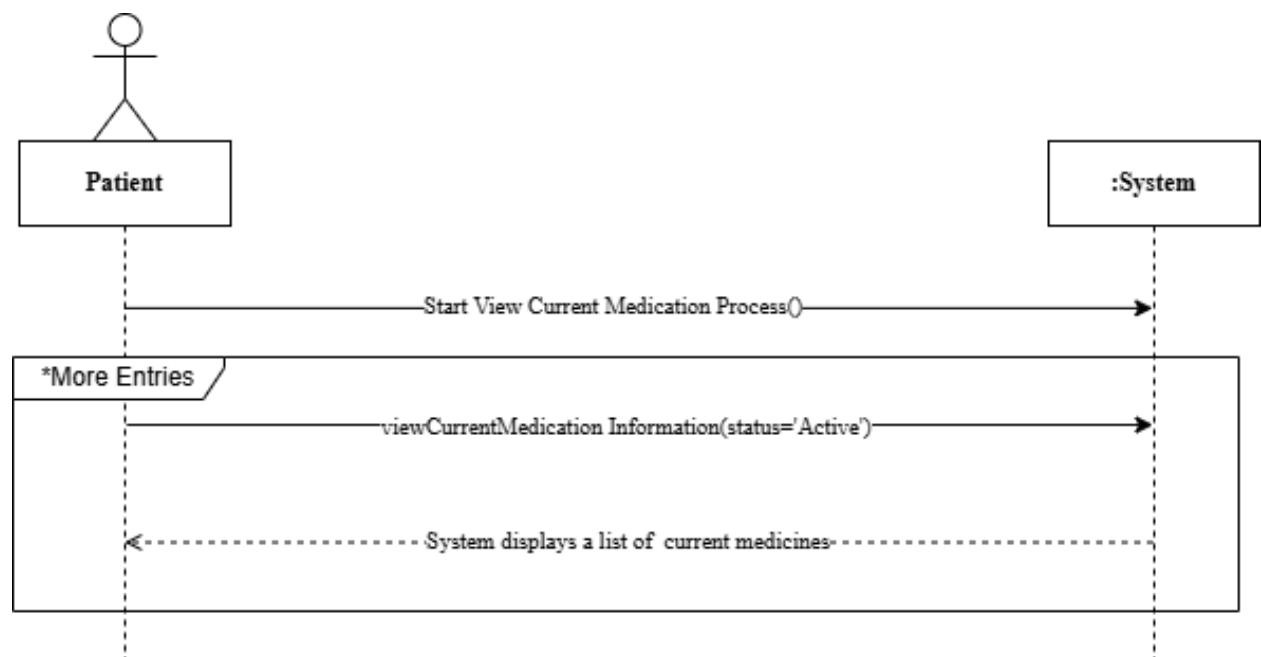
SSD 19: Apply Date Range Limitation



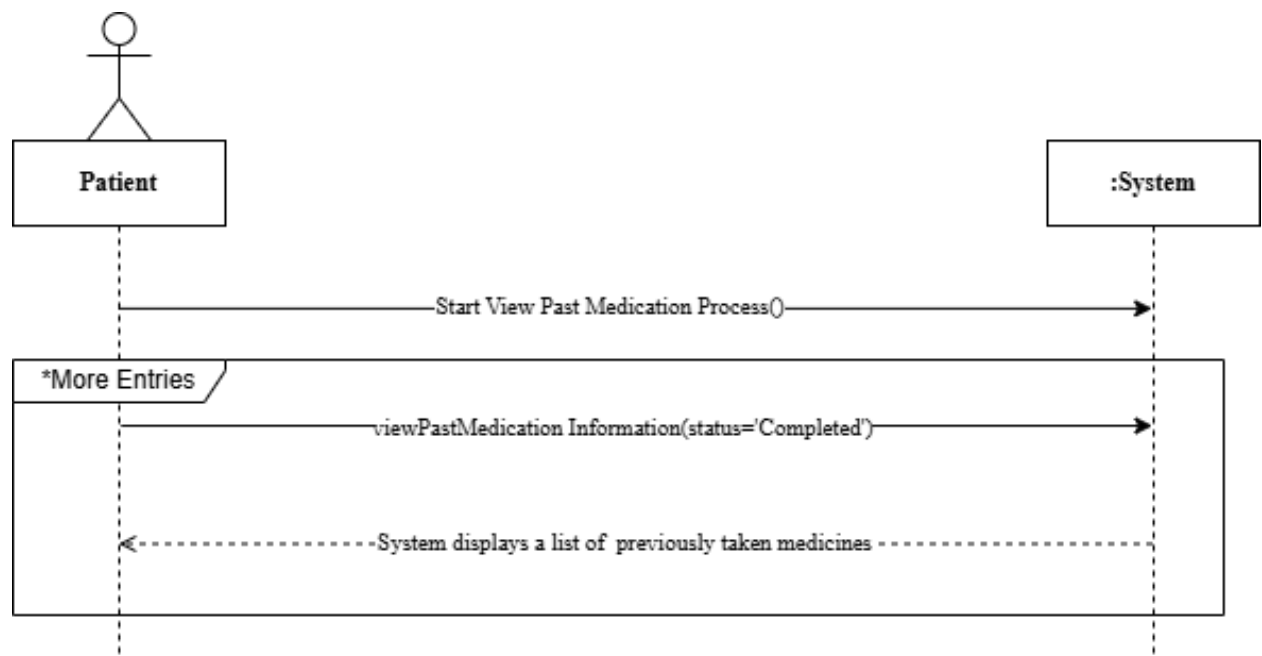
SSD 20: Add Allergies List



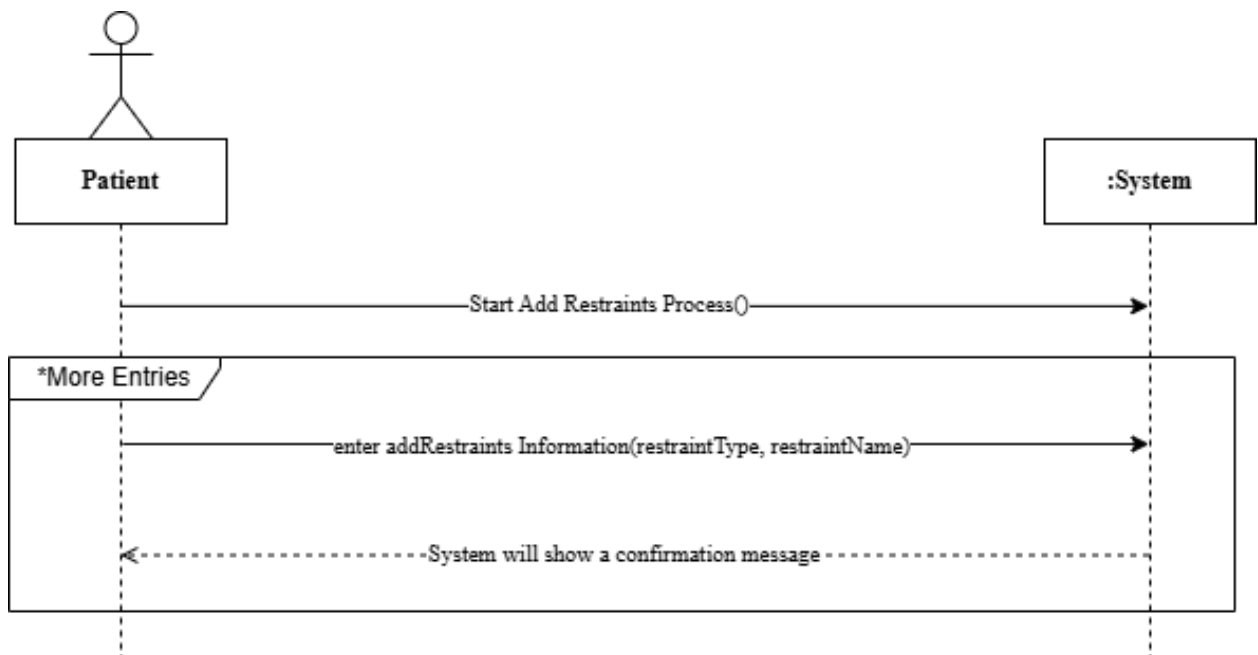
SSD 21: View Current Medication



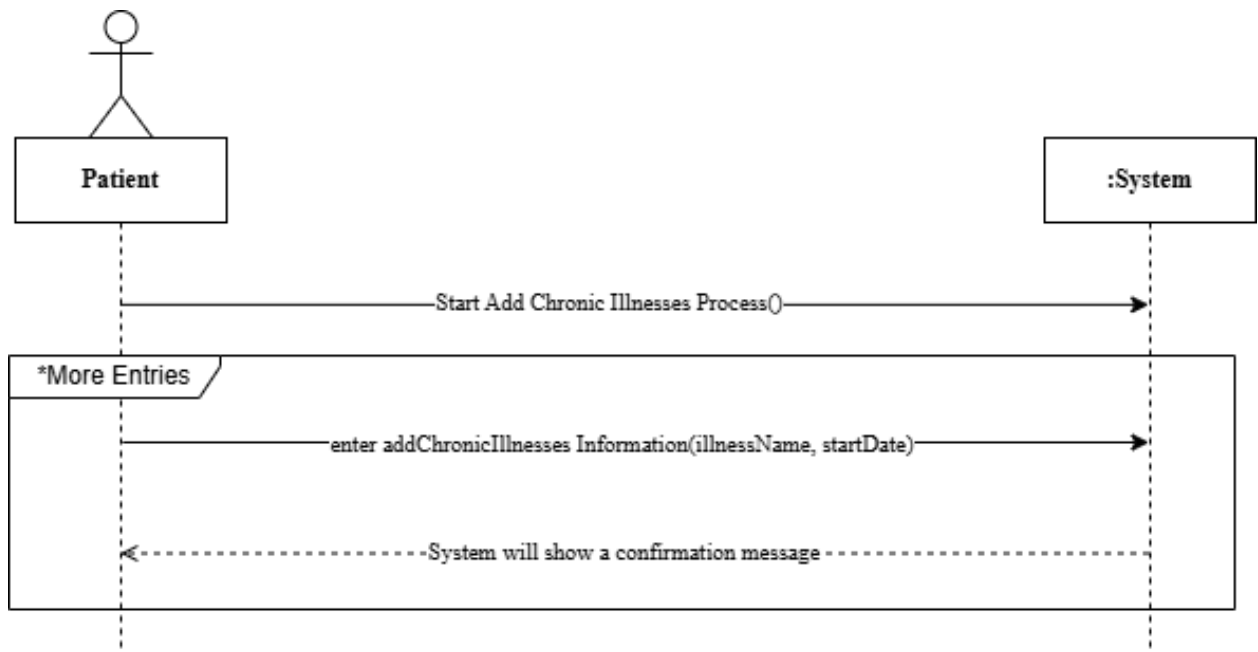
SSD 22: View Past Medication



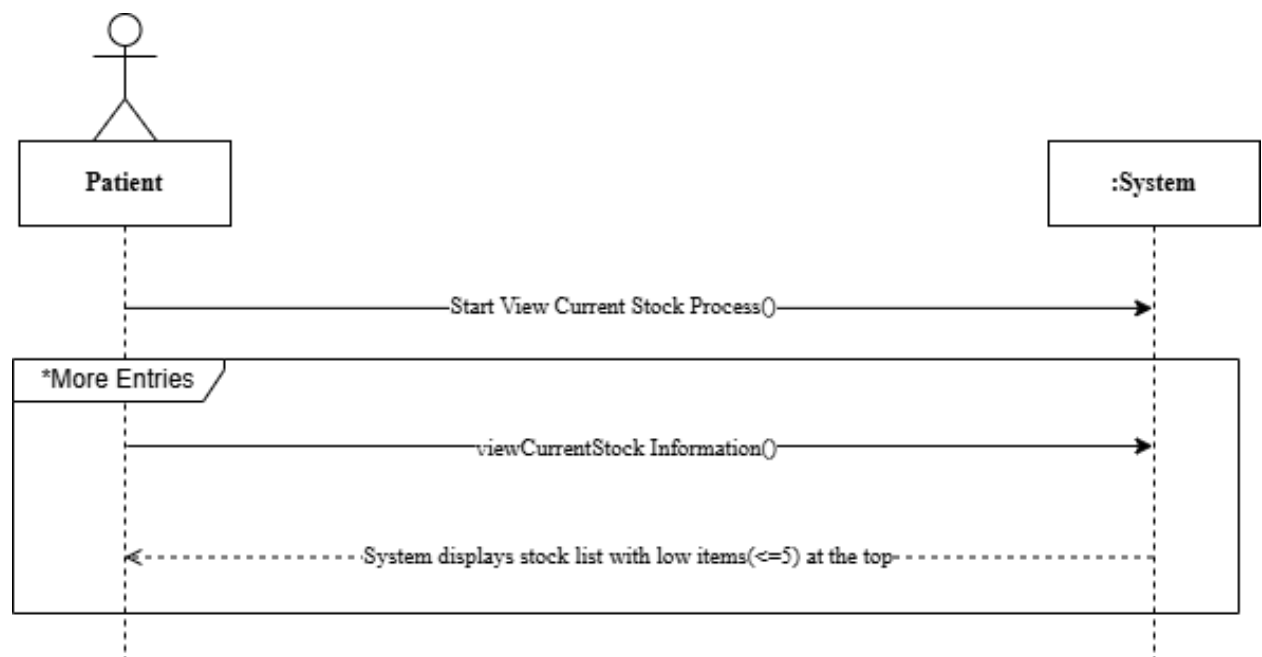
SSD 23: Add Restraints



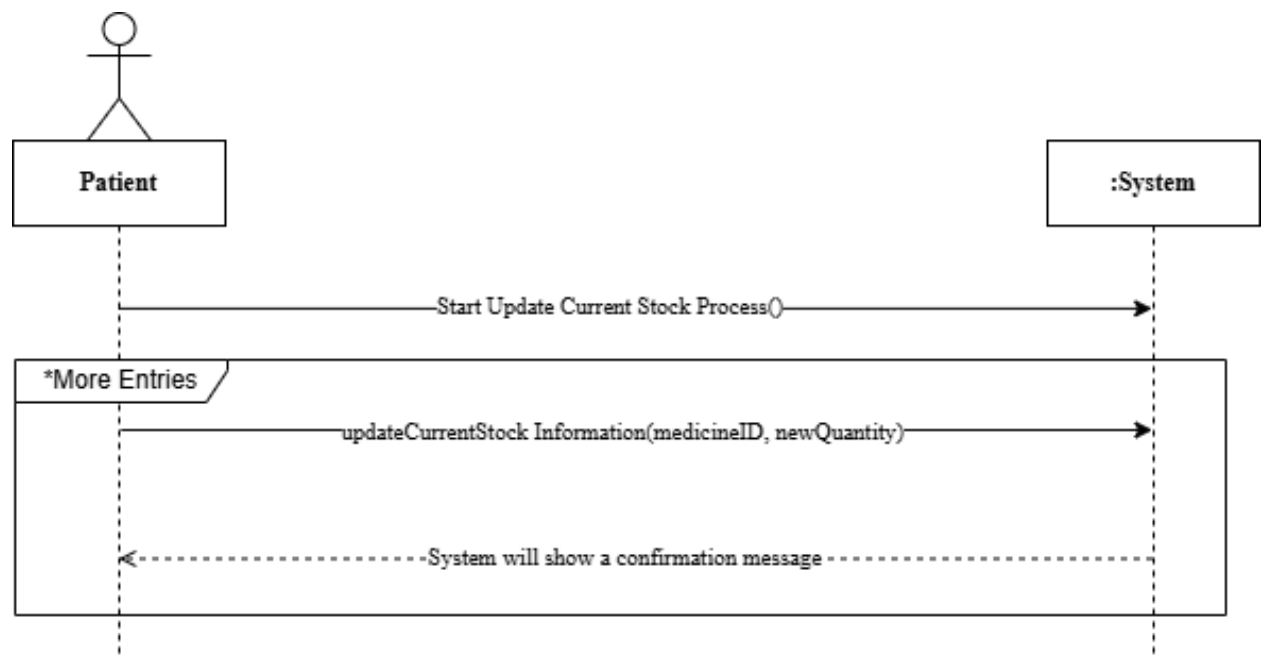
SSD 24: Add Chronic Illnesses



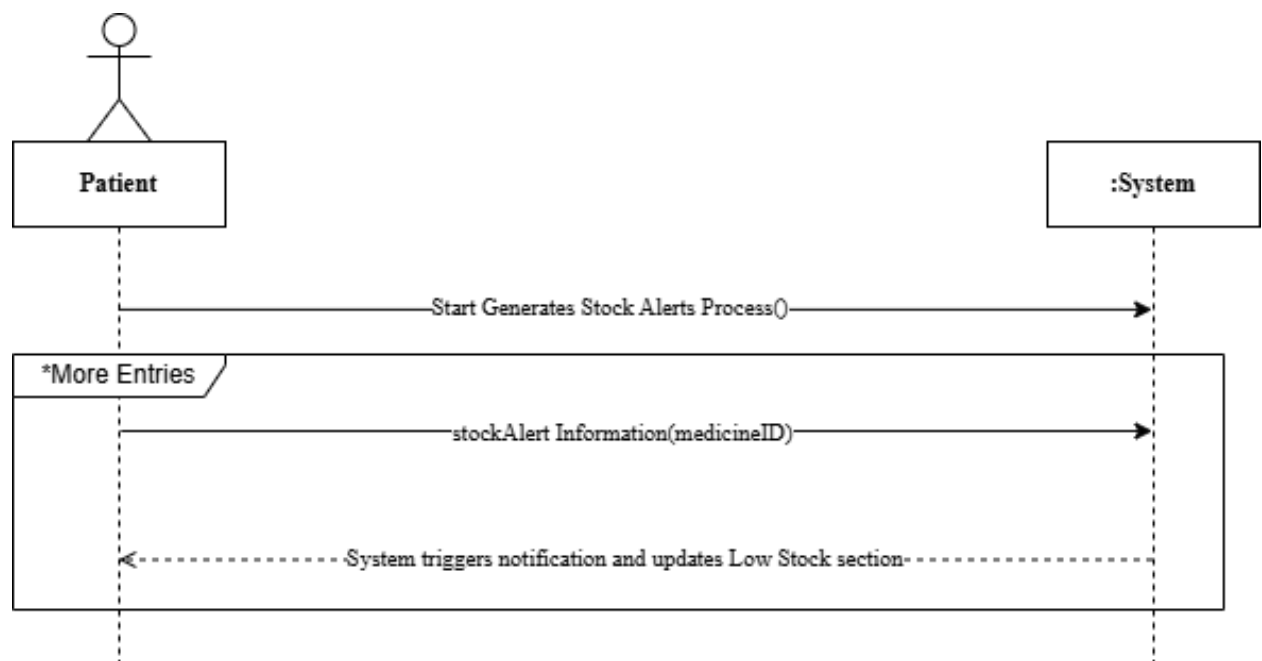
SSD 25: View Current Stock



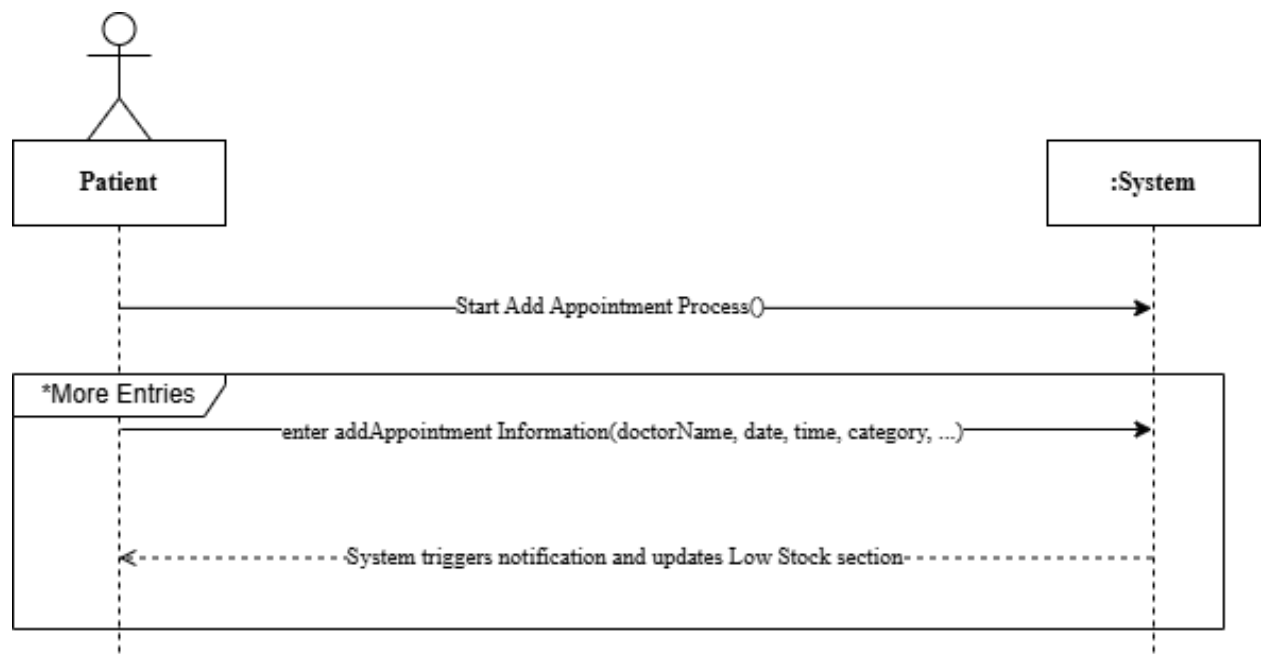
SSD 26: Update Current Stock



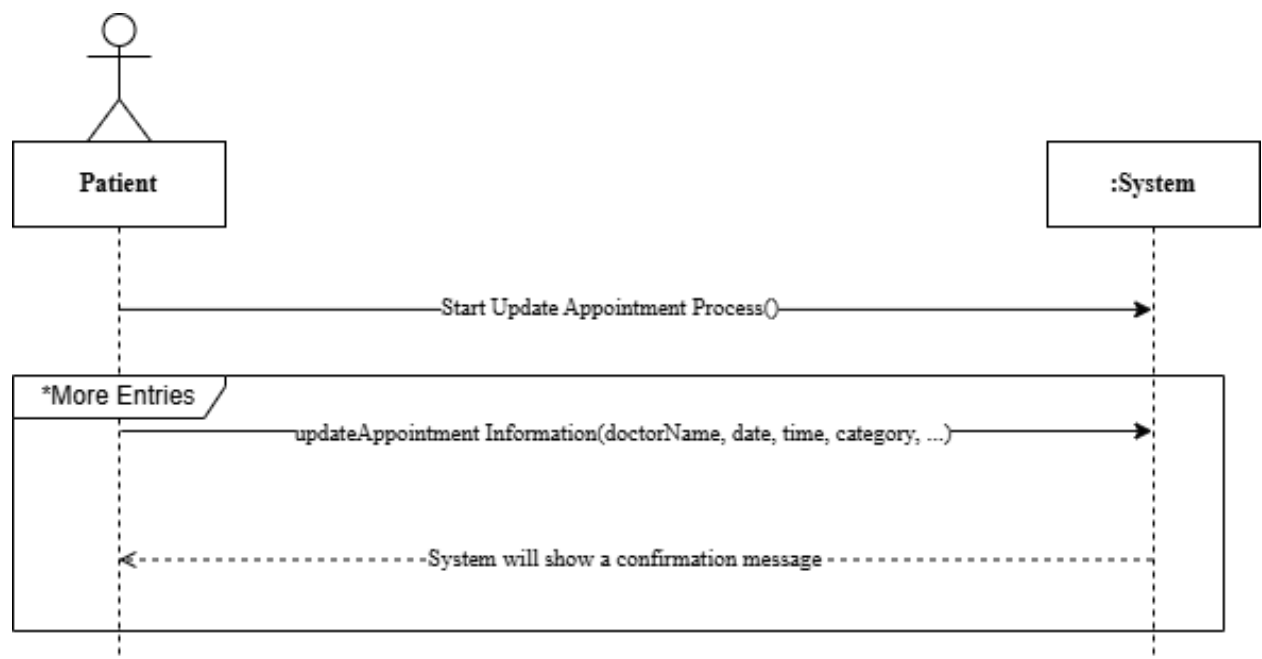
SSD 27: Generates Stock Alerts



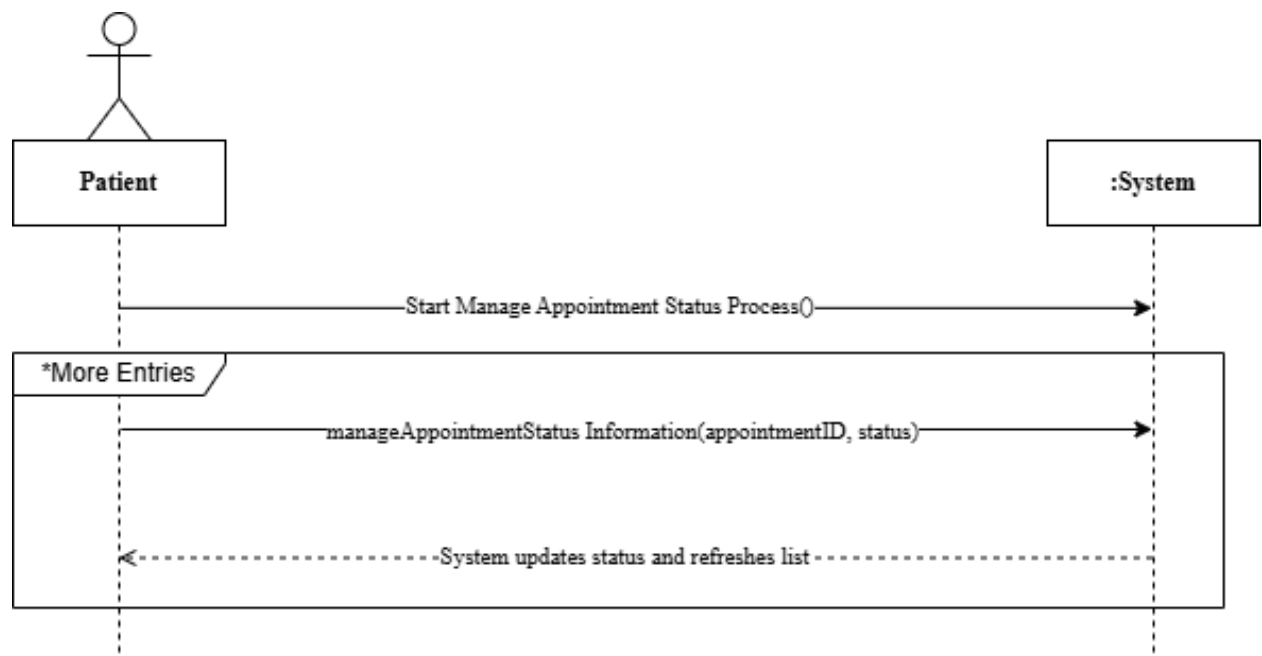
SSD 28: Add Appointment



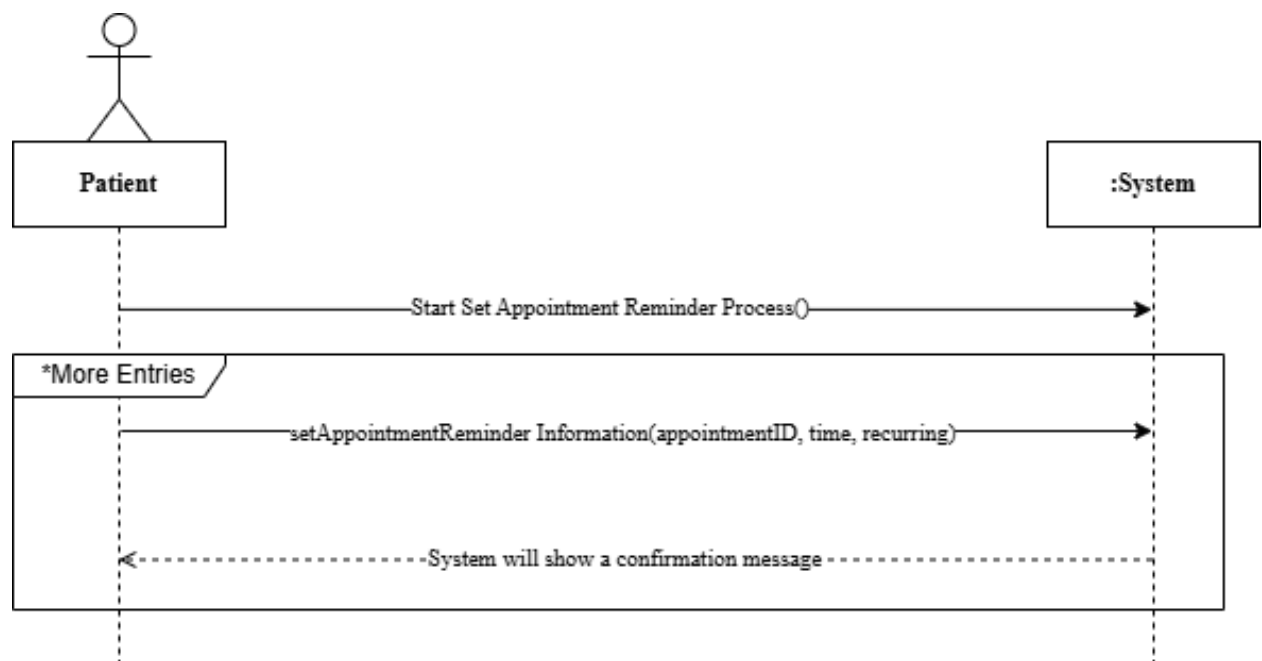
SSD 29: Update Appointment



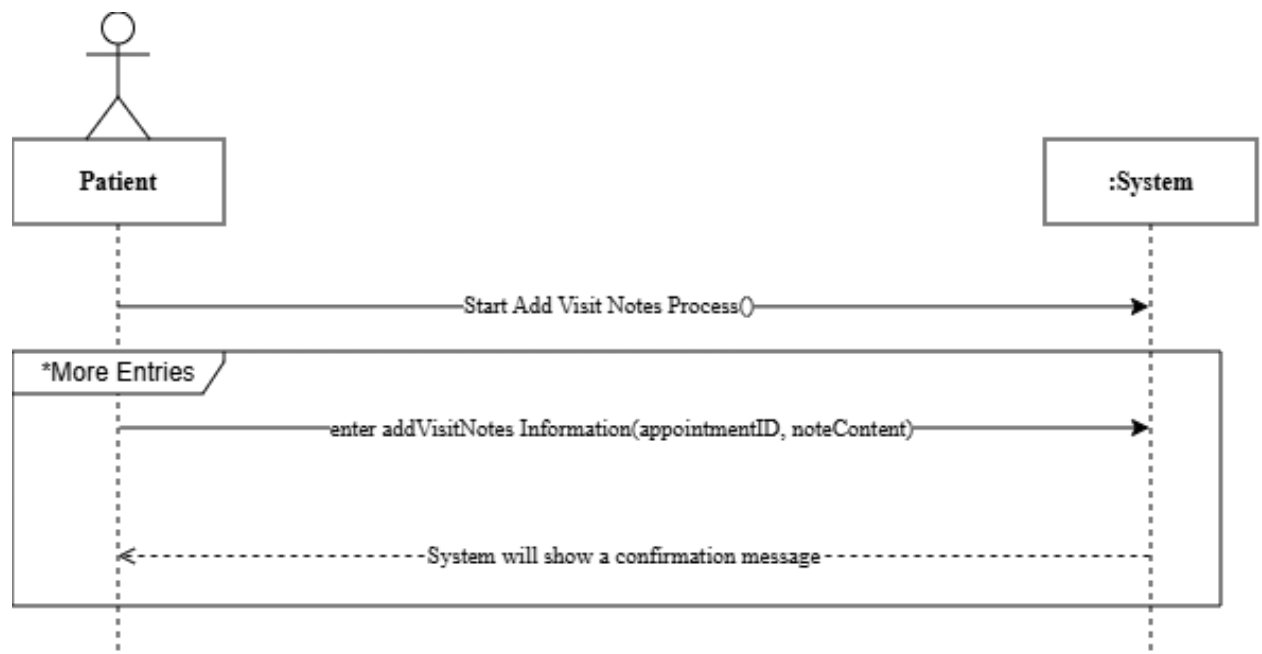
SSD 30: Manage Appointment Categories/Status



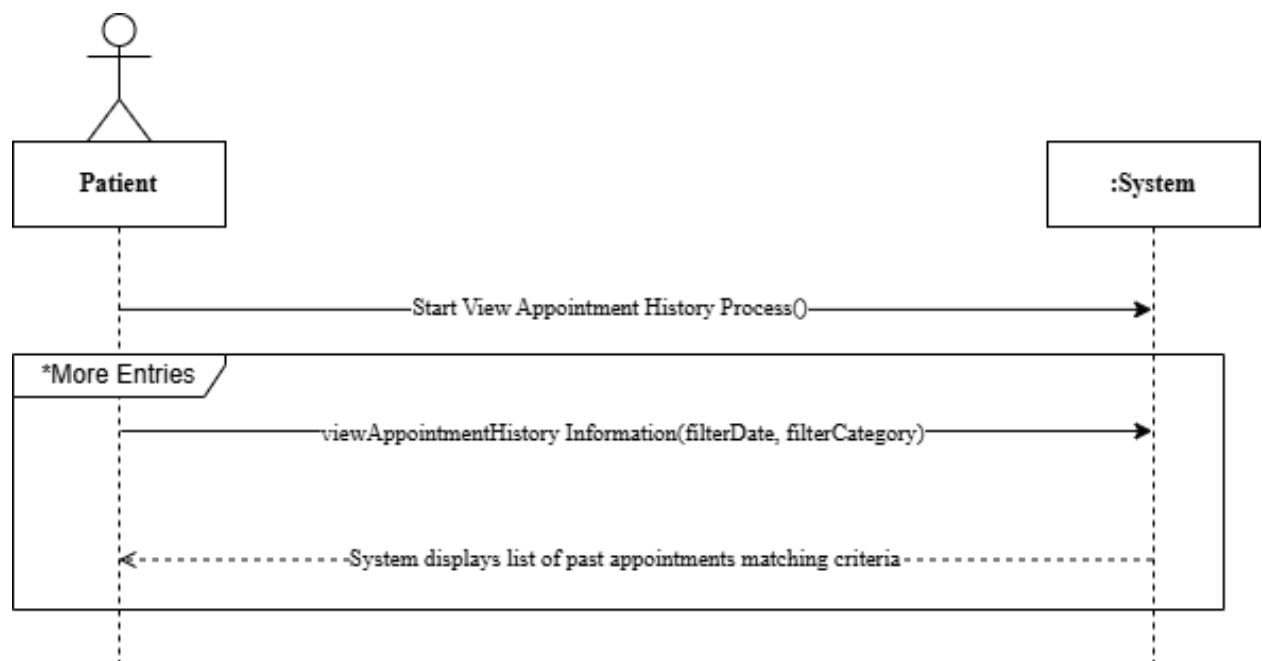
SSD 31: Set Appointment Reminder



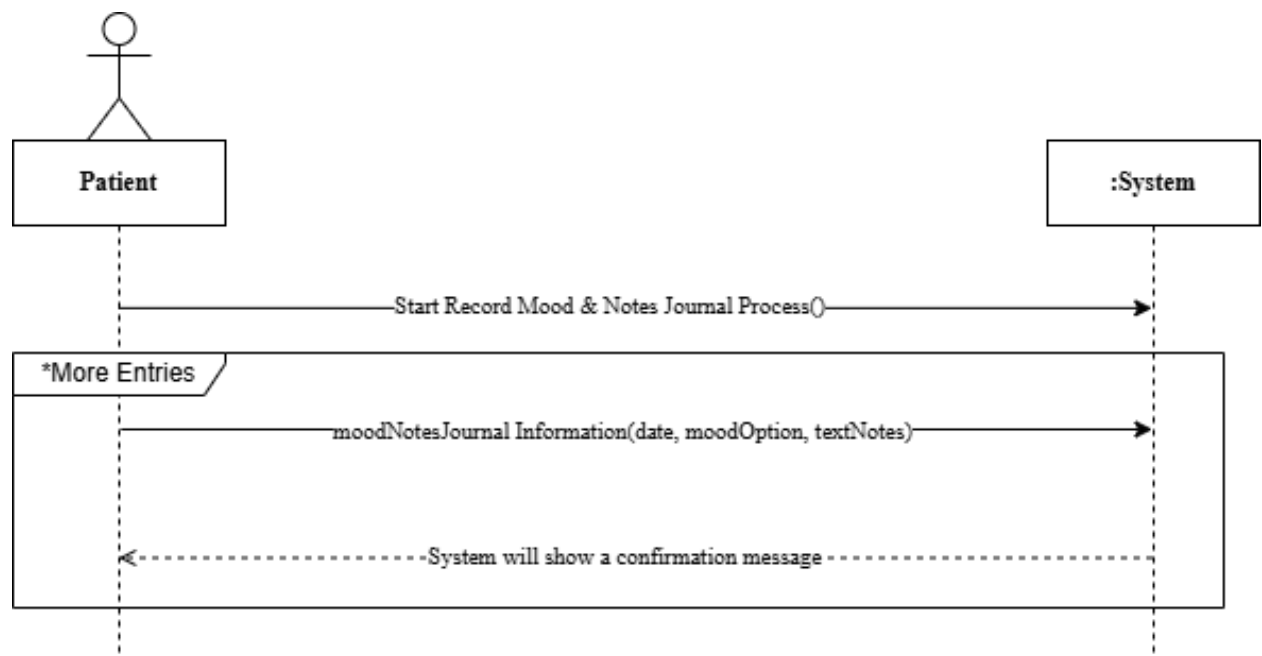
SSD 32: Add Visit Notes



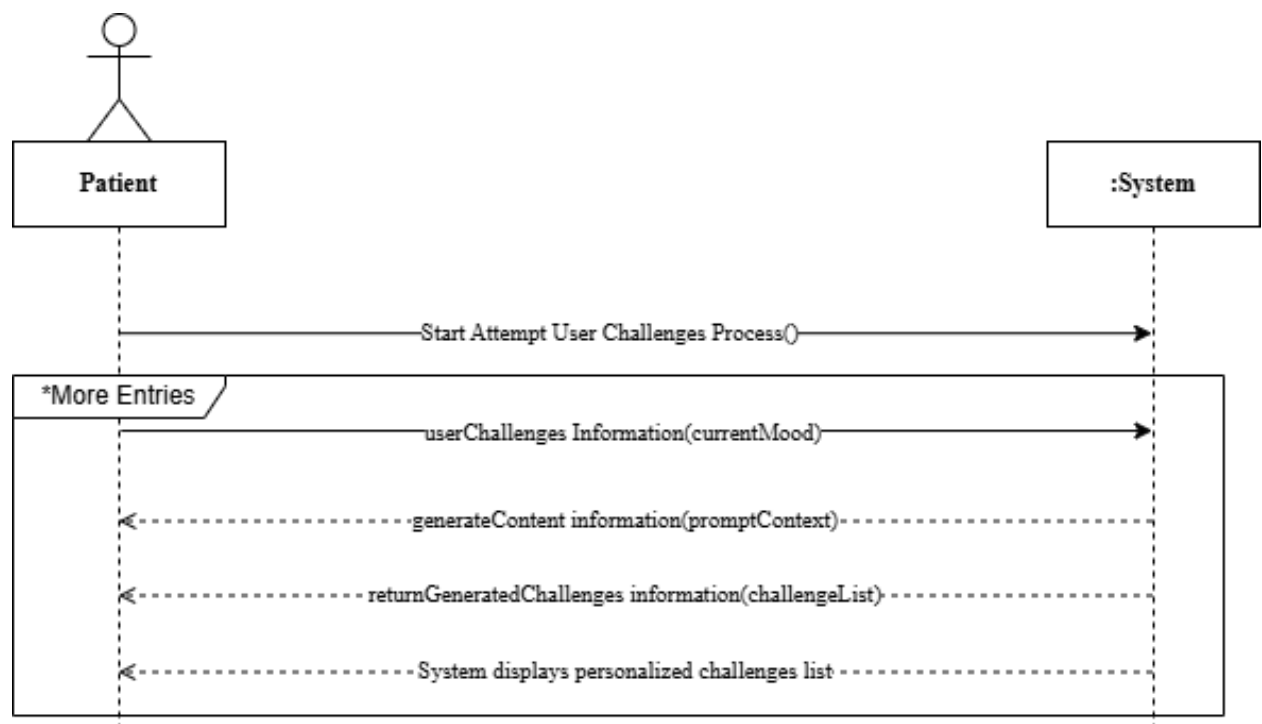
SSD 33: View Appointment History



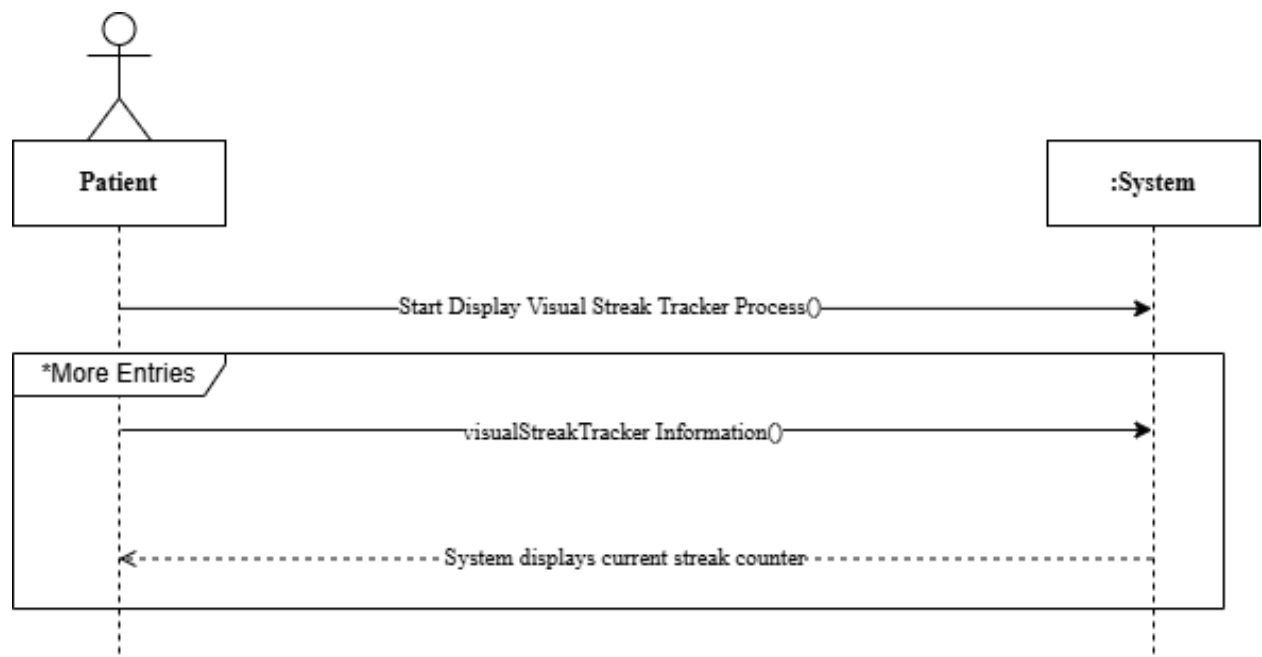
SSD 34: Record Mood & Notes Journal



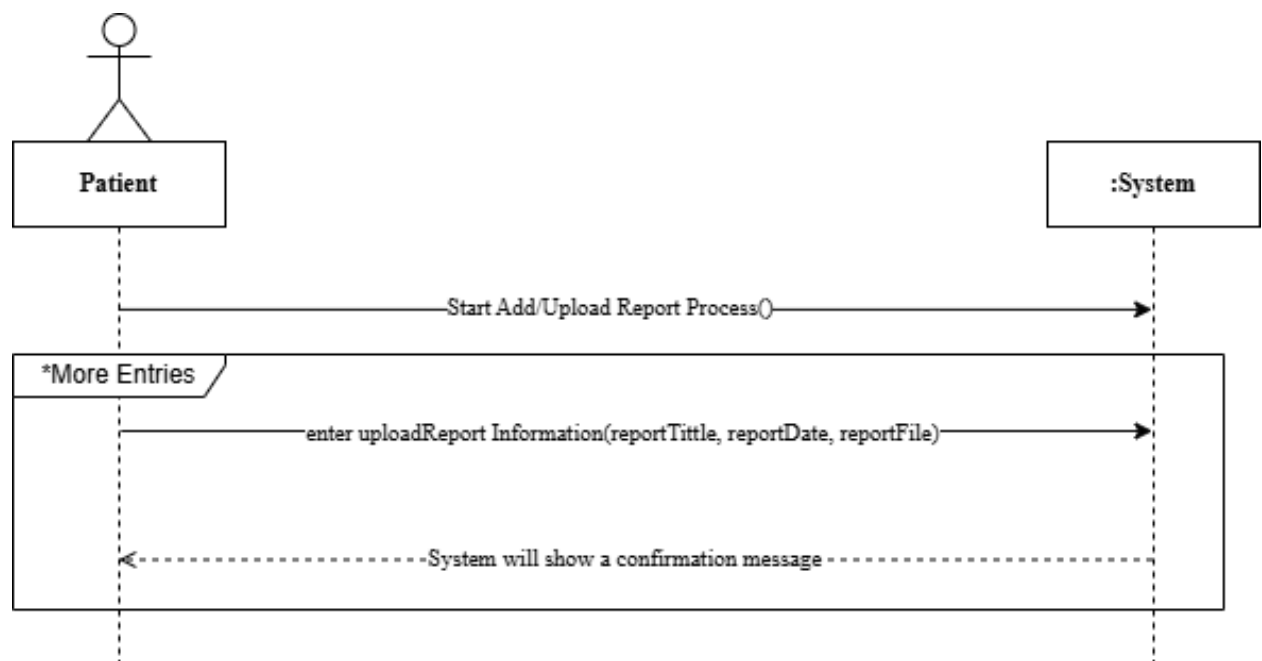
SSD 35: Attempt User Challenges



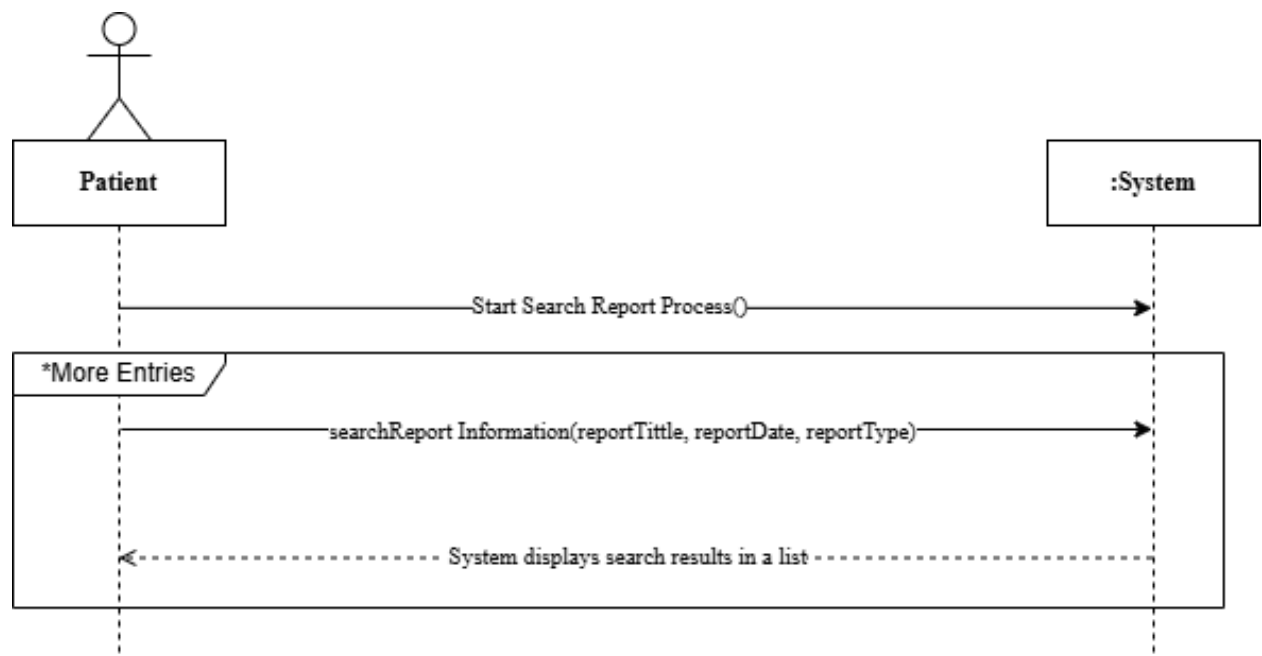
SSD 36: Display Visual Streak Tracker



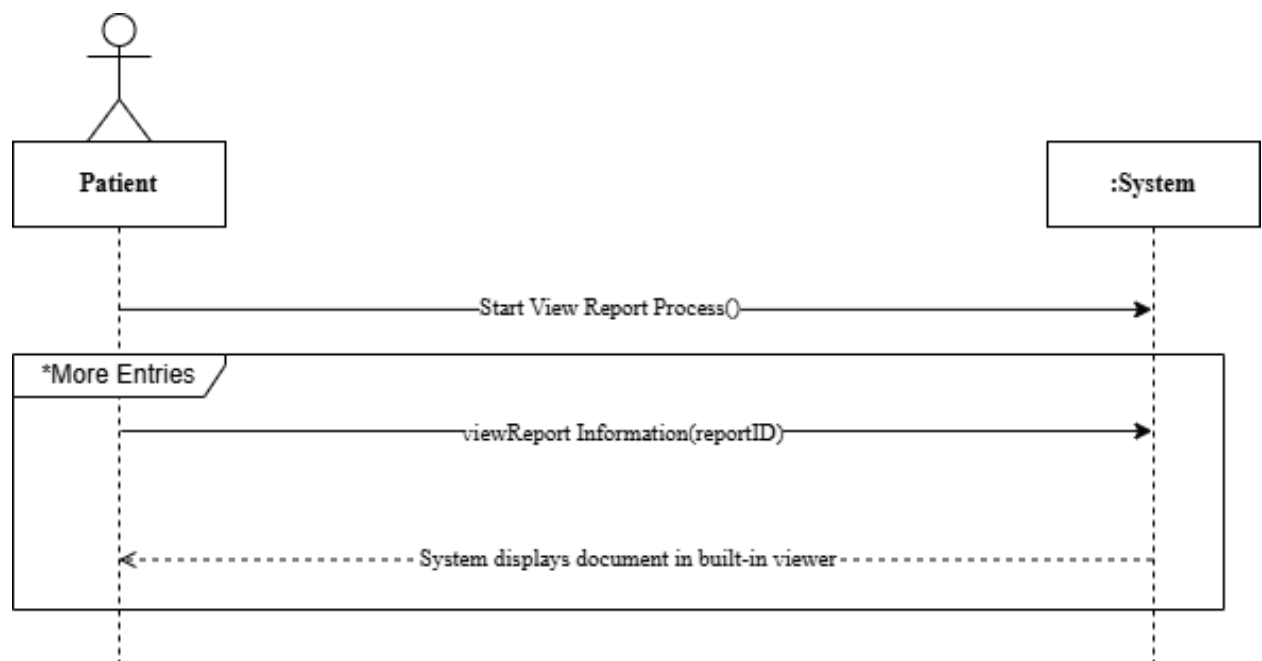
SSD 37: Add/Upload Report



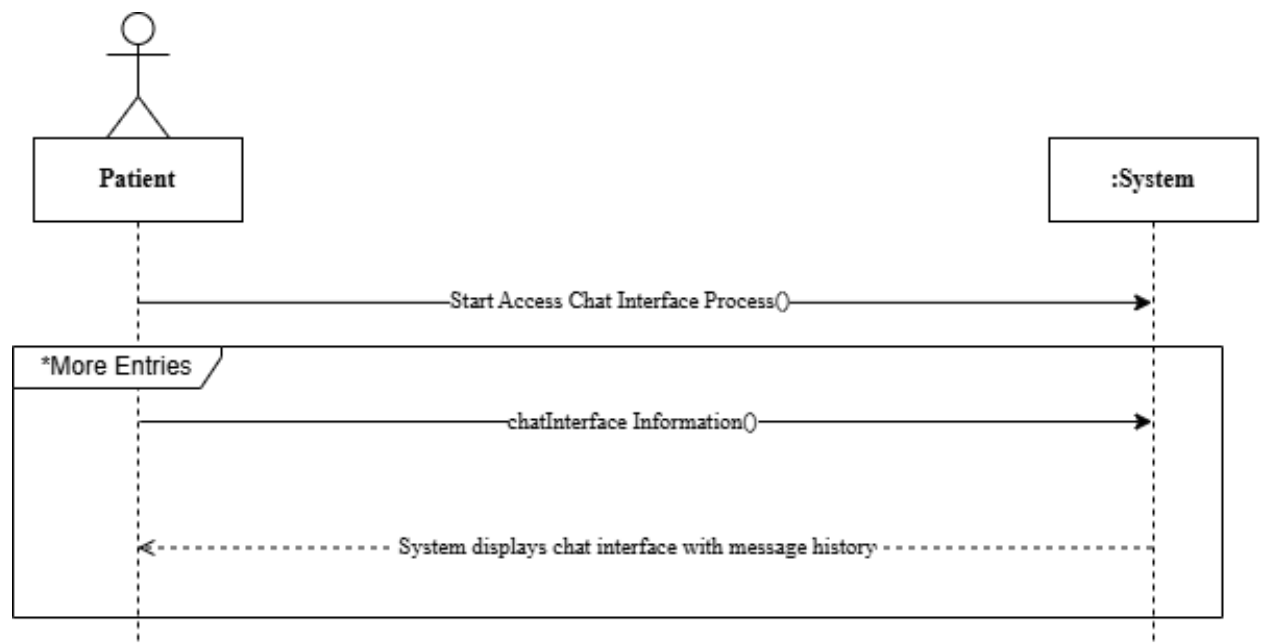
SSD 38: Search Report



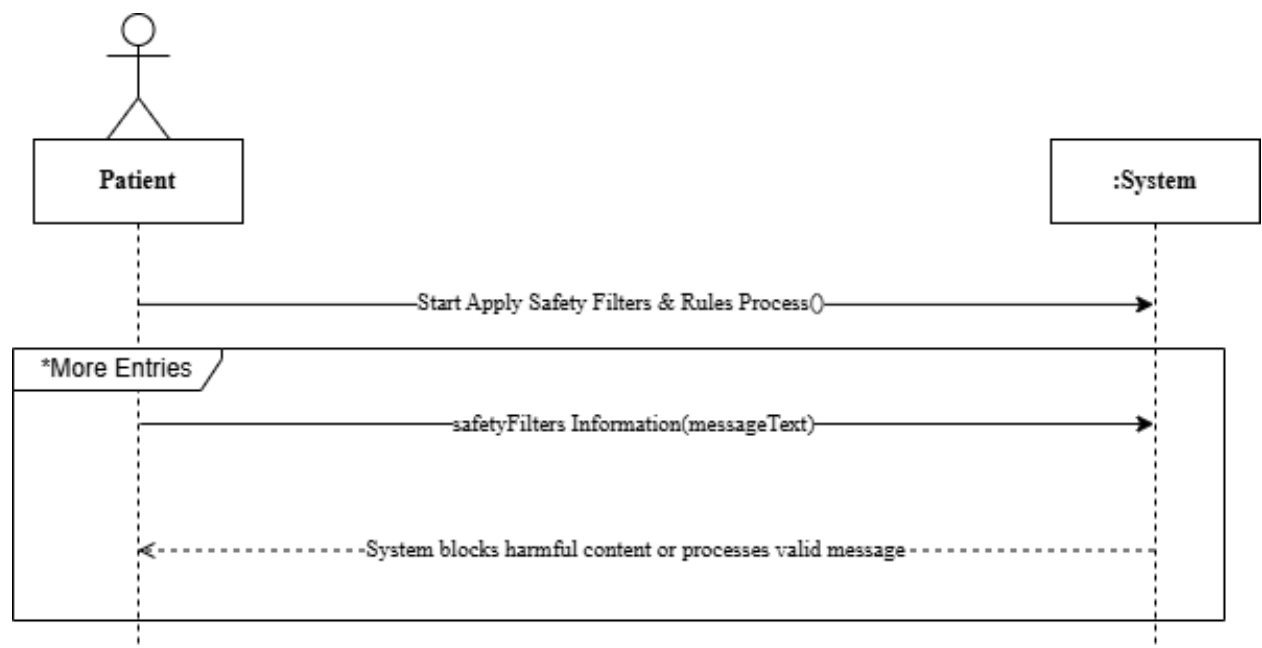
SSD 39: View Report



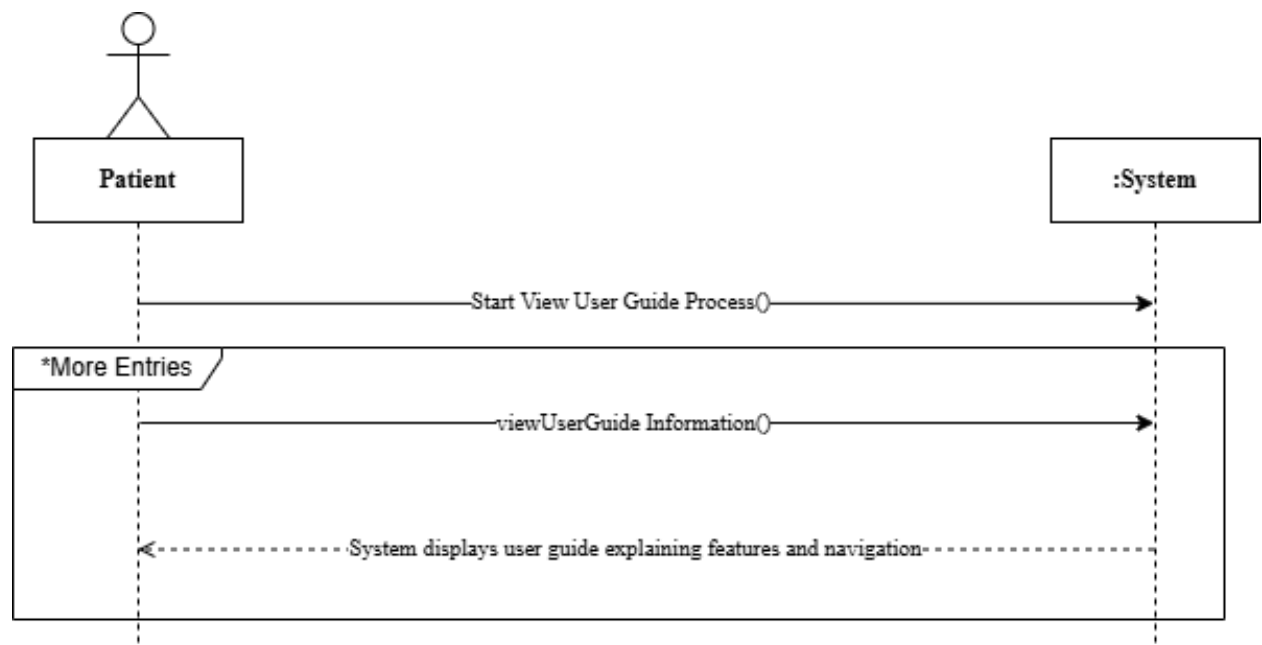
SSD 40: Access Chat Interface



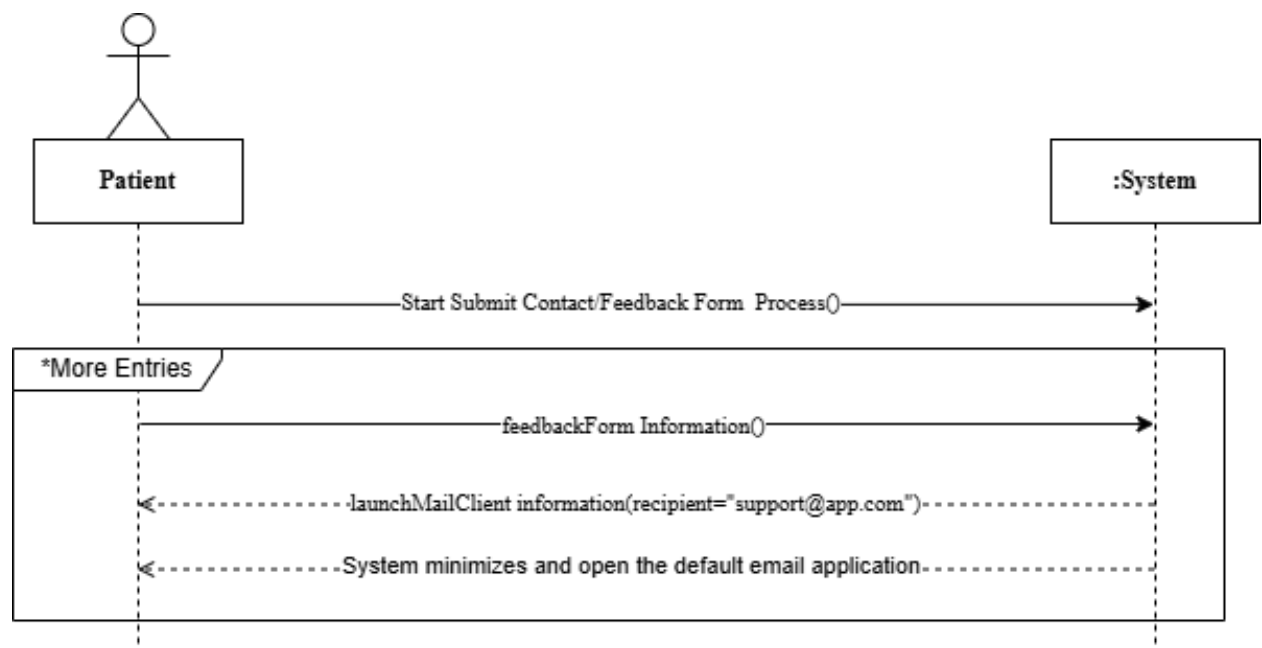
SSD 41: Apply Safety Filters & Rules



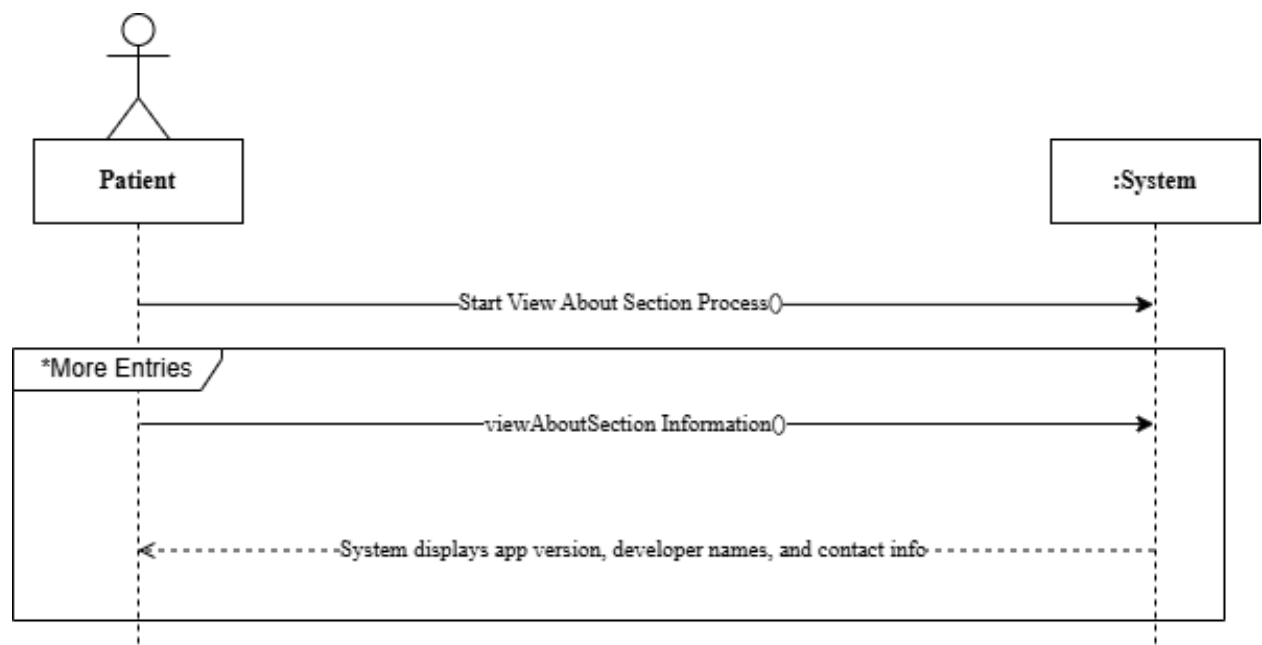
SSD 42: View User Guide



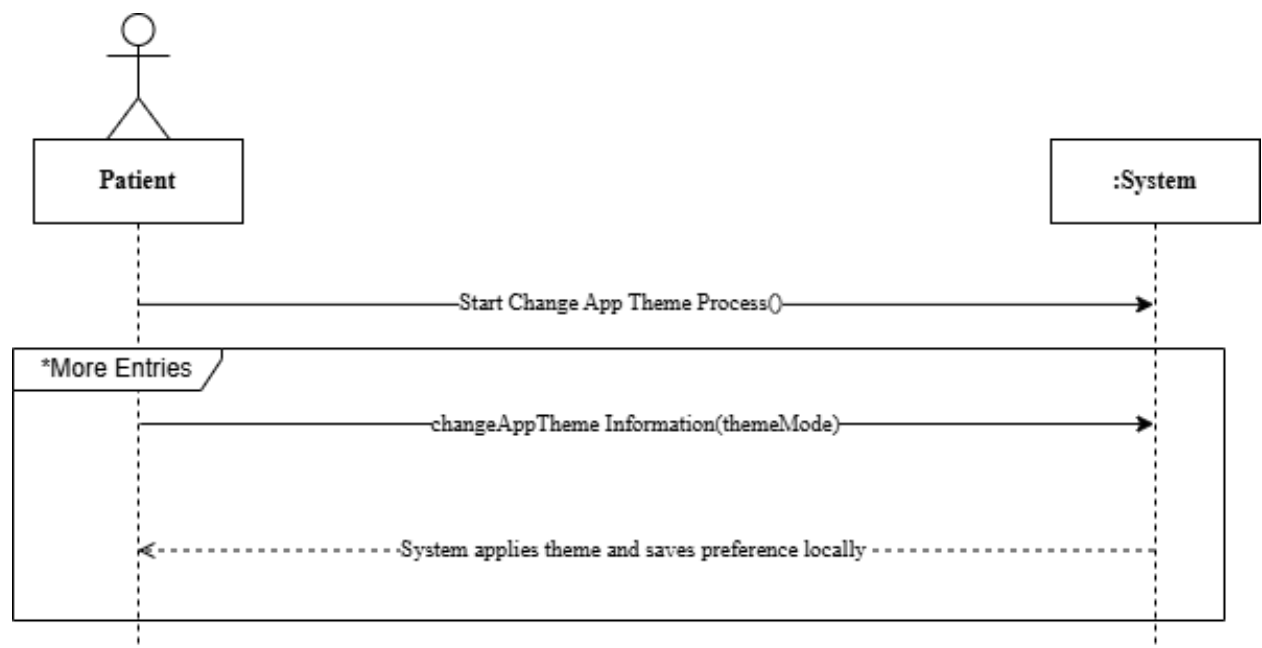
SSD 43: Submit Contact & Feedback Form



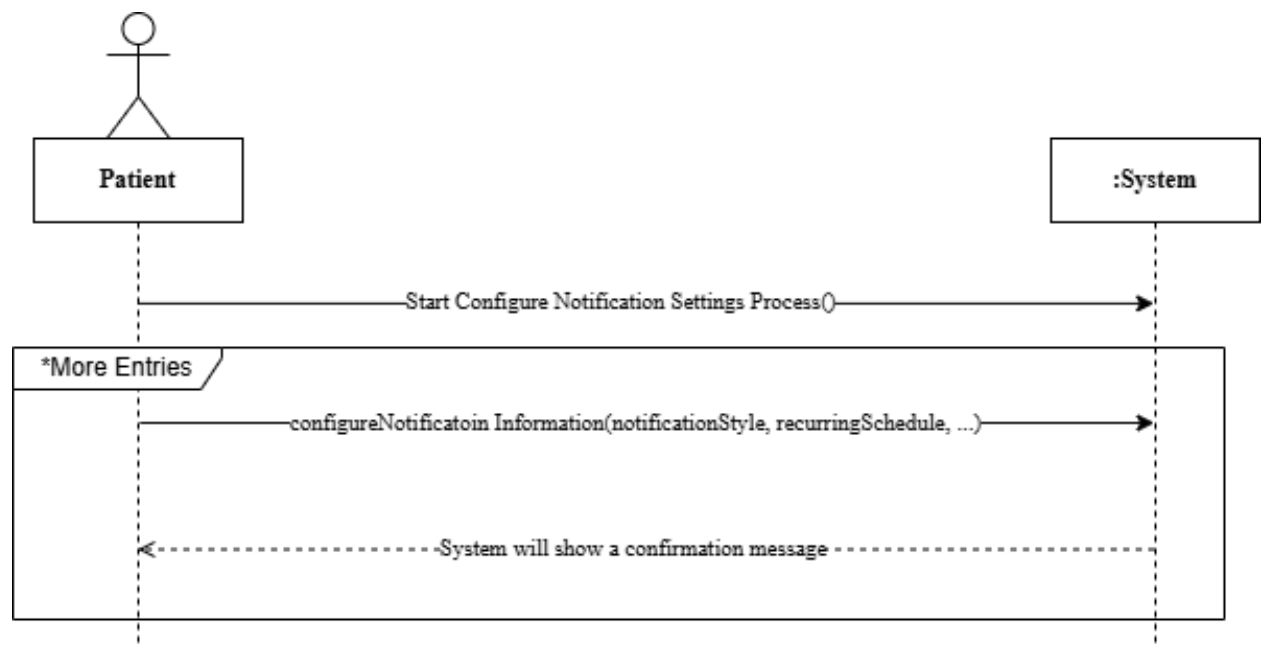
SSD 44: View About Section



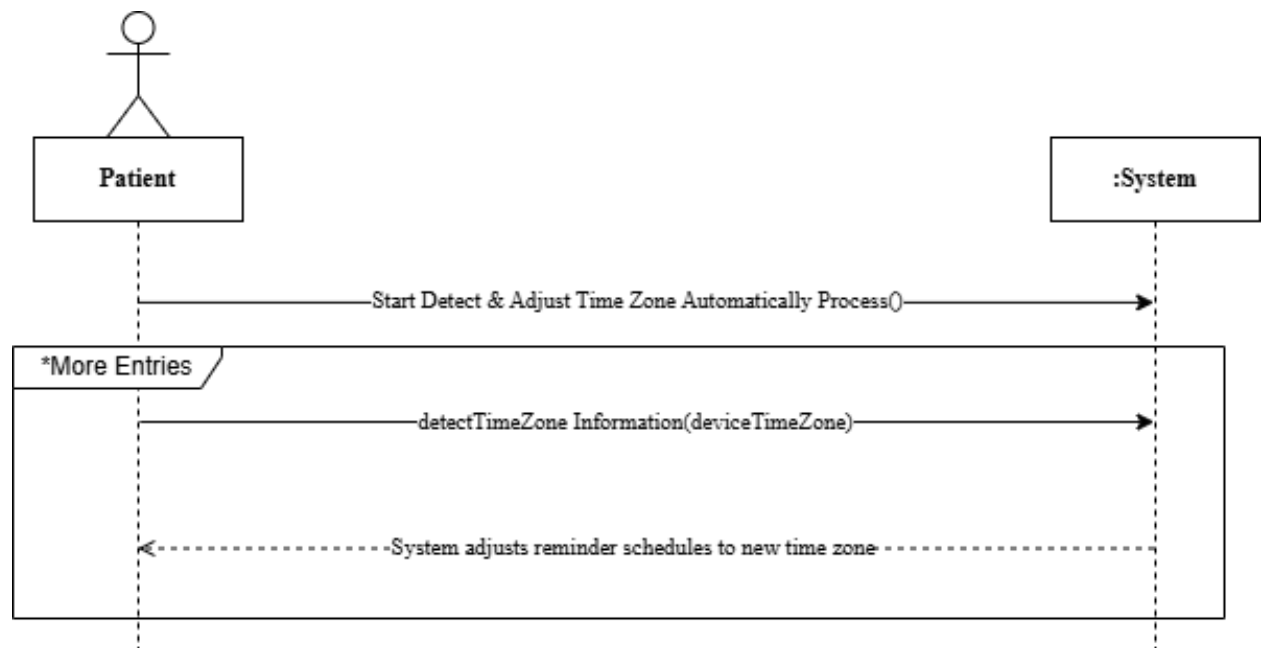
SSD 45: Change App Theme



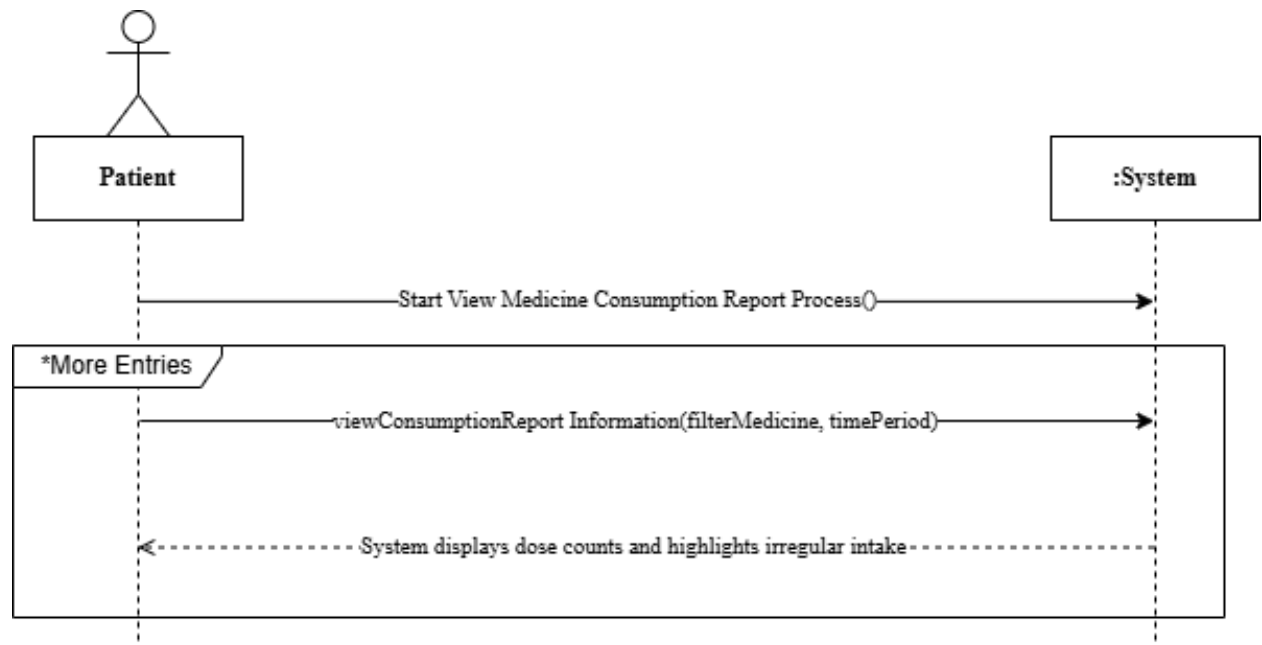
SSD 46: Configure Notification Settings



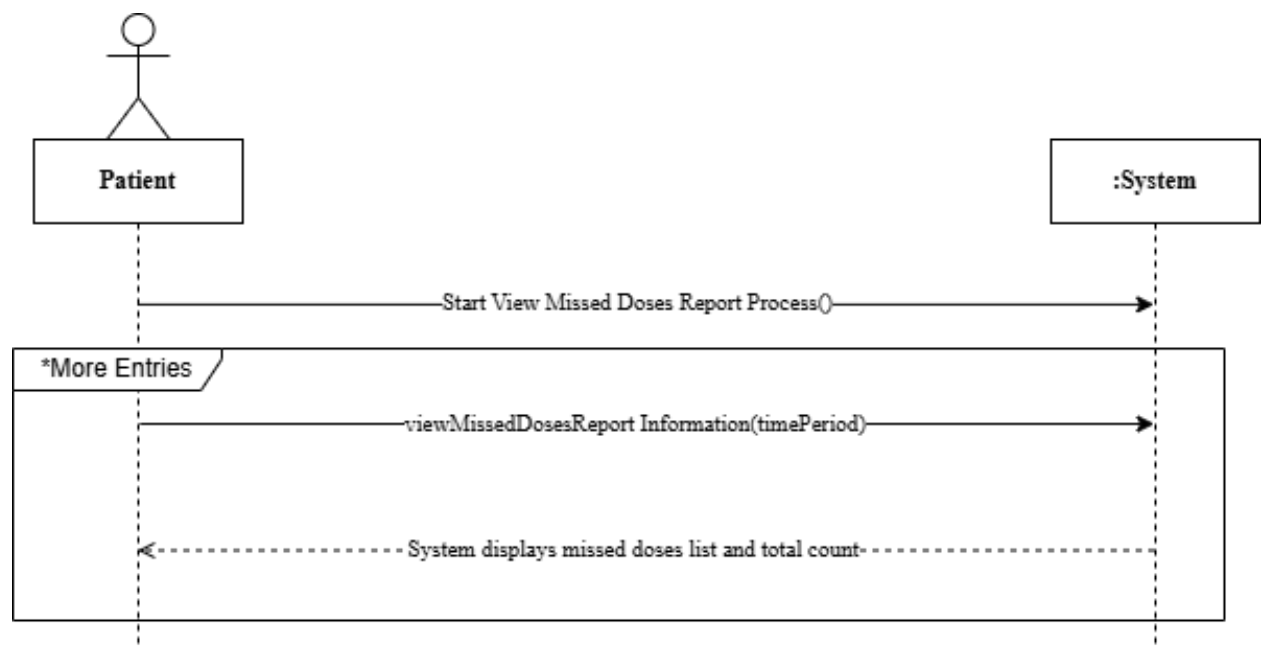
SSD 47: Detect & Adjust Time Zone Automatically



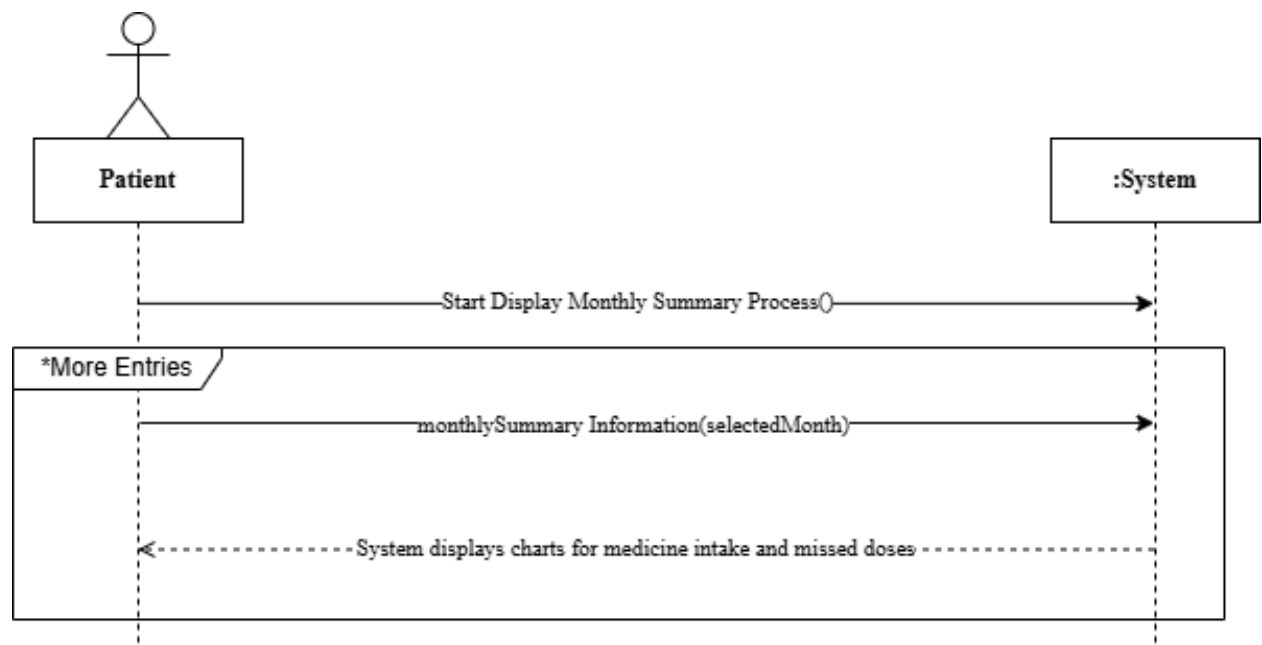
SSD 48: View Medicine Consumption Reports



SSD 49: View Missed Doses Reports



SSD 50: Display Monthly Summary



SCHEMA DIAGRAM

FOR

Personal Medicine Reminder

Version 1.0

Prepared By

Atif Islam

&

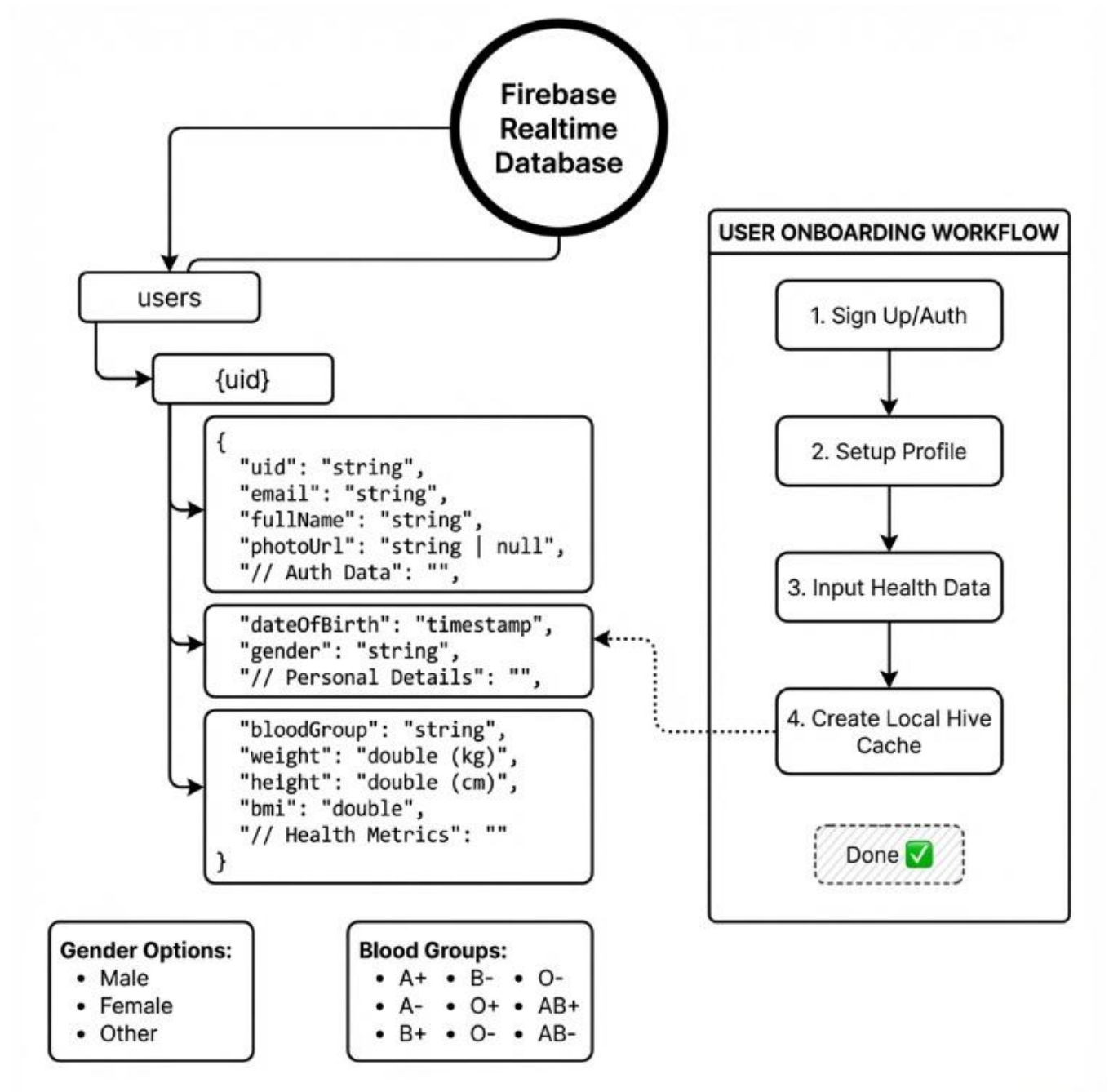
Muhammad Awais Ali

4th Jan, 2026

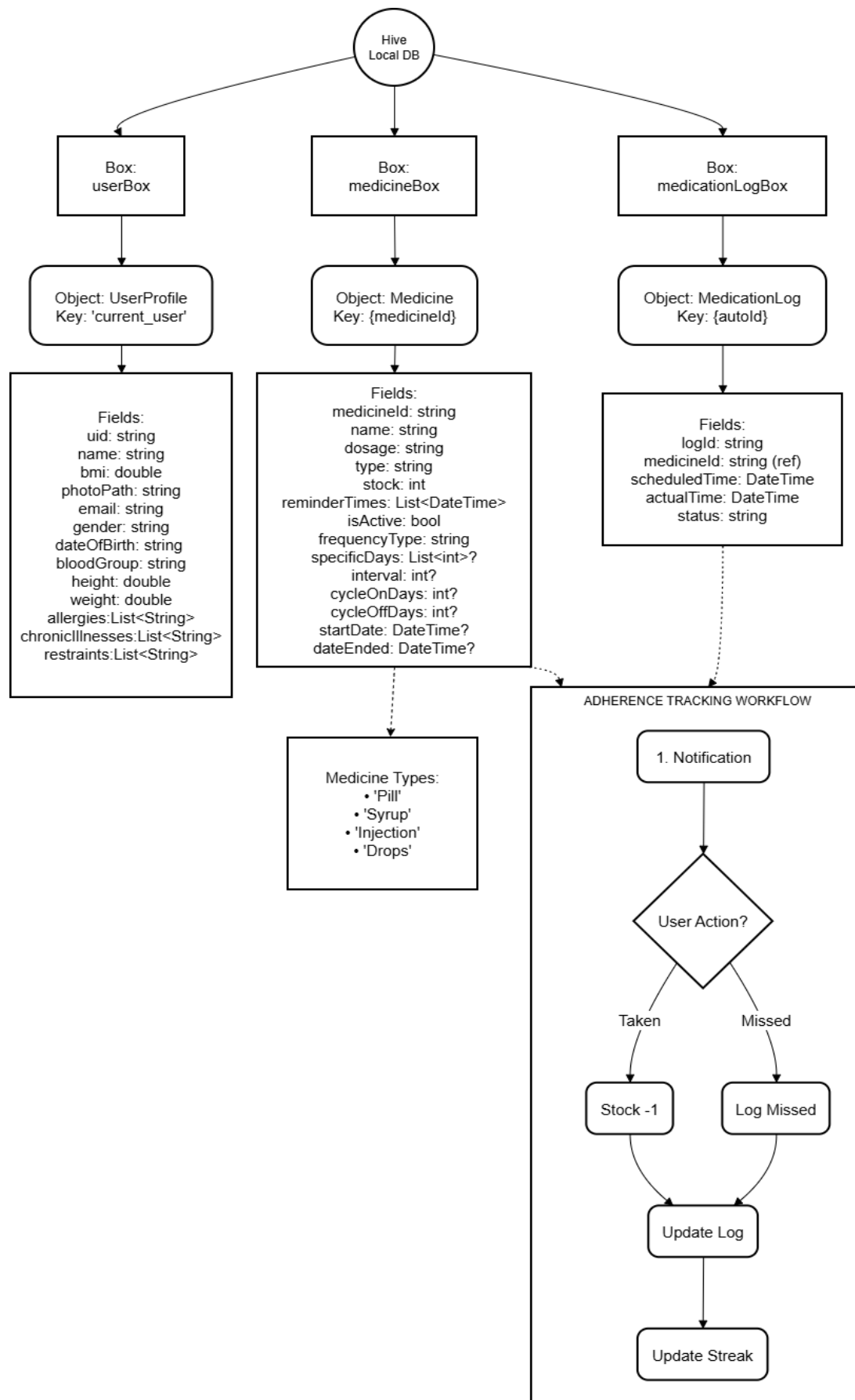
Revision History

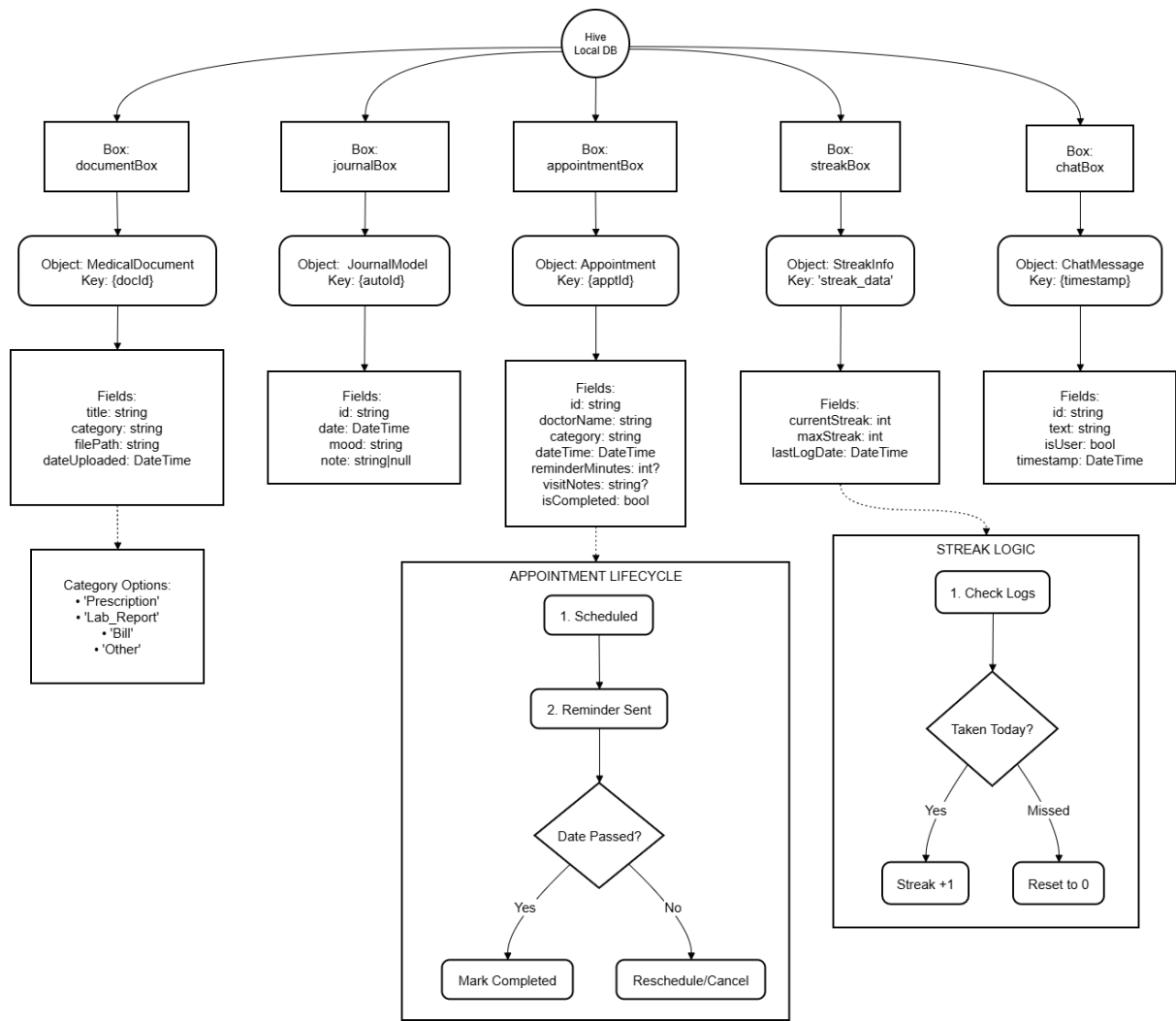
Version	Description	Author	Date
1.0	This contains the schema diagram for Personal Medicine Reminder App	Atif Islam & Muhammad Awais Ali	4 th Jan,2026

3.2 Firebase (Firestore)



3.3 Hive





DOMAIN MODEL

FOR

Personal Medicine Reminder

Version 1.0

Prepared By

Atif Islam

&

Muhammad Awais Ali

4th Jan, 2026

Revision History

Version	Description	Author	Date
1.0	This contains the domain model for Personal Medicine Reminder App	Atif Islam & Muhammad Awais Ali	4 th Jan,2026

3.4 Domain Model

